The Rise of Verbal Reinforcement Learning: A Survey

ANONYMOUS, Working draft, USA

Large language models turned natural language from a task description into a learning signal: critiques, rubrics, reflections, and debates now steer agents where scalar rewards once did. We survey this practice as verbal reinforcement learning (VRL), organizing methods by the signal itself: what linguistic artifact is consumed, which decision-problem component it modifies, and when it takes effect. A formal framework with explicit inclusion criteria separates VRL from scalarized preference learning. We synthesize credit assignment for verbal signals, the verbal reward stack, language-space optimizers, embodied settings, evaluation, and feedback-channel safety, and distill decision rules for practitioners.

ACM Reference Format:
Anonymous. 2026. The Rise of Verbal Reinforcement Learning: A Survey. 1, 1 (July 2026), 67 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

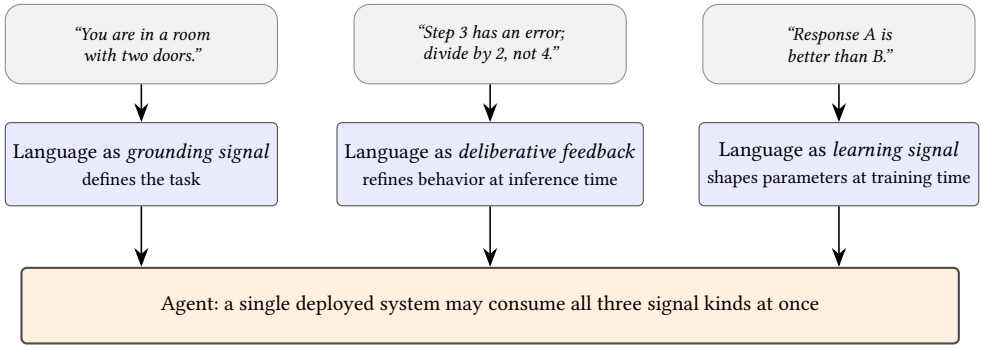

Fig. 1. Language plays three complementary roles as a signal in agentic systems: grounding the task and environment, scaffolding intermediate deliberation, and serving as a learning signal for training. The three roles classify signals, not system stages; a single deployed system may consume all three (§3).

Classical reinforcement learning couples an agent with a well-specified environment, an action space, and a task-specific reward function, and this coupling has produced landmark systems: Atari play from raw pixels [Mnih et al. 2015], championship Go [Silver et al. 2016], chip placement [Mirhoseini et al. 2021], and tokamak plasma control [Degrave et al. 2022]. Each success, however, rests on heavy domain-specific engineering: the environment must be formalized, the interaction space fixed, and the reward function must capture the intended behavior closely enough that optimizing it yields what the designer meant [Vamplew et al. 2021]. Writing such specifications is a long-standing bottleneck [Amodei et al. 2016], and it binds most tightly exactly where modern agents are deployed: tasks whose goals are ambiguous, context-dependent, and difficult to encode as a scalar objective.

Large language models [Achiam et al. 2023; Brown et al. 2020] have opened an alternative supervisory channel: natural language itself. Instead of fixing all task intent in advance, verbal feedback communicates corrections, preferences,

Author's Contact Information: Anonymous, Working draft, Minneapolis, USA.

2026. Manuscript submitted to ACM

and constraints on the fly, in a form that agents can increasingly interpret and act on. Frontier models can describe environments [Wang et al. 2023b], identify bugs in code [McAleese et al. 2024], critique generated outputs [Madaan et al. 2023], and translate stated preferences into training signals [Hong et al. 2024a; Meng et al. 2024]. Much as self-supervised learning [Chen et al. 2020; Devlin et al. 2019] widened the range of data that could supervise model development, verbal feedback is widening the range of signals that can supervise agent improvement.

We call the resulting family of methods **verbal reinforcement learning (VRL)** and define membership by two criteria. First, the feedback is expressed in natural language, whether self-generated, supplied by a human, or produced by an external tool or model, and its linguistic structure is functionally consumed by the improving agent rather than collapsed to a scalar or a preference bit before use. Second, the feedback participates in an improvement loop: it effects an output revision, a memory write, or a parameter update. Both criteria are required, and together they fence VRL off from adjacent paradigms. Plain instruction following fails the second criterion, since no improvement loop exists. Vanilla reinforcement learning from human feedback (RLHF) [Christiano et al. 2017; Ouyang et al. 2022] and direct preference optimization (DPO) [Rafailov et al. 2023] satisfy only the second, because the verbal judgment is reduced to a preference bit before learning; we treat them as the scalar endpoint of a compression spectrum rather than as VRL proper (§6). Language-conditioned reinforcement learning treats language as an input to the policy rather than as feedback on its behavior. The same criteria settle boundary cases (reward-code generation, in which language builds a reward function that later emits scalars, still qualifies) and scope the definition by modality: only the feedback need be textual, so embodied agents with visual observations remain in scope when the corrections they receive are verbal (§10). Section 2 develops the canonical, formal version of this fence within a language-augmented sequential decision framework.

The term itself has a short but crowded genealogy. Shinn et al. [2023] coined it to describe self-reflective agents that improve by storing and rereading their own verbal self-critiques, a specific instance of what we classify as deliberative feedback. Our usage is deliberately broader, covering any method that meets the two criteria above rather than only the reflective mechanism for which the term was coined; §2 states the generalization precisely. The nearest adjacent coinage, Natural Language Reinforcement Learning [Feng et al. 2024], recasts individual reinforcement-learning components such as value functions in language space; it is a framework proposal rather than a survey, and in our terms one formal instantiation of the training-stage cell. Farther back, Luketina et al. [2019] surveyed reinforcement learning informed by language at a time when language could only inform a scalar-reward learner; in the systems reviewed here that relation is inverted, with language serving simultaneously as the policy medium, the feedback, and increasingly the optimizer trace. The term also has an older life in behaviorist psychology, which §2 takes up alongside the formal genealogy.

Early results indicate that the verbal channel is more than a convenience. Ahn et al. [2022] used language to ground high-level instructions in physical affordances, enabling real-world robotic planning. Shinn et al. [2023] reached 91% pass@1 on HumanEval through verbal self-reflection alone, and Madaan et al. [2023] showed that iterative verbal critique yields gains across diverse generation tasks. Even in fully scalarized form, human judgments proved potent: Ouyang et al. [2022] trained a 1.3B-parameter model on preference judgments that outperformed the 175B GPT-3 baseline, and a recurring question in this survey is how much additional value survives when the linguistic structure of feedback is retained rather than compressed away. Nor are these isolated demonstrations: verbal-feedback methods now appear across coding assistants [Jimenez et al. 2024; Yang et al. 2024a], scientific discovery [Bran et al. 2024; Lu et al. 2024], robotics [Wang et al. 2023b], mathematical reasoning [Hatamizadeh et al. 2026; Liu et al. 2026b; Shi et al. 2025a], clinical decision support [Huang et al. 2025a; Zhou et al. 2025e], and education [Ahtisham et al. 2026; Qian et al. 2025].

These methods differ in mechanism: some operate during inference, others modify training data, and some redefine the task itself. What they share is the use of natural language to guide, refine, and ground agent behavior. We therefore organize the field along a single axis, when a verbal signal takes effect and what it modifies, yielding three pillars: language as *grounding signal*, which defines the task by specifying goals, states, actions, and reward structure; language as *deliberative feedback*, which refines behavior at inference time without parameter updates; and language as *learning signal*, which shapes parameters at training time. The pillars classify signals, not systems: a single agent may consume signals in several regimes at once, and we make no claim that methods traverse the pillars as a lifecycle loop.

This growth has been accompanied by a crowded shelf of surveys, yet none takes the feedback signal itself as its object of study. Prior overviews organize by mechanism, such as correction timing [Kamoi et al. 2024; Pan et al. 2023], test-time scaling dimensions [Zhang et al. 2025i], or rollout stage [Surana et al. 2026]; by agent component, such as memory [Zhang et al. 2025b; Zheng et al. 2025], cognitive architecture [Sumers et al. 2023], or the component under self-evolution [Fang et al. 2025b; Gao et al. 2025; Tao et al. 2024]; by algorithm family, such as DPO [Xiao et al. 2024b], RLHF [Kaufmann et al. 2023; Liu et al. 2026c], or reinforcement-learning technique [Srivastava and Aggarwal 2025; Wang et al. 2024g; Zhang et al. 2025f]; or by the producer of judgments [Gu et al. 2024; Li et al. 2024c,a; You et al. 2026]. Reward-centric treatments come closest to our concern [Ji et al. 2025; Liu et al. 2025c; Wei et al. 2025c; Wu 2025], but they presuppose that feedback must become a reward, which is precisely the design decision a signal-centric account interrogates. The nearest precedent in spirit, a formalization of human feedback for natural language generation by format, objective, and use [Fernandes et al. 2023], predates the agentic turn, so reward-code grounding, rubric rewards, and experiential memory fall outside its frame. To our knowledge, no existing survey combines a signal-centric organizing axis with a formal treatment of language as signal, coverage that runs from problem definition through inference-time deliberation to training, and a first-class treatment of the safety of the feedback channel. That combination is the gap this article fills.

This article makes four contributions.

(1) **A signal-centric formalism.** We define VRL by the two-criterion fence above and develop a language-augmented sequential decision framework that specifies where a linguistic artifact can enter the decision problem (goal, observation, action prior, reward, transition prior, or improvement operator), separating VRL from RLHF, instruction following, and language-conditioned reinforcement learning (§2). The taxonomy built on this framework assigns each signal by what it modifies at the moment it takes effect (§3).
(2) **Decision rules and practitioner guidance.** We give an explicit assignment rule for placing methods, decision rules for hybrid cases such as reward-code generation and experiential memory, and a prescriptive guide that keys the choice among verbal-feedback mechanisms to task verifiability, feedback latency and cost, and risk (§3, §13).
(3) **A credit-assignment and optimization synthesis.** We follow verbal signals into the machinery that consumes them: the verbal reward stack that runs from binary verifiers through rubrics and generative reward models to process reward models (PRMs) (§7); outcome-level, process-level, and token- or span-level credit assignment, together with the pairing of feedback forms and algorithm families (§8); and optimizers that update prompts, memory, and context in language space rather than weights (§9). Throughout, we state plainly which formal guarantees exist and which do not.
(4) **Positioning and agenda.** We compare this survey with more than a dozen prior surveys and overview papers along a fixed set of dimensions (organizing axis, feedback types, stages covered, formalism, practitioner guidance, and feedback safety), and we distill a research agenda from the gaps the comparison exposes (§13).

The remainder of the article proceeds as follows. Section 2 presents the formal framework and the full definitional fence, and Section 3 introduces the taxonomy, its assignment rule, and a worked coding-agent example. Three sections then develop the pillars: Section 4 covers language as grounding signal, including goal, state, and action grounding and reward-code generation; Section 5 covers language as deliberative feedback, including self-critique, externally grounded critique, multi-agent debate, experiential memory, and evaluator feedback within search; and Section 6 covers language as learning signal along a compression spectrum whose scalar endpoint is vanilla RLHF and DPO. Section 7 surveys the verbal reward stack, Section 8 treats credit assignment and optimization with verbal signals, and Section 9 covers language-space optimizers. Section 10 extends the account to embodied and multimodal settings, Section 11 synthesizes benchmarks for feedback quality and utilization, and Section 12 examines attacks on the feedback channel and defenses against them. Section 13 collects the practitioner guide, the comparison with prior surveys, and the research agenda; Section 14 concludes. Practitioners deciding which mechanism to adopt may read Sections 3 and 13 first; readers interested in theoretical questions may proceed from Section 2 directly to Section 8.

2 A Formal Framework for Verbal Reinforcement Learning

The methods surveyed in later sections share a decision-theoretic skeleton but attach language to it in different places. This section fixes that skeleton once. We define a language-augmented partially observable Markov decision process (POMDP), enumerate the points at which a linguistic artifact, written ℓ throughout, can enter it, and introduce an improvement operator $\mathcal{I}$ whose three realizations underpin the taxonomy of §3. We then restate the two criteria that fence verbal reinforcement learning (VRL) off from adjacent paradigms, trace the genealogy of the term, and close by stating plainly which formal guarantees exist and which do not.

2.1 The Language-Augmented Decision Process

We begin from a standard POMDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \Omega, T, R, O, \gamma)$ [Kaelbling et al. 1998]: a Markov decision process with state space $\mathcal{S}$, action space $\mathcal{A}$, transition kernel T, reward function R, and discount γ [?], made partially observable by an observation space Ω and an observation function O through which the agent perceives state only indirectly. Let $\mathcal{L}$ denote the set of finite natural-language strings. A *linguistic artifact* $\ell \in \mathcal{L}$ is any string that the system produces or consumes as a signal: a task description, a critique, a rationale, a stored lesson, or a generated fragment of reward code. The agent is a configuration $\kappa = (\theta, c, m)$ comprising model parameters θ, a working context c (the prompt and any in-context material), and a persistent memory m; the policy π_κ maps interaction histories to distributions over actions, which in language settings are themselves strings.

The construct that distinguishes VRL from ordinary language-conditioned control is the *improvement operator*

$$\mathcal{I} : (\kappa, \ell) \longmapsto \kappa', \tag{1}$$

which consumes an artifact and returns a modified agent configuration. Three minimal realizations recur throughout the literature. An *output revision* alters only c: a critique is appended to the context and the agent regenerates, as in iterative self-refinement [Madaan et al. 2023], and the effect vanishes when the episode ends. A *memory write* alters only m: a reflection or distilled lesson is stored and retrieved in later episodes [Shinn et al. 2023; Zhou et al. 2025a], so the effect persists without touching the weights. A *parameter update* alters only θ: the artifact, or a signal derived from it, changes the model itself and therefore all future behavior. Composite operators exist; Constitutional AI first revises outputs against written principles and then trains on the revisions, chaining the first and third realizations [Bai et al. 2022].

Language can enter this augmented process at six points.

(i) *Goal specification.* A task description $g \in \mathcal{L}$ defines what counts as success, often without any executable reward; the true objective is latent and reaches the agent only through language, the setting formalized as learning from language feedback in §2.4.

(ii) *State and observation.* Observations may be linguistic (text environments, tool outputs, error traces) or verbalized descriptions of non-linguistic state; language then acts as a state abstraction whose fidelity bounds what the policy can distinguish (§4).

(iii) *Action prior.* Instructions and subgoals reshape the distribution over actions before any learning occurs; in hierarchical designs an LLM planner emits subgoals in $\mathcal{L}$ that a low-level actor must realize [He et al. 2024].

(iv) *Reward.* Timing is essential here. At *specification time*, a generator compiles ℓ into an executable reward function $\hat{R} = C(\ell)$, as in reward-code generation [Ma et al. 2024a]: the linguistic structure is consumed by the compiler even though the resulting function emits scalars. At *runtime*, a judge model maps trajectories to evaluations, verbal or scalar, on the fly [Zheng et al. 2023]. §7 develops this stack in full.

(v) *Transition prior.* Pretrained linguistic knowledge supplies an implicit world model: predictions about the consequences of an action can be elicited in language before, or instead of, environment interaction [He et al. 2024].

(vi) *The improvement operator.* Critiques, reflections, and preference rationales serve as the argument ℓ in Eq. (1); this entry point hosts the second and third pillars of the taxonomy.

The taxonomy of §3 classifies a signal by which of these points it acts on and, for the last, by which realization of $\mathcal{I}$ consumes it. Signals that instantiate the goal, observation, action-prior, or reward components of $\mathcal{M}$, points (i) through (iv), act at problem-definition time and constitute the grounding signal of §4; artifacts consumed by output revision or memory write at inference time constitute deliberative feedback (§5); artifacts distilled into parameter updates constitute the learning signal (§6). Point (v) is the exception, and we state it plainly: a transition prior is not written down when the problem is defined but elicited from the pretrained model during interaction, and no method we survey consumes a linguistic artifact at this point. The taxonomy therefore operationalizes five of the six entry points; the transition prior enters the survey only through the theoretical analysis of §2.4 and stands as an entry point the literature has yet to occupy. Two caveats keep the formalism honest. First, the framework is an organizing lens, not a claim that every method admits a POMDP formulation, and the boundary is concrete. Open-ended web and tool-use tasks specify no transition kernel and no reward function; success is adjudicated post hoc by a judge or a human, so T and R are notational conveniences rather than defined objects. Dialogue sets the agent against an interlocutor whose state admits no compact Markovian summary. And once memory writes alter m, later episodes unfold in an environment the agent itself has changed, breaking stationarity (§8). In these regimes we use the formalism as vocabulary and do not import its theorems. Second, multi-turn language interaction typically carries a two-level structure, an utterance-level decision process layered over a token-level one, and both benchmark design and algorithm design increasingly make this decomposition explicit [Abdulhai et al. 2023; Zhou et al. 2024c].

2.2 The Definitional Fence

With this notation, the definition of §1 can be restated precisely. A method belongs to VRL if and only if (i) the feedback is expressed in natural language, whether self-generated, provided by a human, or produced by an external tool or model, and its linguistic structure is functionally consumed by the agent rather than being collapsed to a scalar or

preference bit before use; and (ii) the feedback participates in an improvement loop, that is, an output revision, a memory write, or a parameter update.

Criterion (i) can be phrased through a scalarization map $\sigma : \mathcal{L} \to \mathbb{R}^k$: if replacing ℓ by $\sigma(\ell)$ everywhere leaves the method's behavior unchanged, the linguistic structure was not functionally consumed and the criterion fails. Two terms in this test need pinning down. *Unchanged* means distributional, not token-level, invariance: the distribution over updated configurations κ', and hence over subsequent behavior on the method's own task distribution, is the same under ℓ and $\sigma(\ell)$. *Small* k means that k is fixed by the method rather than growing with the content of ℓ, and it is counted per judged unit: a preference bit, a grade, a fixed vector of rubric scores, or one score per reasoning step is a scalarization, however many steps the judged trajectory contains; retaining the text is not. Criterion (ii) requires that ℓ appear as the argument of $\mathcal{I}$, or construct a component of $\mathcal{M}$ that subsequently drives $\mathcal{I}$, as a compiled reward function does.

The test is still a counterfactual, and §2.4 concedes that no measured quantity certifies it, so we state the inspection procedure used to adjudicate membership throughout this survey. For each method we trace the artifact from its producer to the operator that consumes it and mark the *scalarization boundary*, the point at which language collapses into the numbers the learner receives; §7 develops this construct in full. What crosses the interface to $\mathcal{I}$ decides the case. Criterion (i) holds when some consumer downstream of the artifact reads its structure: a compiler that parses ℓ into reward code [Ma et al. 2024a], a reviser that conditions on a critique [Madaan et al. 2023], a retriever that matches stored lessons [Shinn et al. 2023], or a training loss computed over the text itself. It fails when the only consumer is a judge or annotator whose output is the scalar. The intermediate formats that dominate practice resolve by the same trace. A rationale-annotated preference qualifies when the rationale enters the training distribution or a revision loop, and does not when only the bit survives. A step-level critique qualifies when downstream machinery converts its content into credit, as when critique text is converted into token-level credits [Xie et al. 2025b], and reduces to process supervision when only per-step scores survive [Lightman et al. 2024]. A generative process reward model reasons in language before scoring [Zhao et al. 2026]; its boundary sits at verdict emission, so the linguistic structure is consumed inside the reward computation while the optimizer receives per-step scalars, the same route by which compiled rewards enter, unless the critique is separately routed into revision or critique-level supervision [Wang et al. 2026c], which moves the boundary later. The trace reduces the counterfactual to a question about dataflow; the residual judgment is empirical, namely whether a consumer positioned to read the structure in fact exploits it, and only the missing certificate of §2.4 would replace that judgment with a measurement. §6 arranges learning-signal methods along the resulting compression spectrum, from methods that keep ℓ verbatim in the training distribution to methods that retain only $\sigma(\ell)$.

Both criteria are required, and each excludes a neighboring paradigm. Plain instruction following fails criterion (ii): the instruction conditions the policy, but nothing is revised, written, or updated, so no improvement loop exists. Vanilla RLHF [Ouyang et al. 2022] and DPO [Rafailov et al. 2023] fail criterion (i): the annotator's verbal judgment is collapsed to a preference bit before it reaches the update, and the content that would make the signal verbal, the reason one response beats another, is discarded; we treat these methods only as the scalar endpoint of the compression spectrum. Language-conditioned RL [Luketina et al. 2019] treats language as an input to the policy rather than as feedback on its behavior; improvement is driven by an ordinary scalar reward, so language never enters the loop of criterion (ii). One boundary case the criteria settle explicitly: language consumed at specification time to construct a reward function satisfies criterion (i), because the generator exploits the full structure of ℓ, and criterion (ii) through the training loop the constructed reward drives [Ma et al. 2024a]. Finally, the criteria fix the modality scope: the feedback

signal is natural-language text, while observations may be non-textual, as in embodied settings where verbal corrections accompany visual state (§10).

2.3 Genealogy and Neighboring Formalisms

The phrase predates machine learning: in mid-twentieth-century behavioral psychology, verbal reinforcement denoted approving utterances an experimenter used to shape a subject's responses [Greenspoon 1955; Krasner 1958]. In the LLM era the term was introduced by Shinn et al. [2023], whose subtitle, "Language Agents with Verbal Reinforcement Learning," named a specific mechanism: agents reflect verbally on task feedback and keep those reflections in an episodic buffer that guides later attempts, an instance of $\mathcal{I}$ realized as a memory write. We generalize the term deliberately to any method satisfying the two criteria above, flagging the departure from the original, narrower sense.

The closest formal ancestor of our framework is Natural Language Reinforcement Learning [Feng et al. 2024], which extends the core objects of RL into language counterparts: the language value function replaces a scalar estimate with a narrative articulating the rationale behind an evaluation, and Bellman backups and policy iteration are redefined analogously, with LLMs implementing the resulting operators. Its authors argue that scalar value restricts an agent's understanding of its environment and report gains in effectiveness and interpretability on multi-step agentic tasks. In our notation, NLRL relocates the codomain of the value function from $\mathbb{R}$ to $\mathcal{L}$; the contribution is a framework, not an overview of the field, and the language-space optimizers of §9 build directly on this move.

A second neighbor replaces reward maximization with conditional generation: the feedback-conditional policy [Luo et al. 2025] learns the posterior over responses given feedback and bootstraps online under positive feedback conditions. Its motivating argument, that scalarizing nuanced feedback discards richness, is the probabilistic form of criterion (i): the feedback string survives into the training objective itself.

A third neighbor makes memory the learned object: Memento [Zhou et al. 2025a] casts continual adaptation as a memory-augmented Markov decision process (M-MDP) in which the policy improves by rewriting and retrieving episodic cases, with the underlying LLM never fine-tuned. The M-MDP makes precise the sense in which memory is persistent state rather than inference-time scaffolding, the boundary question experiential memory poses for the taxonomy (§3); the credit-assignment problem that memory writes create [Yan et al. 2026] returns in §8.

Two earlier organizing efforts situate ours. Language grounding in RL predates LLMs [Branavan et al. 2009], and Luketina et al. [2019] surveyed that first wave, dividing it into language-conditional RL, where language is part of the problem, and language-assisted RL, where language aids learning. Our framework is the LLM-era successor to that division: language is no longer only an input or an auxiliary knowledge source but the medium of evaluation and revision, and the policy itself is a language model, so every entry point of §2.1 is realizable by a single substrate. CoALA [Sumers et al. 2023] instead organizes agent computation into memory modules, action spaces, and decision cycles. The two lenses are complementary: CoALA says where computation and storage live inside an agent, whereas we classify the signals that flow through it, and a single CoALA-style architecture can host grounding, deliberative, and learning signals simultaneously (§3).

2.4 What Is Formally Guaranteed, and What Is Not

Formal results exist, but they are recent and localized. Xu et al. [2025c] give the first principled formalization of the learning-from-language-feedback (LLF) problem: sufficient assumptions under which learning is possible even though rewards are latent, a complexity measure, the transfer eluder dimension, that quantifies how informative feedback is, and a no-regret algorithm, HELiX, whose guarantees scale with that dimension. Their separation result is the theoretical

warrant for the field's premise: learning from rich language feedback can be exponentially faster than learning from reward alone. Complementing this, He et al. [2024] analyze a hierarchical planner and actor system inside a POMDP and prove that, under assumptions on the pretraining distribution, an LLM planner performs Bayesian aggregated imitation learning through in-context learning; naive execution of the imitated subgoals incurs linear regret, and epsilon-greedy exploration on top of the imitator restores sublinear regret when the pretraining error is small. The lesson generalizes: language-mediated hierarchies do not escape the exploration problem. At the level of problem statements rather than theorems, LLF-Bench operationalizes the setting, replacing numeric reward with natural-language instructions and feedback and randomizing verbalizations so that agents must genuinely learn from the feedback channel [Cheng et al. 2023].

The list of what is missing is longer. There is no convergence or contraction analysis for language-space Bellman operators, and to our knowledge no analogue of the policy improvement theorem exists for $\mathcal{I}$ in any of its realizations: nothing certifies that a revision, a memory write, or a critique-driven update makes the policy better rather than differently biased. Existing regret results assume that feedback carries reliable information about a latent ground-truth objective; no bound covers self-generated feedback that shares the biases of the policy it evaluates, the regime in which most deliberative methods operate (§12). The separation of Xu et al. [2025c] contrasts the ends of the compression spectrum, rich feedback against scalar reward; no complexity characterization exists for the intermediate formats that dominate practice, step-level critiques or rationale-annotated preferences. Criterion (i) itself lacks a certificate: no measurable quantity attests how much of an artifact's linguistic structure a learner actually exploits, so the fence must be applied by the boundary trace of §2.2, which settles what a mechanism could read but not what it in fact uses. And no guarantee yet addresses noisy, adversarial, or strategically generated feedback channels. These gaps recur as concrete agenda items in §13.

The vocabulary of this section, ℓ for the linguistic artifact, κ for the agent configuration, $\mathcal{I}$ for the improvement operator, and σ for scalarization, does most of its work here, in the entry points, the fence, and the guarantees above; the later chapters inherit the distinctions rather than the symbols. The taxonomy of §3 rests on the mapping of §2.1: entry points (i) through (iv) become its four grounding cells, the three realizations of $\mathcal{I}$ separate deliberative feedback from the learning signal, and the transition prior stands as the unoccupied sixth coordinate. The notation returns where an argument needs it, as $\sigma(\ell)$ does in the fence adjudication of §6 and the credit-assignment analysis of §8; elsewhere the pillar chapters carry these distinctions in prose.

3 Taxonomy

The framework of §2 identifies the sites at which a linguistic signal can enter an agent's operation. This section turns those sites into a taxonomy. We organize the verbal reinforcement learning (VRL) literature along a single axis: *when* a linguistic signal takes effect and, consequently, *what* it modifies. The axis yields three pillars, summarized in Table 1 and illustrated in Figure 2. Three commitments guard against misreadings that this axis invites. First, the pillars classify signals, not systems: a deployed agent typically consumes signals from several pillars at once, and we tag such methods rather than force them into a single cell (§3.1). Second, the axis is not a lifecycle: we make no claim that any method traverses a loop from grounding through deliberation to learning, and the worked example in §3.3 shows coexistence of signal kinds rather than succession of stages. Third, the tree inherits the definitional fence of §2.2: its cells collect fence-passing signals, and where a cell borders an excluded neighbor, plain instruction following beside the grounding cells and vanilla preference optimization beside the learning cells, the pillar discussion below states the criterion that separates member from neighbor.

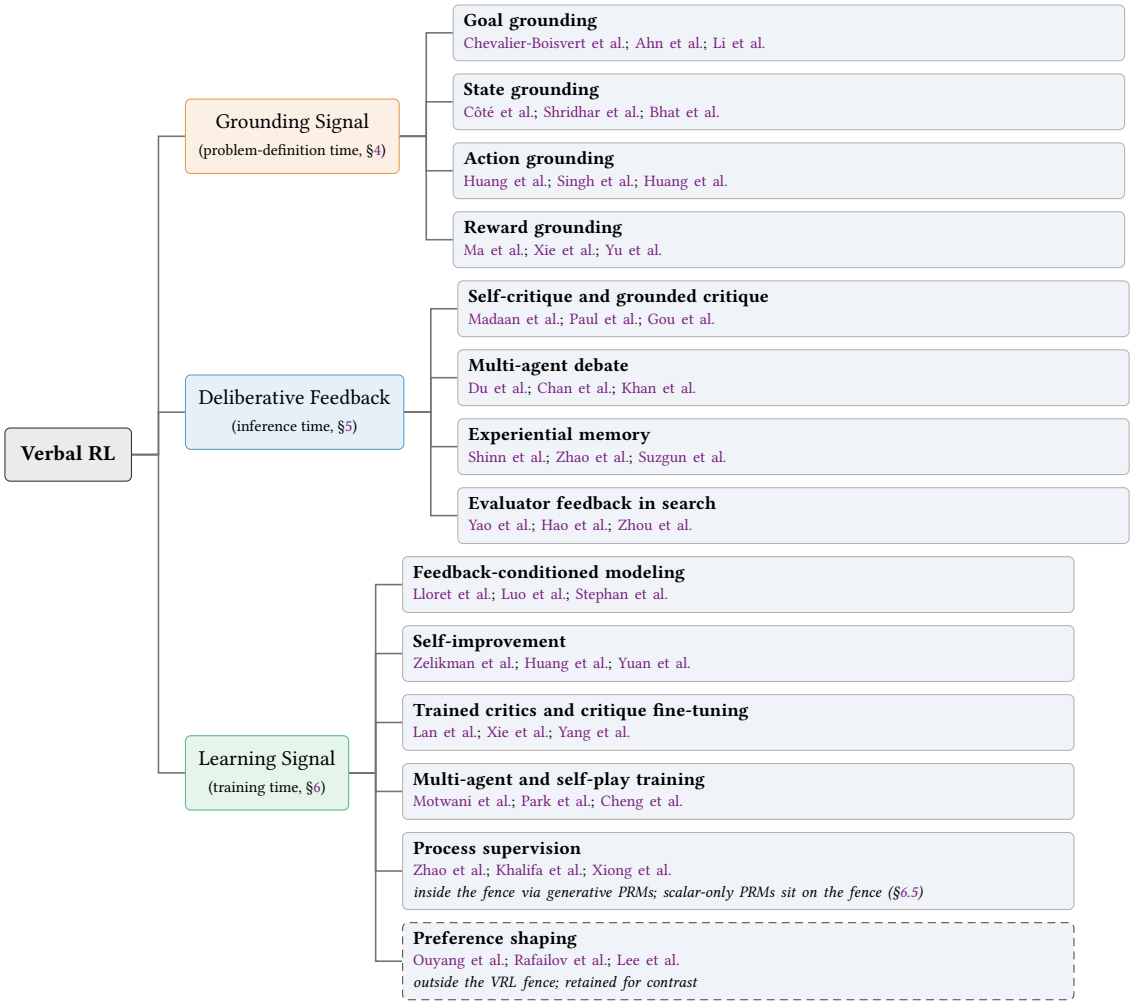

Fig. 2. The three-pillar taxonomy of VRL. Each pillar groups methods by when the linguistic signal takes effect (problem-definition time, inference time, training time) and by what it modifies at that moment. Grounding-cell leaves cite canonical mechanisms together with the environments that isolate them (BabyAI, TextWorld, ALFWorld); under criterion (ii) of §2.2, a grounding signal belongs to VRL only when the component it constructs subsequently drives an improvement loop, so environment citations anchor the settings in which each cell is studied rather than assert membership for every system they host. The dashed preference-shaping leaf lies beyond the VRL fence of §2.2: vanilla RLHF and DPO collapse the verbal judgment to a preference bit before learning, so they appear only as the contrastive scalar endpoint of the compression spectrum (§6).

The first pillar, **language as grounding signal** (§4), acts at problem-definition time and specifies the decision problem itself, using the vocabulary of Markov decision processes [?]: goals, states, actions, and rewards. A natural-language task description such as "You are in a room with two doors" defines what the agent perceives, how it can act, and what counts as success, all before any policy runs. The signal here does not evaluate behavior; it constructs the space in which behavior will later be evaluated. That timing fixes how this pillar passes the definitional fence. A grounding signal does not arrive as feedback on behavior that could be revised, stored, or distilled, so it satisfies

Table 1. Overview of the three VRL pillars. Each column classifies signals by when they take effect and what they modify at that moment; the final row points to the corresponding pillar chapter.

Axis	Grounding signal	Deliberative feedback	Learning signal
When	Problem-definition time	Inference time	Training time
Modifies	Task, state, action, and reward specification	Outputs or reasoning traces	Model weights and training data
Persistence	Defines the task	Single episode (extendable via memory)	All future interactions
Mechanisms	Environment and reward specification	Prompting, critique, memory, search	SFT on conditioned or filtered text; on-policy and multi-agent RL; preference optimization at the scalar endpoint (outside the fence)
Challenges	Grounding gap; compositionality	Feedback quality; compute cost	Signal compression; filter quality
Surveyed in	§4	§5	§6

criterion (ii) of §2.2 only through the clause that admits signals constructing a component of the decision problem that subsequently drives an improvement loop. Reward grounding is the case §2.2 settles by name, the compiled reward driving conventional policy optimization [Ma et al. 2024a]; the other three cells pass by the same route. A grounded goal qualifies when the specification it fixes becomes the objective a training or revision loop optimizes, as when policies are trained directly from language specifications [Li et al. 2026a]; a verbal state description qualifies when it serves as the observation substrate of a closed feedback loop rather than a one-shot prompt [Bhat et al. 2024]; a grounded action interface qualifies when execution outcomes return as language that revises the plan [Huang et al. 2022b; Sharma et al. 2022]. When the constructed component merely conditions a frozen policy and no improvement loop ever runs, the method is plain instruction following or language-conditioned RL [Luketina et al. 2019] and stays outside the fence. The environments that isolate grounding, BabyAI, TextWorld, and ALFWorld among them [Chevalier-Boisvert et al. 2018; Côté et al. 2018; Shridhar et al. 2020], appear in Figure 2 because they anchor the settings in which each cell is studied; citing them asserts no membership for the plain instruction-following baselines they host.

The second pillar, **language as deliberative feedback** (§5), acts at inference time and refines outputs or reasoning traces through critique, memory retrieval, debate, or search, without updating parameters [Madaan et al. 2023; Shinn et al. 2023]. A message such as "Step 3 has an error; try dividing by 2, not 4" redirects a single generation episode but leaves the model unchanged for future tasks. The signal modifies a context, not a weight.

The third pillar, **language as learning signal** (§6), acts at training time, where verbal feedback is distilled into gradient updates that persistently reshape the policy [Yuan et al. 2024; Zelikman et al. 2022]. A judgment such as "Response A is better than B because it addresses the edge case" becomes a preference pair or a training target that alters model weights and therefore affects all subsequent interactions.

The distinction that carries the taxonomy is *persistence*: a grounding signal defines the task space, deliberative feedback improves a single episode, and a learning signal permanently reshapes the policy. Throughout, the vocabulary of Markov decision processes (MDPs) serves as an organizing lens rather than a claim that every method admits an MDP formulation. Many language-agent settings are partially observable, non-Markovian, or lack explicit transition and reward functions, and §2 makes precise where the formalism stops applying.

Table 2. The assignment rule applied to recurring hybrid cases. Placement follows what the linguistic signal modifies at the moment it takes effect; secondary roles are tagged, not suppressed. P1 denotes grounding signal, P2 deliberative feedback, and P3 learning signal.

Hybrid case	Primary	Secondary role
Reward-code generation [Ma et al. 2024a; Xie et al. 2024]	P1	Constructed reward later trains a policy (P3)
Experiential memory [Shinn et al. 2023; Suzgun et al. 2026]	P2	Lessons persist across episodes, bordering long-term adaptation
Constitutional AI [Bai et al. 2022]	P3	Self-critique generates the revisions at inference time (P2)
Feedback-conditioned fine-tuning [Luo et al. 2025]	P3	Critique persists as conditioning context for generation (P2)

3.1 An Explicit Assignment Rule for Hybrid Methods

Methods whose signals cross pillar boundaries are common enough that a taxonomy without a placement procedure invites arbitrariness. We therefore adopt an explicit assignment rule:

A method is placed by what its linguistic signal modifies at the moment the signal takes effect.

The rule resolves the recurring hybrid cases, and Table 2 records the outcome together with each method's secondary role, which we tag rather than suppress. In reward-code generation, a language model turns a natural-language behavior description into an executable reward function [Ma et al. 2024a; Xie et al. 2024]. The linguistic signal takes effect when the reward function is constructed, at which moment it modifies the task specification, so the primary placement is grounding; the constructed reward subsequently drives conventional policy optimization, a secondary learning-signal role that §7 takes up. In experiential memory, an agent writes verbal lessons or strategies during earlier attempts and retrieves them on later ones [Shinn et al. 2023; Suzgun et al. 2026]. Each retrieved lesson takes effect inside the inference-time context and modifies a reasoning trace, never a weight, so the primary placement is deliberative feedback, while the persistence of the memory store across episodes borders long-term adaptation and is flagged as such. Constitutional AI generates self-critiques and revisions at inference time, but these are intermediate products: the revised outputs become training data, and the signal that persists takes effect as a parameter update [Bai et al. 2022], so the primary placement is learning signal with the inference-time critique as a secondary role. Feedback-conditioned fine-tuning is the converse case: motivated explicitly against compressing critiques, the feedback-conditional policy keeps the critique as conditioning context while maximum-likelihood training on response-feedback pairs carries the persistent signal [Luo et al. 2025]; that signal takes effect as a weight update, so the placement is again learning signal, with the conditioning role tagged as deliberative. Because the pillars classify signals rather than systems, these methods legitimately reappear in more than one pillar chapter, each time viewed from the side of the signal under discussion.

3.2 Why Temporal Scope

Alternative organizations of this literature exist. One could classify by feedback source (human, model, or tool), by modality (free text, scalar score, or executable code), or by a binary split on whether parameters are updated. The temporal-scope axis subsumes the last of these: training-time signals are exactly the parametric ones, but the binary split conflates signals that define a task with signals that revise an output, two regimes with little in common beyond frozen weights. Against organizations by source or modality, the decisive argument is that the same sentence produces fundamentally different effects depending on when it is consumed. Consider the critique "The gripper pressure is too

high." Embedded in a task description, it constrains what counts as a valid grasp and grounds the problem. Surfaced during planning, it helps a robot recover from a dropped cup and serves as deliberative feedback. Fed to a reward-code generator whose output trains the policy, it permanently shapes the learned notion of a successful grasp and becomes a learning signal. The sentence is identical in all three cases; its temporal scope alone determines its effect. Each temporal regime also poses its own technical challenge: grounding methods must bridge the gap between language and executable task components, deliberative methods must secure feedback quality without circular self-evaluation, and learning methods must compress rich verbal signals into gradient updates without discarding what made them informative. Organizing by temporal scope groups methods that share failure modes and remedies, which an organization by source or modality would scatter across categories.

3.3 Working Example: A Coding Agent

The pillars are complementary, and a single system can consume signals in all three regimes. Consider an agent tasked with resolving a GitHub issue [Wang et al. 2025d]. Grounding signals define the problem: the natural-language issue description states the goal, the repository structure delimits the state space, and the test suite implicitly encodes a reward function. During the attempt, deliberative feedback drives the repair loop: the agent generates a patch, executes the tests, reads tracebacks and error messages as verbal critique, and revises. If successful and failed trajectories are later harvested to fine-tune the underlying model through preference optimization, the same traces re-enter as learning signals that persistently improve the policy. We stress what this example does and does not show. It does not show a pipeline in which one regime hands off to the next, nor a lifecycle that any single method implements. It shows one system in which signals of the three kinds coexist, consumed on different timescales, by different components, with different persistence. A traceback is deliberative feedback when the running agent conditions on it and a learning signal when it reappears in a training corpus; the classification follows the signal, not the system.

3.4 Reading Guide

Sections 4, 5, and 6 survey the three pillars in turn, each closing with the shared bottleneck that limits its methods. All three consume the same evaluative substrate, the verbal reward stack of rubrics, LLM judges, and process reward models (PRMs), which §7 dissects before the cross-cutting chapters on credit assignment, optimization, embodiment, evaluation, and safety; the full chapter-by-chapter roadmap appears at the end of §1.

4 Pillar 1: Language as Grounding Signal

The first pillar collects methods in which language is consumed to define the decision problem itself. Verbal feedback maps onto specific elements of the Markov decision process [MacGlashan et al. 2017; **?**]: goal grounding specifies what the agent should optimize (§4.1), state grounding determines what it perceives (§4.2), action grounding resolves how it can act (§4.3), and reward grounding compiles what counts as success (§4.4). Under the assignment rule of §3, a method belongs to this pillar when its linguistic signal modifies a component of the task specification at the moment it takes effect, before any behavior is generated or evaluated. As in §3, the MDP vocabulary serves as an organizing lens rather than a formal commitment: several settings below are partially observable or lack explicit transition models, and we note where the formalism thins. The pillar matters because many failures of language-based agents stem not from poor optimization but from poor grounding. Feedback that cannot be mapped to executable actions or valid reward conditions will not improve the agent, and a recurring lesson across all four subcategories is that more detailed verbal

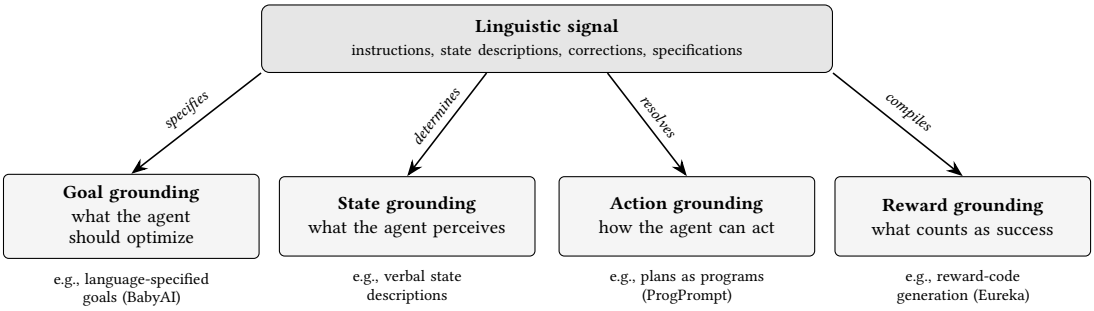

Fig. 3. Grounding signals map verbal input onto the four MDP components at specification time, fixing what the agent optimizes, perceives, does, and counts as success before any behavior is generated or evaluated. Each component box lists one exemplar mechanism discussed in this section.

specification does not by itself yield better grounding; the binding constraint is whether the mapping from language to MDP components is precise enough to act on.

Grounding language in sequential decision making predates large language models. Mapping instructions to actions was formulated as an RL problem well before pretrained interpreters existed [Branavan et al. 2009; Misra et al. 2017], policy sketches used linguistic annotations to decompose multitask learning [Andreas et al. 2017], and agents acquired grounded word meanings in simulated 3D worlds [Hermann et al. 2017]; Luketina et al. [2019] organize this earlier literature into language-conditional and language-assisted RL, and language-conditioned imitation extended it to robot control [Lynch and Sermanet 2021]. The LLM era reverses the engineering burden. Earlier systems trained task-specific interpreters for a fixed environment, whereas current systems inherit interpretation from pretraining and spend it compiling specifications, which lets language reach parts of the MDP, most notably the reward function, that previously required manual engineering.

4.1 Goal Grounding

Goal grounding uses verbal input to specify what the agent should achieve. It requires parsing instructions into objects, relations, and target conditions [Chaplot et al. 2018], filtering candidate plans by physical executability [Ahn et al. 2022; Driess et al. 2023], and decomposing goals into verifiable subgoals. In MDP terms, the grounded goal fixes the task over which every later optimization or revision runs, so goal-grounding errors are inherited by both remaining pillars. Recent work extends the mechanism to multi-agent coordination [Zhang et al. 2025k], hierarchical goal decomposition via offline RL [Hu et al. 2025a], and training directly from language specifications without manually designed reward functions [Li et al. 2026a]. The persistent challenge is compositional generalization: agents must follow goals assembled from familiar primitives in unfamiliar combinations. BabyAI isolates the problem in gridworlds [Chevalier-Boisvert et al. 2018], state-of-the-art LLMs still reach only about 75% coverage under compositional constraints [Sakai et al. 2025], and compositional policy representations improve sample efficiency on novel combinations [Cohen et al. 2025].

4.2 State Grounding

Where goal grounding defines what to achieve, state grounding defines what the agent perceives. Verbal state descriptions are the native observation space of text environments [Côté et al. 2018; Yao et al. 2020], but they arise whenever visual or physical state is summarized into language. Shridhar et al. [2020] show that agents trained on verbal state descriptions

transfer to embodied 3D settings, evidence that language can act as a modality-invariant state representation; later work adds closed-loop verbal state feedback for robotic planning [Bhat et al. 2024] and scene-graph-structured grounding [Huang et al. 2025c]. Viewed as an RL object, a verbal state description is a state abstraction, and the question it must answer is sufficiency: does the abstraction preserve the information needed to distinguish actions with different values? Verbalization is lossy, and it can silently break the Markov property when omitted details carry decision-relevant history. The practical challenge is therefore information preservation rather than descriptive richness, a specific instance of the grounding gap above.

4.3 Action Grounding

Given a goal and a perceived state, the agent must determine how to act. Action grounding resolves verbal instructions into skills, tool calls, or motor commands, and it requires both that instructions map to executable actions and that execution outcomes return as language to revise subsequent steps. Huang et al. [2022b] close this loop in Inner Monologue, where verbalized success detection and scene description feed back into an LLM planner, and Sharma et al. [2022] show that short corrective utterances can amend a mistaken step without disrupting the rest of the plan. A parallel line makes the executable form explicit by compiling instructions into programs: ProgPrompt prompts the LLM with program-like specifications of the available actions and objects so that generated plans are situated and executable [Singh et al. 2022], and VoxPoser composes language into 3D value maps that a motion planner consumes [Huang et al. 2023], in both cases recasting instruction following as constrained code generation. Recent systems tighten the closed-loop integration: multi-modal grounded replanning corrects ambiguous instructions with visual cues [Kim et al. 2025], constraint-aware visual programming detects failures proactively [Zhou et al. 2025c], corrective planning spans physical, logical, and semantic errors across hierarchical control levels [Joublin et al. 2024], and visually grounded task-and-motion planning couples LLM reasoning with feasibility checking [Zhang et al. 2025c]. The central difficulty is granularity alignment: the same instruction can denote a temporally extended skill ("make coffee"), a discrete subgoal ("go to the kitchen"), or a motor command ("close the gripper"), each demanding a different language-to-control interface, and the chosen granularity in turn sets the horizon over which exploration and credit assignment operate (§8). One response is to make the intermediate action vocabulary itself linguistic: RT-H inserts fine-grained language motions such as "move arm forward" between task and actuation, so that human corrections arrive in the same space the policy already predicts [Belkhale et al. 2024]. We examine such embodied instantiations in §10; here they illustrate the general mechanism.

4.4 Reward Grounding: From Language to Reward Functions

The preceding subcategories ground objectives, observations, and actions; reward grounding closes the loop by grounding success itself. Its distinguishing feature is that language is compiled into the reward function and thereby becomes part of the MDP, rather than serving as a direct runtime learning signal. This is the boundary case the definitional fence of §2 settles: the linguistic structure is functionally consumed at specification time, so criterion (i) is satisfied even though the compiled function later emits scalar rewards. The assignment rule places these methods here, their downstream training role is recorded among the hybrid cases of §3, and their runtime counterparts, in which rubrics, generative judges, and process reward models compute rewards during training rather than before it, are the subject of §7.

The language-to-reward line is best read as a spectrum ordered by how directly language specifies the reward function; because these systems are also the core of the embodied reward pipeline, we describe the mechanisms here and leave their quantitative results to §10. At one end, language writes reward code. Eureka runs an evolutionary

search over LLM-generated reward programs, feeding training statistics back as textual reflection to mutate the code, and outperforms expert-designed rewards on a large majority of its test environments [Ma et al. 2024a]; Text2Reward generates dense, human-refinable reward code that matches or beats expert-written rewards on most of its manipulation tasks [Xie et al. 2024]. Follow-on work widens the same mechanism: DrEureka extends generation beyond the reward to the domain-randomization configuration needed for simulation-to-real transfer [Ma et al. 2024b], CARD refines reward code from trajectory-based feedback without retraining a policy at every iteration [Sun et al. 2025a], PROF carries the paradigm to offline settings through preference optimization [Sun et al. 2025b], and reward functions can be generated from video demonstrations [Zeng et al. 2024]. In the middle of the spectrum, language selects the parameters of a structured reward rather than writing it from scratch: in Language to Rewards, instructions and corrections set reward parameters that a real-time optimizer consumes, making the reward function the interface between language and low-level control and substantially outperforming a primitive-skill baseline built on Code as Policies [Liang et al. 2022; Yu et al. 2023]. At the far end, when the environment exposes no compilable state, the specification is grounded through a learned judge: RL-VLM-F queries a vision-language model for preferences over pairs of image observations under a text goal, sidestepping noisy raw scores [Wang et al. 2024f], and purpose-trained vision-language reward models push the same idea toward general-purpose reward provision [Lee et al. 2026; Schroeder et al. 2026]. Consistent with the modality scope fixed in §2, the verbal signal in these systems is the language half, the goal text and any generated critique or reasoning, while the grounding is visual. §10 examines these judges in their embodied context, together with benchmark evidence on where their task understanding breaks down.

The RL-theoretic consequences differ from those of the other subcategories. Reward-grounding errors are specification failures rather than estimation noise: a mis-compiled reward is optimized exactly, so misspecification surfaces as confident, wrong behavior, a risk we take up with reward hacking in §12. Iterative refinement loops such as Eureka's reflection are therefore best understood as search in reward-function space, an instance of the language-space optimization treated in §9. Reward grounding also interacts with sparsity and credit: LaRe compiles language into symbolic latent-reward evaluators that redistribute delayed episodic rewards [Qu et al. 2024, see §8], and Agent-RLVR shows that verbal guidance can densify rather than replace a sparse verifiable reward on software-engineering tasks, a mechanism the runtime reward stack of §7 treats in detail [Da et al. 2025].

4.5 Synthesis: Shared Failure Modes and When to Ground

Two failure modes recur across the four subcategories. The first is the grounding gap: added specification detail does not monotonically improve grounding, because the binding constraint is the precision of the language-to-MDP mapping, whether instructions resolve to executable actions, descriptions to sufficient state, and success criteria to checkable conditions. Controlled evidence points the same way: agents trained with more diverse and informative language feedback generalize better and adapt faster, a study §10 returns to [Xi et al. 2024a]. The second is compositionality: goals assembled from familiar primitives fail in novel combinations [Sakai et al. 2025], reward code composes interacting terms whose joint effect is hard to anticipate, and single utterances mix granularities, so every subcategory inherits the difficulty of composing linguistic pieces into one coherent formal object. Reward grounding adds a third failure mode, fidelity: a specification can be syntactically valid and executable yet fail to mean what its author intended, and because the grounded object is the task definition itself, no later feedback corrects it unless the loop reaches back to specification, which is precisely what Eureka's reflection and CARD's trajectory feedback do. The stakes of mis-grounding scale with deployment: an incorrectly grounded goal or reward in a coding assistant, a robot, or a clinical decision-support system produces behavior that is optimized, confident, and wrong, and §12 returns to who bears the cost of these failures.

These observations delimit when the pillar is the right tool. Grounding signals are the natural choice when the decision problem is not yet well posed: no programmatic reward exists, the state space has no canonical encoding, or the action interface is undetermined. In that regime language does work no scalar signal can, converting intent into a formal problem. Once grounding has produced a well-posed problem, the remaining pillars act on behavior within it: deliberative feedback revises individual episodes (§5), and learning signals durably reshape the policy (§6). The practitioner guide in §13 turns this division of labor into explicit selection guidance.

5 Pillar 2: Language as Deliberative Feedback

With the task specified through a grounding signal (§4), we turn to methods that refine behavior within an episode without updating model parameters, the regime closest to test-time compute scaling [Snell et al. 2024]. The four cells of this pillar differ in where critique originates: the model itself or an external tool, peer models, past episodes, or parallel candidate paths. Following the assignment rule of §3, a method belongs here when the linguistic feedback it produces is consumed in the current context window; when the same feedback is instead compressed into a dataset, a reward, or weights, the method moves to §6. We discuss self-generated and tool-generated critique together, since they share one loop structure and differ mainly in the origin and reliability of the feedback.

5.1 Self-Critique and Externally Grounded Critique

The simplest deliberative loop asks the model to critique its own output and revise accordingly. Self-Refine [Madaan et al. 2023] formalized the pattern, building on the demonstration in Constitutional AI [Bai et al. 2022] that models can evaluate outputs against written principles. The central obstacle is circularity: the critic and the generator are the same model and share the same blind spots. Replacing self-generated critique with evaluation from a separately trained critic improves reasoning [Paul et al. 2024], evidence that feedback quality, not the revision mechanism, is the bottleneck. Self-correction degrades when the critique source shares the generator's failure modes [Huang et al. 2024a; Kamoi et al. 2024], and the degradation is sharpest in smaller models [Olausson et al. 2024]. Mitigations include Socratic decomposition of the task into independently checkable steps [Shi et al. 2025a] and parallel refinement across several candidates [Wang et al. 2025i]. None of these helps when the error stems from missing knowledge, which no amount of introspection can supply.

External grounding breaks the circle by anchoring critique in deterministic tool output [Schick et al. 2023]. Execution traces, unit-test verdicts, search results, and API responses [Chen et al. 2024a; Gou et al. 2024b] give the model verified evidence from which to generate corrected outputs [Gou et al. 2024a; Wang et al. 2024b; Yang et al. 2024a], and reinforcement learning can teach models to exploit such feedback more effectively [Gehring et al. 2024], an early instance of the migration toward §6 that recurs throughout this pillar. Grounding shifts the dominant failure mode from blind spots to trust asymmetry: the tool output is accurate, but the model may misattribute the root cause behind a reported trace. The approach also inherits practical costs, including infrastructure overhead and latency, and it remains confined to domains that expose a verifiable oracle.

5.2 Multi-Agent Debate

Multi-agent debate distributes critique across parallel model instances, often assigning distinct personas to widen perspective [Hong et al. 2024b]. Exchanging critiques improves reasoning even among identical models [Du et al. 2024], role diversity strengthens the discussion further [Chan et al. 2023], models can be trained to revise their reasoning

under conflicting opinions [Liu et al. 2026a], and adaptive heterogeneous line-ups improve mathematical reasoning [Zhou and Chen 2025].

A second wave of work asks when this apparatus pays for itself. Reframed as a test-time scaling technique combining collaborative refinement with diverse exploration, debate offers limited advantage over single-agent scaling on mathematical reasoning but becomes more effective as problem difficulty rises and model capability falls; on safety tasks, collaborative refinement can increase vulnerability, while diverse agent configurations gradually reduce attack success [Yang et al. 2025c], a finding §12 revisits. A systematic evaluation of five representative debate methods across nine benchmarks and four foundation models reaches a blunter verdict: debate often fails to outperform single-agent chain-of-thought and self-consistency despite much higher inference cost, and model heterogeneity is the one intervention that consistently helps [Zhang et al. 2025a]. This matches the repeated observation that debaters sharing biases converge to redundant consensus [Estornell and Liu 2024; Liang et al. 2024a] and that current debate methods do not reliably beat simpler single-agent strategies [Choi et al. 2025].

Communication structure is the other economic lever. Full-broadcast designs incur substantial token overhead [Choi et al. 2025], motivating sparse topologies [Li et al. 2024b], per-round retention of only the responses that maximally disagree with one another and with the majority vote [Nguyen et al. 2026], and explicit confidence expression, so that an agent holding a genuine knowledge advantage communicates it rather than capitulating to the group [Lin and Hooi 2025].

Debate also functions as an oversight protocol in which a weaker judge arbitrates between stronger debaters. Debate lifts non-expert model judges to 76% accuracy and human judges to 88%, against naive baselines of 48% and 60% [Khan et al. 2024], and it beats consultancy across tasks, outperforming direct question answering under information asymmetry, with mixed results without it [Kenton et al. 2024]. Because the same mechanism recurs across pillars, we classify debate systems by where the transcript ends up: consumed in the context window (this pillar), distilled into training data [Motwani et al. 2024; Subramaniam et al. 2025], scored as a reward channel [Park et al. 2025], or internalized into a single model's weights [Yi et al. 2026], all destinations surveyed in §6.

5.3 Experiential Memory

Where the preceding cells regenerate feedback from scratch in every episode, experiential memory persists lessons for reuse. Dedicated surveys already map this territory as a component question, cataloguing episodic, semantic, and procedural stores together with their design and evaluation [Du 2026; Zhang et al. 2025b]; our treatment is deliberately narrower and role-based. Memory enters VRL when the stored artifact is itself a linguistic feedback signal that a later episode consumes as deliberative context. This stance also resolves the boundary question that memory poses for any taxonomy: lessons persist across episodes, yet we classify by the moment of consumption, since a retrieved reflection conditions the current episode exactly as a fresh critique would. Work formalizing deployment-time learning as a lifecycle stage of its own, after pre-training and post-training, makes the same cut [Guo et al. 2026]. A complementary boundary separates memory from language-space optimization, and §9 states the deciding rule in full: in brief, a store the agent itself reads and writes during episodes acts as deliberative feedback and stays here, whereas a context curated by an outer loop against a task metric is a parameter of the system.

The literature is best organized by what persists. At one end sits reflective text: Reflexion stores verbal self-reflections in an episodic buffer that conditions subsequent trials, raising HumanEval pass@1 to 91% over the prior state of the art of 80% from GPT-4 [Shinn et al. 2023]; it is also the paper that coined verbal reinforcement learning in the narrow self-reflective sense that §2 generalizes. Successors distill rather than transcribe. ExpeL extracts cross-task insights from

paired successes and failures [Zhao et al. 2024], MetaReflection compresses reflections from failed trials into portable instructions [Gupta et al. 2024], and Agent Workflow Memory induces reusable workflows from past trajectories [Wang et al. 2024e]. ReasoningBank distills strategies from self-judged successes and failures and couples the store to test-time scaling, spending extra compute to generate contrastive experiences that improve the memory itself [Ouyang et al. 2025], while Dynamic Cheatsheet lets a black-box model curate its own concise strategy notes, more than doubling Claude 3.5 Sonnet's AIME accuracy once algebraic insights persisted across questions [Suzgun et al. 2026]. A third rung structures the store: Zettelkasten-style interlinked notes [Xu et al. 2025b], production-oriented consolidation reporting 91% lower p95 latency and more than 90% token-cost savings versus full-context prompting [Chhikara et al. 2025], hierarchical collaboration graphs for agent teams [Zhang et al. 2025e], and reconstruction-time retrieval that adapts memory access to the evolving reasoning context [Ji et al. 2026]. The executable end of the axis descends from Voyager's skill library of code [Wang et al. 2023b]: procedural memory at instruction and script granularity, which transfers from stronger to weaker models [Fang et al. 2025a]; subagent code preserved on the argument that textual reflections cannot reliably guarantee efficient re-execution [Zhang et al. 2026d]; skills managed as long-lived, testable assets under explicit lifecycles [Lin et al. 2026; Yang et al. 2026b; Zhou et al. 2026]; and governance frameworks that admit skill updates only on attributed evidence of success [Liu et al. 2026f].

A second axis tracks who writes. Append-only accumulation gives way to utility-based refinement that prunes stale or misleading entries [Cao et al. 2025; Zhang et al. 2026e], then to write policies trained with parametric reinforcement learning: Memory-R1 learns explicit add, update, delete, and no-op operations from outcome reward using only 152 training pairs [Yan et al. 2025b], AutoMem treats memory management as a trainable metamemory skill [Wu et al. 2026b], and SkillRL co-evolves a hierarchical skill library with the agent's policy during training [Xia et al. 2026]. Beyond these lies meta-evolution of the memory system itself, searching a modular design space of encode, store, retrieve, and manage operations [Zhang et al. 2025j] or treating the whole pipeline as an evolvable program refined from diagnosed failures [Liu et al. 2026d; Zhang et al. 2026c]. The trained rungs stress the assignment rule of §3 without breaking it. The linguistic signal is still the stored text, and it still takes effect in the context window rather than in the weights, so these methods stay in this pillar. The outcome reward that trains the write policy carries no linguistic structure, so it fails criterion (i) of the definitional fence (§2) and is not a verbal signal at all; the parametric loop around the writer is conventional reinforcement learning applied to VRL machinery, not a verbal learning signal, and it does not move these methods to §6.

Persistence cuts both ways. Poor memories propagate mistakes, growing stores demand increasingly complex retrieval [Park et al. 2023], stale knowledge misleads in non-stationary environments [Zhang et al. 2025b], and adversarially injected memories can cause cross-session drift [Dong et al. 2026b], a risk examined in §12. The sharpest counterweight is evidence that continuous consolidation itself corrupts: agents that preserve raw episodes double the accuracy of counterparts forced to consolidate after every interaction, which suggests that consolidation should be explicitly gated rather than habitual [Zhang et al. 2026b]. Evaluation is meanwhile consolidating around benchmarks probing accurate retrieval, test-time learning, long-range understanding, and selective forgetting [Belikova et al. 2026; Hu et al. 2025b; Tan et al. 2025b; Wei et al. 2025a], to which §11 returns.

5.4 Evaluator Feedback in Search

The final cell covers deliberation that branches. Rather than refining a single trajectory, these methods explore multiple candidate paths and use verbal feedback to decide which to expand, prune, or abandon [Wang et al. 2025g]. The inclusion criterion is the verbal evaluator signal consumed during search, not the search procedure itself: decoding or sampling

strategies that involve no linguistic feedback, such as majority voting over sampled chains, fall outside VRL. Tree of Thoughts extends chain-of-thought prompting [Wei et al. 2022] with tree search over intermediate thoughts scored by self-assessment [Yao et al. 2023a]; Reasoning via Planning [Hao et al. 2023] and Graph of Thoughts [Besta et al. 2024] generalize the topology to non-linear dependencies such as aggregation and refinement. The distinctive advantage is that verbal evaluation applies at every node, so unproductive paths are abandoned early; the distinctive cost is that exploring many states multiplies model calls, making these methods far more expensive than single-pass generation [Xie et al. 2023].

Synthesis: when this pillar wins. Three variables organize the four cells. The first is feedback quality, the binding constraint everywhere: self-critique is the cheapest and least reliable source, debate buys reliability only from genuine diversity, and tool output is the most reliable signal but exists only where the domain exposes an oracle. The second is circularity, which grounding breaks, debate dilutes, and memory can quietly reintroduce by entrenching early errors. The third is compute: debate and search multiply model calls within an episode, whereas memory amortizes deliberation across episodes and can even invert the cost curve, as when a small model with a refined experience pool outperforms a larger memoryless one [Cao et al. 2025]. Deliberative feedback is therefore the pillar of choice when weights are frozen behind an API, when tasks recur so that lessons amortize, and when problems are hard relative to model capability. When training access exists and the task distribution is stationary, the same transcripts are usually better internalized as learning signals at a fraction of the inference cost (§6); §13 turns these conditions into an explicit decision procedure.

6 Pillar 3: Language as Learning Signal

The pillars discussed so far leave the model's parameters untouched: grounding signals (§4) define the task, and deliberative feedback (§5) steers a single episode. This pillar covers methods that convert verbal feedback into training signals, producing persistent improvement in the policy itself. We organize the literature along a compression spectrum: how much linguistic structure survives on its way into the parameter update. At one end, feedback-conditioned modeling (§6.1) preserves entire critiques as conditioning context; self-improvement (§6.2) uses feedback only as a filter over model-generated data; trained critics and self-play occupy the middle bands (§6.3, §6.4); process supervision (§6.5) compresses judgments to step-level scores and stays inside the VRL fence only on the strength of its generative variants, an adjudication that subsection states explicitly; and preference shaping (§6.6) reduces a comparative verbal judgment to a single bit. The spectrum expresses a basic trade-off: richer linguistic signals retain more information but resist standard policy-gradient machinery, while scalar compression enables well-understood optimization at the cost of discarding the structure that motivated verbal feedback. A second axis, online versus offline, cuts across every band and determines whether feedback is queried on fresh policy samples or inherited from a fixed corpus; it bites hardest in preference shaping. Throughout, we name each family's optimization recipe, since compression level largely determines the optimizer: methods that preserve language train by supervised fine-tuning on conditioned or filtered text, the middle bands increasingly use on-policy and multi-agent reinforcement learning, and scalar compression inherits the standard preference-optimization toolkit. Figure 5 summarizes the spectrum.

6.1 Feedback-Conditioned Modeling

At the least-compressed end, feedback-conditioned modeling keeps critiques intact by conditioning the model directly on natural-language feedback during training. The canonical training signal is a triple (x, v, a^*): an input, a critique of an initial response, and the revised output. A central design question is whether the critique remains in the training context.

Early work discarded it, supervising only on the refined output [Chen et al. 2024b; Scheurer et al. 2023]; ALT [Lloret et al. 2024] showed that retaining the critique as conditioning context lets the model learn an explicit mapping from critique to correction. The feedback-conditional policy (FCP) of Luo et al. [2025] gives the family its clearest formal statement: on the argument that compressing nuanced feedback into a scalar reward discards richness and induces scale imbalance, the policy is trained by maximum likelihood on offline response-feedback pairs to approximate the feedback-conditional posterior, then refined by an online bootstrapping stage that generates under positive conditioning and collects fresh feedback. Feedback integration can also be treated as a trainable skill rather than an emergent property: Klissarov et al. [2026] convert single-turn verifiable tasks into multi-turn didactic interactions and train the model to predict the teacher's critiques, internalizing the external signal and lifting a smaller model's multi-turn performance to near that of a model an order of magnitude larger. Scope is a complementary concern: RLVF [Stephan et al. 2024] compiles a high-level verbal instruction into a synthetic preference set specifying where the feedback should and should not apply, reducing overgeneralization by 30% while matching in-context baselines on adherence. The receptiveness that makes these methods attractive is also their risk: models may learn to follow critiques indiscriminately, even when they are wrong [Sharma et al. 2024].

6.2 Self-Improvement

One step down the spectrum, self-improvement treats language as a filter rather than a conditioning signal. The core mechanism is generate-then-filter: the model samples candidate trajectories, verbal feedback determines which survive, and the survivors become supervised fine-tuning data for the next iteration [Singh et al. 2023]. The filter defines what is learned: final-answer correctness in STaR [Zelikman et al. 2022], adherence to constitutional principles [Bai et al. 2022], majority agreement across diverse samples [Huang et al. 2022a], or the model's own judgment applied across iterations [Yuan et al. 2024]. Because language acts only as a gate, the loop runs cheaply at scale, but filter quality is the bottleneck: when one model serves as both generator and judge, its systematic biases compound over rounds [Shumailov et al. 2023]. Recent work attacks this bottleneck from two directions. Multiagent finetuning [Subramaniam et al. 2025] applies self-improvement to a society of models initialized from the same base, training each member on data generated through their interactions; the resulting specialization preserves diverse reasoning chains and sustains improvement over many more rounds than single-agent self-improvement. Agent-R [Yuan et al. 2025a] replaces trajectory-level filtering with targeted revision, using Monte Carlo tree search to splice failed paths onto adjacent correct ones so the model learns to recover from its first erroneous step, improving over baselines by 5.59% across three interactive environments.

6.3 Trained Critics and Critique-Based Fine-Tuning

The families above hold the feedback source fixed. A distinct line of work trains the critic itself, treating critique generation as a capability to optimize rather than a free byproduct of a frozen model. Supervision for the critic can come from aggregation: MultiCritique [Lan et al. 2024a] distills critiques from multiple agents into fine-tuning and preference data, yielding a 7B critic that surpasses comparable and larger open-source models and approaches 70B models and GPT-4 on critique benchmarks. It can come from downstream effect: CTRL [Xie et al. 2025a] trains a critic with reinforcement learning to maximize the correction performance of a fixed code generator, with no human supervision; the critic transfers across generators and yields up to 106.1% relative improvement on challenging code-generation benchmarks through iterative critique and revision. Critique-RL [Xi et al. 2025] diagnoses a subtle failure of purely indirect reward: optimizing the critic only on whether the actor's revision improved raises helpfulness but leaves discriminability poor, so a first on-policy stage reinforces discriminability with rule-based rewards before a second

stage adds refinement-based reward, gaining 9.02% in-domain and 5.70% out-of-domain on Qwen2.5-7B. For step-level mathematical critique, DeepCritic [Yang et al. 2025a] combines fine-tuning on long-form deliberate critiques with reinforcement learning over human or Monte Carlo step labels, producing a 7B critic that outperforms existing LLM critics, GPT-4o included, on error-identification benchmarks, and AutoMathCritique [Xi et al. 2024b] pairs an actor with a critic trained on an automated corpus of step-level feedback. The two-player pattern extends to interactive agents: critique-guided improvement [Yang et al. 2025b] trains both a critic that produces fine-grained assessments and an actor that learns to exploit them, arguing that natural-language feedback steers exploration away from local optima better than scalar rankings; its small trained critic exceeds GPT-4 in feedback quality. AutoMathCritique also illustrates the classification rule of §3: the same trained critic supervises exploration at test time (Pillar 2) and supplies training data (Pillar 3), so placement follows the feedback's destination, not the mechanism that produced it.

6.4 Multi-Agent and Self-Play Training

When both sides of the feedback loop are trained jointly, VRL becomes a multi-agent learning problem and the algorithmic toolkit shifts accordingly. MALT [Motwani et al. 2024] post-trains a generator-verifier-refiner pipeline by sampling role-specific search trees, grading final outputs against ground truth, and propagating reward back to each role-conditioned model by value iteration, reporting relative improvements of 15.66% on MATH, 7.42% on GSM8K, and 9.40% on CSQA; we revisit its credit-propagation scheme in §8. MAPoRL [Park et al. 2025] co-trains discussion participants with multi-agent reinforcement learning against a verifier that scores correctness plus incentives for corrective and persuasive contributions, finding that training individual models in isolation is insufficient to induce collaboration. Self-play makes the loop adversarial and removes the need for external annotation. SPAG [Cheng et al. 2024a] improves reasoning across a broad range of benchmarks through reinforcement learning on adversarial word-game outcomes; language self-play [Kuba et al. 2025] dispenses with training data entirely by casting capability as performance in a game played against itself; and Multi-Agent Evolve [Chen et al. 2025b] instantiates proposer, solver, and judge roles from a single model and co-evolves all three, averaging a 4.54% gain on Qwen2.5-3B-Instruct. SPC [Chen et al. 2025c] turns the same idea toward critic training, evolving a critic against a sneaky generator fine-tuned to produce hard-to-detect erroneous steps and raising error-detection accuracy on ProcessBench from 70.8% to 77.7%. Communication itself can carry the learning signal: rewarding messages by their influence on other agents doubles win rates in a social deduction game and elicits emergent accusation and evidence-giving [Sarkar et al. 2025], while dialogue-game environments show interactive GRPO training yielding balanced improvements where supervised fine-tuning degrades untargeted skills [Horst et al. 2025]. Persuasion-balanced training [Stengel-Eskin et al. 2024] addresses a failure specific to trained debaters: optimizing only against adversarial persuasion degrades acceptance of beneficial persuasion; preference optimization over recursive dialogue trees teaches models to accept persuasion exactly when appropriate and stabilizes mixed-strength debate teams. Debate finally closes the loop between this pillar and §5: latent agents [Yi et al. 2026] distill debate transcripts into a single model that matches or exceeds explicit debate with up to 93% fewer tokens, and a family of scalable-oversight methods trains against adversarial verification, from prover-verifier games that reward checkability [Kirchner et al. 2024], to debate outcomes as rewards for sequential decision-making [Sukovic and Radanovic 2024], to self-play-trained debaters that improve judge accuracy [Arnesen et al. 2024] and debate-enhanced supervision for weak-to-strong generalization [Lang et al. 2025]. We take up the oversight and safety implications of this family in §12.

6.5 Process Supervision

Process supervision reduces judgment to scalar scores over individual reasoning steps: annotators assess each intermediate step, and the assessments train process reward models (PRMs). In the canonical construction the labels are click-level, positive, negative, or neutral per step [Lightman et al. 2024], so little linguistic structure is ever produced, let alone consumed. This band therefore requires an explicit fence adjudication, which we state here rather than leave implicit. Under the scalarization test of §2.2, a vector of per-step scores is $\sigma(\ell)$ no less than a single preference bit; criterion (i) grants no exemption for k equal to the number of steps, so discriminative PRMs trained on click-level labels fail it exactly as vanilla RLHF does. What places the band inside the fence in Figure 5 is not the count of scalars but the boundary trace of §2.2: generative PRMs reason in language before emitting step scores, so linguistic structure is consumed inside the reward computation, the same route by which compiled rewards qualify [Liu et al. 2025a; Zhao et al. 2026], and step-level critiques whose text is converted into credit qualify outright (§8). The scalar-only core of the family sits on the fence itself, retained for lineage and contrast. That lineage matters because even this maximal reduction pays off: step-level supervision substantially outperforms outcome-level supervision [Lightman et al. 2024; Uesato et al. 2022], since localized signals permit more precise credit assignment, and the cost of step annotation has driven automation [Luo et al. 2024; Wang et al. 2024d]. Step-level granularity has real limits: annotation remains costly, and an individually correct step is hard to define outside mathematics and code, one reason those domains dominate the empirical literature. We keep this brief: the design space of verbal reward models, generative PRMs included, is the subject of §7, and where in a trajectory credit should land is treated in §8.

6.6 Preference Shaping: The Scalar Endpoint

Preference shaping is the maximal compression: a comparative judgment between two responses becomes a single preference bit that drives the policy update. Building on preference-based reinforcement learning [Christiano et al. 2017], InstructGPT [Ouyang et al. 2022] established the modern recipe at scale, training a reward model on pairwise rankings and optimizing the policy online with proximal policy optimization [Schulman et al. 2017; Stiennon et al. 2020]. Direct preference optimization (DPO) [Rafailov et al. 2023] removes the explicit reward model and trains directly on preference pairs, and a family of successors varies the implicit objective [Azar et al. 2024; Ethayarajh et al. 2024; Hong et al. 2024a; Meng et al. 2024], while RLAIF [Lee et al. 2023] replaces human annotators with AI judges to scale collection. The online-offline distinction matters most in this band. Online optimization against a learned reward model queries feedback on fresh policy samples as the policy moves, whereas offline objectives such as DPO inherit the coverage of a fixed preference corpus and never observe the policy's new failure modes; iterated schemes that re-collect preferences over successive policy versions occupy the middle ground [Yuan et al. 2024]. Under the inclusion criteria of §1, preference shaping marks the outer boundary of VRL: once the judgment is compressed to a preference bit before any learning occurs, no linguistic structure is consumed by the learner, so vanilla RLHF and DPO sit outside the paradigm [whose open problems are surveyed by Casper et al. 2023]. We retain them here as the endpoint of the compression spectrum because they clarify, by contrast, what the methods above preserve.

Synthesis: compression versus optimizability. Read as a whole, this pillar trades signal richness against ease of optimization. Full critiques carry the most information per interaction but demand either a conditioning mechanism or a consumer trained to parse them; scalar preferences plug into mature optimizers but pay in expressiveness and sample efficiency. Evidence for the price of compression is accumulating: GOLF [Huang et al. 2026] aggregates group-level critiques into off-policy scaffolds injected in sparse-reward regions and reports a 2.2× sample-efficiency improvement

Table 3. The verbal reward stack. Each rung is characterized by where linguistic structure collapses into the scalar consumed by the optimizer (the scalarization boundary) and by the granularity at which the signal attaches to the trajectory.

Rung	Reward computation	Scalarization boundary	Granularity	Representative work
Verifiable outcome	Programmatic check against ground truth	At the environment; no linguistic structure in the reward	Episode	[Guo et al. 2025b; Lambert et al. 2024a]
Rubric reward	Judge checks instance-specific criteria; item scores pooled	At aggregation of criterion verdicts	Response	[Gunjal et al. 2025; Huang et al. 2025e; Viswanathan et al. 2025]
Generative reward model	Judge writes principles and critique, then a verdict	At verdict emission; critique retained as an audit trail	Response	[Chen et al. 2025a; Liu et al. 2025b; Zhang et al. 2025g]
Process reward	Step-level verification, increasingly verbalized	Per step (per token at the limit)	Step or token	[Khalifa et al. 2025; Wang et al. 2024d; Xu et al. 2026]

over reinforcement learning trained on scalar rewards alone, and Text2Grad [Wang et al. 2025h] converts free-form critiques into span-level rewards, a construction we analyze as a credit-assignment method in §8. The choice among bands is conditional rather than absolute. Feedback-conditioned methods suit settings where critiques are abundant and trustworthy; self-improvement suits verifiable domains where a cheap filter exists; trained critics pay off when a frozen judge is the bottleneck; process supervision and preference shaping suit large-scale pipelines that can tolerate compression in exchange for throughput. The online-offline axis compounds each of these choices, because off-policy verbal feedback ages as the policy moves, exactly as off-policy preferences do. We assemble these observations into an explicit decision procedure in the practitioner guide of §13.

7 The Verbal Reward Stack

The learning-signal pillar (§6) examined how verbal feedback is compressed into parameter updates. This section asks the prior question: where does the reward itself come from? Training-time VRL has converged on four reward constructions, and we argue they are best read as a single progression: verifiable outcome rewards, rubric rewards, generative reward models, and process reward models (PRMs). Two properties order the rungs. The first is how much linguistic structure participates in computing the reward. The second, which we call the *scalarization boundary*, is the point in that computation at which language collapses into the scalar the optimizer consumes. Ascending the stack moves the boundary later in the pipeline and attaches the signal at finer granularity, at rising verification cost. Table 3 summarizes the progression; §7.5 makes the boundary explicit and traces its consequences.

7.1 Verifiable Outcome Rewards

The bottom rung is reinforcement learning with verifiable rewards (RLVR), a term introduced by the Tulu 3 post-training recipe, which combined supervised fine-tuning, preference optimization, and RL against programmatic correctness checks in a fully open pipeline [Lambert et al. 2024a]. DeepSeek-R1 supplied the flagship result: pure RL against verifiable outcomes, with no human-labeled reasoning trajectories, produced emergent self-reflection and verification behavior and outperformed counterparts trained by supervised learning on human demonstrations across verifiable

mathematics, coding, and STEM tasks [Guo et al. 2025b]. In the vocabulary of §2, RLVR is the degenerate case of the stack: scalarization happens at the environment itself, and no linguistic structure enters the reward. We include it because it is the baseline every higher rung generalizes, and because its limitations motivated the climb. Verifiable rewards are sparse in agentic settings; Agent-RLVR densifies them with teacher-style verbal guidance, plans and error feedback attached to failed attempts, lifting pass@1 on SWE-Bench Verified from 9.4% to 22.4%, and to 27.8% with a reward model trained on the guided data [Da et al. 2025]. Verifiable rewards also reach only domains with checkable answers, the gap the rubric rung addresses. Even within their domain, measured gains deserve caution: a position paper on RLVR's hidden costs shows that budget mismatches, attempt inflation with calibration drift, and benchmark contamination confound many headline results, several of which shrink substantially or disappear under budget-matched reproduction [Wu et al. 2025a]. We revisit this measurement problem in §11; it matters here because every rung above inherits RLVR's optimization machinery, and often its evaluation habits.

7.2 Rubric Rewards

Rubric rewards extend verifiability to open-ended domains by replacing the ground-truth check with instance-specific criteria that a judge evaluates and an aggregation step pools into a scalar. Rubrics as Rewards is the anchor method: scoring rollouts against per-instance rubrics yields relative gains of up to 31% on HealthBench and 7% on GPQA-Diamond over LLM-as-judge baselines that assign direct Likert scores, and the structured criteria let smaller judges score more consistently [Gunjal et al. 2025]. Checklist feedback makes the same move for instruction following: per-instruction criteria scored by AI judges and verifier programs form an RL reward that is the only method in its comparison to improve on every benchmark, including a four-point gain in hard satisfaction rate on FollowBench [Viswanathan et al. 2025].

The immediate bottleneck is rubric supply. One response is scale: a bank of over 10,000 human-, LLM-, and jointly authored rubrics anchors RL for open-ended tasks, improving open-ended benchmarks by 5.2% from only 5K+ samples and outperforming a 671B model by 2.4% [Huang et al. 2025e]. Another is synthesis: coarse-to-fine generation produces a roughly 110k-example multi-domain rubric dataset whose post-trained 14B model reaches 69.3 on HealthBench, a result its authors report as surpassing proprietary frontier models [Li et al. 2026b], while contrastive generation derives hard rules and principles from preference pairs, beating size-matched reward-model baselines by 8.4% [Liu et al. 2025d]. A third response makes rubrics dynamic, since static criteria lag the evolving policy and invite exploitation: OnlineRubrics elicits criteria during training from pairwise comparisons between current and reference policies, gaining up to 8% over static rubrics [Rezaei et al. 2025]; EvoRubric folds rubric generation into the policy itself behind a multi-level verification pipeline [Guan et al. 2026]; a self-rewarding variant uses the policy as its own grader, with a 32B model trained on 4,000 samples exceeding GPT-5 on HealthBench Hard [Ye et al. 2025]; and case-conditioned rubrics raise a 4B model's HealthBench-Hard score from 7.0 to 27.5 with 2k samples of medical dialogue [Wang et al. 2025e]. Verification cost is the remaining axis: rather than checking every implicit criterion, an adversarial critic can propose only the likeliest violations for an external validator to check, outperforming exhaustive verification with fewer verifications [Wu et al. 2025b]. Rubrics also repair outcome-reward pathologies inside verifiable domains: outcome-only RL rewards "Miracle Steps," correct answers reached without valid derivations, whereas a process-oriented rubric reward raises Verified Pass@1024 on AIME2024 from 26.7% to 62.6% while cutting Miracle Step incidence by 71% [Yuan et al. 2025b].

The closest prior survey treats rubrics as a framework recurring across evaluation, RL, and safety, organized into evaluative, training, and intrinsic levels [Chen et al. 2026b]. Our reading is complementary but different: we place rubrics as one rung of a reward stack ordered by scalarization and granularity, which exposes what they share with the

verifiers below and the generative judges above. For this rung, scalarization occurs at aggregation: linguistic structure survives through criterion checking, but once item verdicts are pooled, anything the rubric failed to anticipate is invisible to the training signal, a slack we return to in §7.5.

7.3 Generative Reward Models

The third rung verbalizes the reward computation itself. Generative verifiers recast reward modeling as next-token prediction: the verifier writes a verification chain of thought before its verdict, inheriting instruction following and majority voting over sampled rationales, and improves best-of-N accuracy from 5% to 45.3% on algorithmic tasks and from 73% to 93.4% on GSM8K [Zhang et al. 2025g]. DeepSeek-GRM extends this to generalist reward modeling through self-principled critique tuning, online RL that teaches the model to generate principles and then critiques, and shows that sampling more reward computations at inference, guided by a meta reward model, can outperform scaling the reward model's size at training time [Liu et al. 2025b]. A growing cluster treats verdict production as a reasoning task in its own right: chain-of-rubrics reward models self-generate rubrics or reference solutions before judging, outperforming far larger open and proprietary models by up to 4.9% [Chen et al. 2025a]; reward reasoning models spend adaptive test-time compute on hard judgments [Guo et al. 2025a]; J1 converts judgment itself into a verifiable-reward RL problem [Whitehouse et al. 2025]; EvalPlanner decouples evaluation planning from execution, reaching 93.9 on RewardBench [Saha et al. 2025]; and further variants optimize judgment-thinking traces with offline and online RL [Huang et al. 2025b], extend judges to long-horizon reasoning [Hong et al. 2025b], or pair a reasoning reward model with a dedicated benchmark (§11) [Zhou et al. 2025b].

Supervision for the judges is increasingly self-generated. Self-taught evaluators iterate without any human preference labels, improving a 70B judge from 75.4 to 88.3 on RewardBench [Wang et al. 2024c]; jointly predicting critiques and scalar rewards improves reward accuracy by 3.7%–7.3% [Yu et al. 2024]; self-training on unlabeled data yields foundation reward models that emit labels with rationales [Wang et al. 2025f]; and meta-rewarding has the model judge its own judgments, the recursive rung of the stack, lifting the AlpacaEval 2 win rate from 22.9% to 39.4% [Wu et al. 2024]. The critique, however, is not automatically load-bearing: generative reward models trained only on verdict correctness learn to guess right outcomes from unsound critiques, and supervising the critique itself, by rewarding similarity to human critiques, outperforms outcome-only training [Wang et al. 2026c], closing the loop between the reward layer and the human verbal feedback of §6. For this rung, scalarization occurs at verdict emission: the verbal computation demonstrably improves the verdict and leaves an auditable trail, but the optimizer still consumes a scalar unless the critique is separately routed into revision or a language-space update (§9).

7.4 Process Reward Models

The fourth rung moves the signal inside the trajectory. PRMs descend from human step-level annotation [Lightman et al. 2024], which automation quickly replaced: Math-Shepherd constructs step labels automatically and lifts GSM8K accuracy from 77.9% to 84.1% through step-by-step PPO [Wang et al. 2024d], and OmegaPRM's divide-and-conquer tree search locates the first error by binary search, collecting over 1.5 million step annotations and improving MATH500 from 51% to 69.4% with weighted self-consistency [Luo et al. 2024]. The subsequent diagnosis was sobering. ProcessBench, an error-localization benchmark whose protocol §11 describes, finds that existing PRMs typically fail to generalize beyond GSM8K/MATH difficulty and underperform prompted critic models [Zheng et al. 2024]. A companion study explains why: Monte Carlo-estimated step labels underperform LLM-as-a-judge and human annotation because completion

models verify steps inaccurately, and best-of-N evaluation quietly drifts PRMs back toward outcome scoring; consensus filtering between the two estimators significantly improves performance and data efficiency [Zhang et al. 2025l].

Generative PRMs are where the judge line and the step line converge. ThinkPRM verifies every step by generating a verification chain of thought, outperforming discriminative verifiers with only 1% of the PRM800K labels [Khalifa et al. 2025]; StepWiser reframes stepwise reward modeling from classification to reasoning, training the judge with RL from rollout outcomes [Xiong et al. 2025]; and DG-PRM maintains a tree of fine-grained reward criteria selected dynamically per step [Yin et al. 2025]. SPARK closes the loop for domains without verifiable answers: synthetic step-level verification data trains a PRM that then drives RL, beating ground-truth RLVR at 47.4% versus 43.9% averaged over six mathematics benchmarks [Rahman et al. 2025], and OmegaPRM-style pipelines now extend to multimodal reasoning with over 700k automatically generated step annotations [Du et al. 2025]. At the finest granularity, response-level rubrics can be converted into token-level credit through a relevance discriminator [Xu et al. 2026], at which point the reward stack meets the credit-assignment problem of §8.

7.5 The Scalarization Boundary

We can now state the section's organizing claim plainly. Every rung ends the same way: a scalar crosses the interface to the optimizer. What distinguishes the rungs is where the collapse happens. Verifiers scalarize at the environment; rubrics at aggregation, after criterion-level checking; generative reward models at verdict emission, after a full verbal computation; PRMs per step; token-level methods per token. What is lost at the boundary is everything the verbal computation established that a magnitude cannot carry: which failure occurred, what would repair it, and why the trajectory earned its score. The gradient sees that a rollout scored 0.3, not that it scored 0.3 because its third step asserted an unsupported lemma. Ascending the stack narrows the loss by shrinking the span each scalar summarizes, but only the language-space optimizers of §9 remove the boundary altogether, applying feedback as text without collapsing it. In the terms of §2, rubric scores and PRM verdicts are verbal reward functions, language occupying the role of the reward function in the underlying decision process; the practitioner question of which rung to adopt, a tradeoff between signal granularity and verification cost, is taken up in §13.

The boundary is also where reward hacking concentrates, because whatever the verbal computation understood but the scalar cannot transmit becomes exploitable slack. The evidence spans every rung. Outcome rewards breed Miracle Steps [Yuan et al. 2025b], and static rubrics are gameable by design [Rezaei et al. 2025]. A systematic study of rubric-based RL locates the exploitation in precisely this slack: when rubrics leave failure modes unspecified, rubric-based verifiers prefer the trained checkpoint while rubric-free judges prefer the base model [Mahmoud et al. 2026], a finding set §12 examines in full. One rung up, generative reward models guess correct verdicts from unsound critiques [Wang et al. 2026c]; one rung further, mis-estimated step labels corrupt PRM training [Zhang et al. 2025l]; and at the base, headline RLVR gains partly dissolve under budget-matched evaluation [Wu et al. 2025a]. The emerging defenses sit, tellingly, at the boundary itself: cross-family judge panels with a verifier-free self-internalization diagnostic [Mahmoud et al. 2026], meta-verification and peer consensus over self-generated rubrics [Guan et al. 2026], format constraints on PRM-driven RL [Rahman et al. 2025], and consensus filtering of step labels [Zhang et al. 2025l]. Whether these patches suffice, or whether the boundary itself must move, is the safety question of §12.

8 Credit Assignment and Optimization with Verbal Signals

Credit assignment, deciding which parts of an agent's behavior deserve blame or praise for a delayed outcome, is where verbal reinforcement learning (VRL) meets the oldest machinery of classical RL. Language complicates the problem and

Table 4. Families of credit-assignment mechanisms in VRL, ordered roughly by how much linguistic content the credit carrier consumes. The first three families redistribute or attribute a scalar; the later families derive credit from language itself.

Family	Credit carrier	Granularity	Representative work
Temporal abstraction	scalar reward over macro actions or turns	span, turn	[Chai et al. 2024; Wei et al. 2025b; Zhou et al. 2024c]
Reward redistribution	holistic scalar re-allocated after the fact	token or time step	[Li et al. 2024d; Qu et al. 2024]
Model-internal attribution	explainability scores or attention flow	token	[Dong et al. 2026a; Koo et al. 2025]
Information-theoretic	belief update about the final answer	turn	[Wang et al. 2025a]
Generative process critique	step-wise critique converted to credit	step, token	[Liu et al. 2025a; Wang et al. 2025h; Xie et al. 2025b; Zhou et al. 2024d]
LLM-as-annotator	subgoal and action judgments by an external LLM	action, subgoal	[Lin et al. 2025b; Pignatelli et al. 2024; Zhong et al. 2024]
Memory-conditioned	credit assigned to memory operations	memory write	[Yan et al. 2026; Zhou et al. 2025a]

offers a distinctive answer to it. It complicates the problem because the natural unit of LLM behavior is a long token sequence, often stretched across a multi-turn interaction, over which a single outcome reward arrives late and sparse; a thousand-token derivation rewarded only at its final answer is a textbook delayed-reward setting. It offers an answer because a verbal signal is the only feedback modality that can state where an error lies: a critique that names the faulty step carries localization information that no scalar carries. This section classifies credit-assignment methods along two axes, the granularity at which credit lands (outcome, step, turn, token, or memory operation) and the carrier of that credit (a redistributed scalar, a model-internal attribution, or critique text), then examines how each feedback form constrains the choice of optimization algorithm, and closes with an honest accounting of the thin theoretical support. Table 4 summarizes the resulting families.

8.1 Outcome, Process, and Reflective Credit

Three levels of credit recur across the literature. *Outcome credit* scores only the final result of a trajectory. It is cheap and, for verifiable tasks, exact, which is why it anchors critic-free group-comparison methods that normalize an outcome reward across a group of sampled rollouts [Guo et al. 2025b]; but broadcasting one scalar uniformly over every token is the crudest assignment possible. *Process credit* judges intermediate steps. Uesato et al. [2022] contrasted process-based and outcome-based feedback on math word problems, and Lightman et al. [2024] established step-level verification as the training target for process reward models (PRMs), which enable finer-grained credit than any outcome signal. *Reflective credit* is the distinctly verbal level: the linguistic content of an evaluation ℓ, not merely its scalarization $\sigma(\ell)$ in the notation of §2, determines where and how the update lands. Shinn et al. [2023] introduced the inference-time form, in which stored self-reflections steer subsequent attempts (§5). Training-time analogues now exist. Text2Grad aligns free-form critiques with the token spans they indict and backpropagates span-level rewards, so the update targets the offending portion of the policy [Wang et al. 2025h]. Feedback-conditional policies go further and refuse to apply σ at all, treating the feedback text as a conditioning variable learned by maximum likelihood on response-feedback pairs, on the argument that compressing nuanced feedback into a scalar discards its richness and induces scale imbalance [Luo et al. 2025]. The trichotomy matters because it fixes what a method can, in principle, learn: outcome credit can rank trajectories, process credit can rank steps, and only reflective credit can say what to change.

8.2 Token- and Step-Level Redistribution

Within a single response, the field has produced four mechanisms for pushing a sparse reward down to tokens, which differ in how much verbal content they consume. A caveat governs every figure quoted in this subsection and the next two: each headline number and each outperformance claim is the cited paper's own report, obtained on that paper's benchmarks against that paper's baselines. No shared benchmark or overlapping baseline set spans these methods, so the numbers show that a mechanism helped in its authors' setting; they support no ranking across methods. The first mechanism coarsens the action space instead of refining the reward. MA-RLHF performs policy-gradient updates over macro actions, sequences of tokens or higher-level constructs, shrinking the temporal distance between action and reward without extra training or inference compute; its authors report gains of up to 30% on summarization and code generation, 18% on dialogue, and 8% on question answering, reaching parity with vanilla RLHF 1.7–2 times faster in training time [Chai et al. 2024].

The second redistributes a holistic scalar. RED re-allocates the single sequence-level reward to individual tokens using an off-the-shelf reward model, with no modification to the reward model and no additional training stage [Li et al. 2024d]. LaRe generalizes redistribution to episodic settings through a latent, multi-dimensional performance evaluation constructed from LLM-generated executable code, with a supporting analysis showing that eliminating reward-irrelevant redundancy improves reward estimation; on some tasks the resulting policies outperform those trained with ground-truth rewards [Qu et al. 2024].

The third attributes credit from model internals. Koo et al. [2025] estimate per-token rewards from SHAP and LIME attributions over the reward model and couple Bayesian optimization of the shaping function with policy training; notably, they show that feature-additive attribution functions preserve the optimal policy of the original reward, one of the few optimality statements in this literature. FlowTracer instead builds an attention-induced graph over tokens, scores each token by the information flow it routes toward the answer, and uses these importances to shape token-level rewards, finding that global flow structure outperforms point-wise internal heuristics [Dong et al. 2026a].

The fourth family is where VRL earns its name: credit is derived from generated critique. CAPO prompts an off-the-shelf LLM to act as a generative PRM that emits all step-wise critiques in one pass, converts them into deterministic token-level credits that refine the uniform rule-based reward of reinforcement learning with verifiable rewards, and votes over multiple critiques for robustness; it needs no auxiliary value model or PRM training and outperforms supervised and RL fine-tuning baselines across four mathematical and three out-of-domain benchmarks [Xie et al. 2025b]. T-REG has the policy self-generate token-level rewards through contrastive prompting and uses them to regularize how the sequence-level preference reward distributes over tokens, avoiding both a trained credit model and external annotators, with self-reported gains of up to 3.8% on AlpacaEval 2 and 4.4% on Arena-Hard [Zhou et al. 2024d]. iStar derives implicit step rewards by alternately optimizing an implicit PRM with the policy through a trajectory-based direct-preference objective, requiring neither extra rollouts nor step labels, with an analysis showing the objective yields a step-wise reward function [Liu et al. 2025a]. Reading down these four mechanisms, verbal content buys error localization without training a dense reward model; the price is trust in the critic, whose failure modes we treat in §7. Because no head-to-head evaluation exists, our comparative guidance rests on what each mechanism requires rather than on the self-reported gains. Temporal abstraction is the cheapest first move when a trusted sequence-level reward already exists, since it changes neither the reward nor the model. Redistribution and model-internal attribution serve the same setting at finer granularity; attribution is preferable when the credit itself must be inspectable, redistribution when the reward model is the only artifact one is willing to trust. Critique-derived credit is the only family that can

localize an error it can also name, and it becomes the default exactly when a critic strong enough to be trusted on the task is available.

8.3 Multi-Turn, Hierarchical, and Memory-Conditioned Credit

Multi-turn interaction stretches the horizon from tokens to episodes and introduces a two-level question: which turn helped, and which tokens within it. ArCHer answers hierarchically, running an off-policy value-based learner that aggregates reward across utterances in parallel with a within-turn token-level policy-gradient learner; its authors report about 100 times the sample efficiency of prior methods on agent tasks, improving with model scale up to 7B [Zhou et al. 2024c]. Wei et al. [2025b] study turn-level reward design directly, extending PPO [Schulman et al. 2017] and group-relative policy optimization to multi-turn variants; in their experiments, well-designed turn-level rewards outperform trajectory-level rewards with greater stability, faster convergence, and 100% format correctness across question-answering datasets. IGPO removes the external annotator entirely by defining the turn reward as the marginal increase in the policy's own probability of producing the correct answer, an intrinsic belief-update signal that needs no reward model or Monte Carlo estimation and directly targets the advantage collapse that afflicts outcome-only multi-turn RL [Wang et al. 2025a]. Benchmarks built for this regime, such as LMRL-Gym's multi-turn dialogue and text-game tasks, exist precisely because algorithm design is the bottleneck [Abdulhai et al. 2023] (§11).

When agents write to persistent memory, the memory operations themselves become the actions that need credit. In Memento, whose memory-augmented decision process §2 formalizes, all policy improvement is realized through memory rewriting and retrieval, so every gain must be credited to memory operations rather than to weight updates [Zhou et al. 2025a]. Memory-R2 exposes a subtle interaction with the optimizer: memory writes make rollouts diverge in their effective environment, violating the shared-environment assumption behind group-relative methods and biasing trajectory-level credit; its remedy pairs a global long-horizon objective with local rerollouts that compare memory operations issued from the same intermediate memory state [Yan et al. 2026]. The lesson connects back to the framework of §2: memory is state, so credit must condition on it. Across the multi-turn regime the choice again turns on requirements, not on the incommensurable self-reported figures. Turn-level reward design is the lightest intervention when individual turns have checkable outcomes; hierarchical value learning earns its added machinery when sample cost dominates and off-policy reuse is essential; intrinsic information-gain rewards fit when no reward model can be trained or afforded; and memory-conditioned credit is mandatory rather than optional once the agent writes persistent memory, because group-relative baselines are biased without it.

8.4 Language Models as Credit Annotators

A complementary family places the LLM outside the policy, as an annotator of another agent's experience. CALM has the LLM decompose a task into elementary subgoals and assess, zero-shot, whether a state-action transition achieves each subgoal, triggering an auxiliary reward at option terminations; evaluated on human-annotated MiniHack demonstrations without examples or fine-tuning, its preliminary results indicate that LLM knowledge is a promising prior for credit assignment without human-designed rewards [Pignatelli et al. 2024]. Language feedback models are trained on LLM feedback over verbalized trajectories to identify desirable actions for imitation learning; their authors report beating strong behavioral-cloning baselines and LLM-as-expert action prediction at matched token budgets, and generalizing to unseen environments with 3.5–12.0% task-completion gains after one adaptation round [Zhong et al. 2024]. In multi-agent RL, an LLM can generate dense, agent-specific rewards from a natural-language description of the team goal; learning a potential-based reward function over multiple LLM queries dampens the effect of ranking

errors and yields faster convergence and higher returns than multi-agent baselines [Lin et al. 2025b]. Group-level critique can also steer exploration rather than credit per se, as in GOLF's critique-derived off-policy scaffolds, analyzed as a learning-signal method in §6 [Huang et al. 2026]. Across this family the credit problem does not disappear; it migrates into the annotator. An LLM that mislocates errors produces systematically biased shaped rewards, which is why annotator quality and its adversarial manipulation (§12) bound what these methods can deliver. The comparative case for annotation is one of scope rather than measured magnitude: it is the natural family when the policy is not itself a language model, or when credit must be injected into an existing RL pipeline without touching its optimizer, and it should be avoided where annotator errors cannot be audited.

8.5 Interactions with the Optimization Family

Different feedback forms pair with different optimization families, and each pairing carries assumptions a practitioner must check. Executable reward functions generated from language pair with online policy-gradient training [Schulman et al. 2017]; pairwise judgments pair with offline preference objectives such as direct preference optimization [Rafailov et al. 2023]; filtered or feedback-conditioned text pairs with supervised fine-tuning [Luo et al. 2025]; and step-level verbal judgments pair with PRMs, which carry the finest-grained credit [Lightman et al. 2024]. Critic-free group comparison [Guo et al. 2025b] adds its own assumption, that rollouts in a group share an environment and a reward scale, and both memory writes [Yan et al. 2026] and turn structure [Wei et al. 2025b] break it in instructive ways.

Each pairing also has consequences inside the decision problem. Rewards specified or judged through language inherit objective mis-specification and reward-hacking risks (§12). States represented through language raise information-loss and aliasing questions: whether a verbal summary preserves the statistics that optimal control requires is rarely tested. Actions generated under LLM priors change exploration: naive execution of planner-imitated subgoals incurs linear regret unless explicit exploration is layered on top, per the hierarchical regret analysis stated in §2 [He et al. 2024]. The practitioner guidance of §13 fixes the feedback form first, keyed to properties assessable in advance, and treats the optimizer as a downstream consequence of that choice.

8.6 The State of Theory

The empirical literature above is almost entirely guarantee-free, and we state this plainly. The one formal treatment of the general problem, the learning-from-language-feedback formalization and its transfer eluder dimension, is presented in §2 [Xu et al. 2025c]. Around this anchor sit partial results: the regret analysis of language-mediated hierarchies [He et al. 2024], optimality preservation under feature-additive shaping [Koo et al. 2025], the redundancy-reduction argument behind latent rewards [Qu et al. 2024], and the step-wise reward derivation underlying implicit PRMs [Liu et al. 2025a]. What is missing is any account of whether critique-derived token credits preserve the optimal policy of the underlying outcome reward, or of when a verbal annotator's localization errors stay benign rather than compounding through training. A dedicated survey of credit assignment for LLM reinforcement learning, covering 47 methods from 2024 to early 2026, reads the field as maturing around PRMs and critic-free group comparison on the reasoning side, while agentic RL generates genuinely new mechanisms, including hindsight counterfactual analysis, privileged asymmetric critics, and turn-level MDP reformulations [Zhang 2026]. We fold the resulting open problems, sample-efficiency conditions under which verbal feedback beats scalar reward, optimality under language-specified rewards, and temporal credit from free-form critique, into the research agenda of §13.

Table 5. The optimization analogy that organizes language-space methods. Each element of numerical optimization has a textual counterpart realized by existing systems.

Numerical concept	Language-space counterpart	Representative realizations
Parameters	Prompt, evolving context, or curated playbook	VML [Xiao et al. 2024a]; ACE [Zhang et al. 2025h]; token priors [Cai et al. 2025]
Gradient	Natural-language critique of the current candidate	TextGrad [Yuksekgonul et al. 2024]; directional feedback [Nie et al. 2024]
Update rule	LLM edit operator applied to the text	OPRO proposal step [Yang et al. 2024b]; GEPA reflective mutation [Agrawal et al. 2025]
Backpropagation	Feedback routed backward through the computation graph	Trace/OPTO [Cheng et al. 2024b]; semantic backpropagation [Wang et al. 2024a]; LLM-AutoDiff [Yin and Wang 2025]
Momentum, curvature	History of responses and scores across iterations	REVOLVE [Zhang et al. 2024a]; OPRO score history [Yang et al. 2024b]
Ensemble	Population of candidate prompts under selection	PromptBreeder [Fernando et al. 2023]; EvoPrompt [Guo et al. 2024]; GEPA Pareto frontier [Agrawal et al. 2025]

9 Language-Space Optimizers

The improvement operator of §2 admits three realizations: an output revision that vanishes with the episode, a memory write that persists across episodes without touching the weights, and a parameter update. A rapidly growing family of methods pushes the middle realization past the agent-written stores of §5: the persistent text is a prompt, an accumulated context, a curated playbook, or the program that orchestrates a compound system, and it is written not by the agent during its episodes but by an outer optimizer that curates it against a task metric. By the assignment rule of §3, such a curated text is a parameter of the system, and this chapter is synthesis rather than a fourth pillar: like §7 and §8, it follows verbal signals into the optimization machinery that consumes them. Verbalized machine learning states this position most directly: an LLM equipped with a prompt is a function parameterized by that prompt, and a second LLM acts as the optimizer that verbally updates the learner, explaining each update as it goes [Xiao et al. 2024a]. The scaffolded-language-model view generalizes this into a semi-parametric account in which the frozen model is the parametric component and prompts, tools, and scaffold code are non-parametric variables trained by an optimizer that interprets language supervision [Lin et al. 2024b]. Trace supplies the sharpest formal object to date, Optimization with Trace Oracle (OPTO), in which the optimizer receives an execution trace together with output feedback and updates heterogeneous text parameters such as prompts and code; execution traces play the role that backpropagated gradients play in automatic differentiation [Cheng et al. 2024b]. Under this reading, the classical optimization loop survives intact but each of its elements acquires a textual counterpart, summarized in Table 5: the critique is the gradient, the prompt or context is the parameter vector, the LLM edit operator is the update rule, and a maintained population of prompts is the ensemble.

9.1 From Prompt Search to Textual Gradients

The earliest optimizers searched instruction space with only scores as feedback. APE has an LLM propose instruction candidates and select among them by measured task performance, yielding instructions competitive with human-written ones [Zhou et al. 2023]. OPRO goes further by placing the optimization trajectory itself, previous solutions paired with their scores, inside the prompt of the optimizer, so that the LLM's context window serves as the optimizer's

working memory; the resulting prompts outperform human-designed ones by up to 8% on GSM8K and up to 50% on Big-Bench Hard tasks [Yang et al. 2024b]. The evolutionary branch maintains populations instead of trajectories. EvoPrompt couples LLMs, acting as mutation and crossover operators, with classical evolutionary algorithms over a prompt population [Guo et al. 2024], and PromptBreeder makes the process self-referential by evolving the mutation prompts alongside the task prompts they improve [Fernando et al. 2023]. At the program level, DSPy abstracts pipelines into declarative modules and compiles them toward a metric by generating and selecting demonstrations [Khattab et al. 2023]; MIPROv2 optimizes instructions and demonstrations across multi-stage programs without module-level labels, and its surrogate-based attribution of program success to individual modules is the language-space analogue of the temporal credit assignment problem treated in §8 [Opsahl-Ong et al. 2024].

Textual-gradient frameworks replace scores with structured critique. TextGrad backpropagates natural-language feedback through the computation graph of a compound system, treating critiques as gradients attached to variables ranging from code to molecular structures; without modification it raises GPT-4o zero-shot GPQA accuracy from 51% to 55% [Yuksekgonul et al. 2024], and metaTextGrad meta-optimizes the optimizer's own prompts and structure [Xu et al. 2025d]. Subsequent work has refined the analogy along familiar axes. Directional feedback, which tells the optimizer which way to move rather than merely that the candidate is wrong, generalizes first-order gradient information to text and markedly stabilizes optimization [Nie et al. 2024]. REVOLVE argues that reacting only to the latest critique is analogous to using only first derivatives, and tracks how responses evolve across iterations [Zhang et al. 2024a]. Semantic backpropagation formalizes how output feedback should be distributed among graph nodes that share a successor, the cleanest statement of structural credit assignment in language space [Wang et al. 2024a], while LLM-AutoDiff extends textual gradients to cyclic, multi-component workflows [Yin and Wang 2025]. The analogy also inherits pathologies: textual gradients can explode, with feedback growing across depth, or vanish, with compression losing specificity across hops, and textual equilibrium propagation replaces global backpropagation with local critic equilibria to restore depth scaling [Chen et al. 2026a]. Beyond prompts, the same machinery optimizes entire agents: agent symbolic learning applies language losses and language gradients to prompts, tools, and pipeline structure jointly [Zhou et al. 2024a]; ADAS searches over agents expressed as code [Hu et al. 2024]; AFlow casts workflow generation as Monte Carlo tree search over code-represented workflows, letting smaller models outperform GPT-4o on specific tasks at 4.55% of its inference cost [Zhang et al. 2024b]; and MASS interleaves prompt and topology optimization for multi-agent systems [Zhou et al. 2025d].

9.2 Reflective Evolution and Training-Free Reinforcement Learning

The strongest current evidence that language space can compete with policy gradients comes from GEPA, which samples system trajectories, diagnoses failures in natural language, proposes prompt updates, and combines complementary lessons from the Pareto frontier of its own attempts. GEPA outperforms GRPO by 6% on average and up to 20% while using up to 35× fewer rollouts, and beats MIPROv2 by over 10% [Agrawal et al. 2025]. The mechanism behind the sample-efficiency gap is informational: a reflective diagnosis extracts far more from each rollout than a scalar reward does. The same logic extends to supervision cost, as self-supervised prompt optimization (SPO) derives both evaluation and optimization signals purely from pairwise output comparisons, matching state-of-the-art optimizers at 1.1%–5.6% of their cost with as few as three samples [Xiang et al. 2025]. Training-Free GRPO transplants the reinforcement-learning template itself into language space: within each group of rollouts it extracts a group relative semantic advantage, a natural-language comparison replacing the numerical advantage, and distills the experience into a token prior injected at inference time; with a few dozen training samples it improves out-of-domain mathematics and web-search performance

of a frozen model [Cai et al. 2025]. Agentic context engineering (ACE) treats the context as an evolving playbook maintained by generation, reflection, and curation, and diagnoses two failure modes of naive context rewriting, brevity bias and context collapse, that its incremental updates avoid [Zhang et al. 2025h]. These methods sit on the boundary with experiential memory: because an outer optimizer curates the context against a task metric, the context is a language-space parameter and belongs here, per the assignment rule of §3 whose memory-side statement appears in §5.

9.3 Hybrid Language-Space and Weight-Space Updates

Recent systems dissolve the boundary with parameter learning deliberately, asking where learned knowledge should live rather than which side wins. SEAL has the model generate its own finetuning data and update directives, so language is the medium of the proposal even though the effect lands in the weights [Zweiger et al. 2025]. Learning to Self-Evolve inverts the dependency by training the context editor itself with reinforcement learning, rewarding each edit by the downstream improvement it produces; a 4B editor trained this way outperforms self-evolution driven by much larger models as well as GEPA and TextGrad [Chen et al. 2026c]. P^2O alternates policy-gradient updates with GEPA-style prompt evolution to rescue advantage collapse on hard instances where every rollout fails, then distills the prompt-induced gains back into the weights, yielding up to 9.5% improvement over GRPO baselines [Lu et al. 2026]. Evolutionary system prompt learning runs the two loops jointly and observes a division of labor, declarative knowledge accumulating in prompts and procedural knowledge in weights; in an easy-to-hard AIME to BeyondAIME setting it lifts RL success from 38.8% to 45.1%, against 40.0% for reflective prompt evolution alone [Zhang et al. 2026a]. SIA extends the joint update to the entire harness, with a feedback agent revising scaffold code, prompts, and weights in one loop [Hebbar et al. 2026].

9.4 When Language-Space Optimization Wins, and How It Fails

The evidence above supports three conditions under which language-space optimization is the right tool. First, when gradient or weight access is unavailable, as with closed models served through APIs, textual optimization is the only persistent learning channel; the semi-parametric framing makes this explicit [Lin et al. 2024b]. Second, when rollouts or labels are scarce, the informational density of verbal feedback dominates: GEPA's rollout advantage over GRPO [Agrawal et al. 2025], SPO's cost profile [Xiang et al. 2025], and Training-Free GRPO's few-dozen-sample regime [Cai et al. 2025] all instantiate this. Third, when the learned artifact must be inspected, audited, or edited by humans, a prompt or playbook is interpretable in a way a weight delta is not, and editing it neither risks catastrophic forgetting nor requires retraining [Lin et al. 2024b; Xiao et al. 2024a]. Parameter updates retain the advantage when the target knowledge is procedural rather than declarative [Zhang et al. 2026a], when abundant data amortizes training cost, when inference budgets cannot absorb ever-longer optimized contexts, and when prompt-carried gains must be internalized for deployment, the role context distillation plays in P^2O [Lu et al. 2026].

The failure modes are equally well documented and temper the optimism. Controlled analysis shows that reflective optimizers frequently misidentify the true causes of errors, defaulting to their prior knowledge rather than genuinely analyzing failures, so the "gradient" can point in the wrong direction [Ma et al. 2024c]. The consequences compound: starting from a defective seed prompt, GEPA degrades GSM8K accuracy from 23.81% to 13.50%, whereas VISTA, which decouples hypothesis generation from prompt rewriting and verifies labeled hypotheses on minibatches, recovers it to 87.57% [Liu et al. 2026e]. Multi-objective settings expose gradient dilution and instruction interference: when one critic must address several criteria jointly, gradient task-focus drops by 59%, and naively merging single-objective optimized instructions degrades judge correlation [Darshan and Divekar 2026]. Gains are also configuration-sensitive,

as systematic benchmarking across multi-agent workflows, protocols, and team sizes finds that prompt optimization sometimes helps substantially and sometimes not at all [Bai and Shi 2026]. Finally, because the critique channel is the update channel, a poisoned or adversarial critique becomes a poisoned update, an attack surface examined in §12.

Two contrast methods mark the edge of the definitional fence. GReaTer optimizes prompts with actual numeric gradients of the task loss computed over the model's reasoning, freeing small models from dependence on an expensive verbal critic [Das et al. 2024], and PromptQuine evolves pruned, seemingly incoherent prompts whose selection signal carries no linguistic meaning yet matches state-of-the-art prompt optimizers [Wang et al. 2025c]. Both optimize text, but neither learns from a verbal signal, so both fall outside VRL; their competitiveness quantifies what the verbal channel buys, namely sample efficiency and auditability rather than raw attainable performance. Relative to prior syntheses of automatic prompt optimization [Ramnath et al. 2025], compound-system optimization [Lee et al. 2025], and context engineering [Mei et al. 2025], the contribution of this section is placement: language-space optimizers realize the improvement operator of §2 as an outer loop that writes to textual parameters, pairing the frozen weights and inference-time consumption of deliberative feedback with the metric-driven, cross-episode persistence of a learning signal, governed by the same credit assignment questions as §8 and feeding the practitioner decision procedure of §13.

10 Language Feedback in Embodied and Multimodal Settings

Most methods in the preceding sections read and write text: the agent's outputs, the feedback on those outputs, and frequently the environment itself are linguistic. Embodied settings break this symmetry: observations arrive as images, video, or proprioception, actions are motor commands, and often only the feedback channel carries language. This section therefore states the modality scope of the survey explicitly, completing the definitional fence of §2: *verbal* refers to the linguistic structure of the feedback signal, not to its carrier, which may be typed text, transcribed speech, or the language half of a multimodal exchange. What qualifies a signal for verbal reinforcement learning (VRL) is that its compositional linguistic content, a phrase such as "move a bit to the left" or a paragraph explaining why a grasp failed, is functionally consumed by the learner rather than collapsed to a scalar before use. A vision-language model (VLM) that watches video thus produces a verbal signal when its judgment is expressed and consumed as language, and leaves scope when only a score survives. The exclusions of §2 likewise carry over: language-conditioned control, where an instruction is an input to the policy rather than feedback on its behavior, remains outside VRL no matter how the instruction is delivered. Prior work has organized LLM- and VLM-integrated reinforcement learning by the role the foundation model plays, as agent, planner, or reward [Schoepp et al. 2025]; we organize this section instead by what the language does to the embodied learner: correct a policy, define a reward, judge behavior, or explain a failure. These four roles enter the underlying decision problem at different points, a distinction we return to at the close of the section.

10.1 Language Corrections on Real Robots

The most direct embodied verbal signal is a human watching a robot and speaking a correction. LILAC maps language onto a learned low-dimensional control space that the human then drives, so each real-time utterance ("no, to the right") refines the shared control space during execution rather than between discrete turns; in a user study on a Franka Emika Panda, this corrections-aware shared autonomy achieved higher task completion than open-loop instruction following, and users preferred it [Cui et al. 2023]. DROC extends what a correction may address: an LLM-based system accepts arbitrary online corrections, from high-level plan failures to low-level skill-parameter adjustments, and distills them into a knowledge base retrieved by textual and visual similarity on novel objects and tasks [Zha et al. 2023]. It needs roughly half the corrections of direct LLM code-generation baselines in the first round and little to none after two iterations,

and because distilled corrections persist across episodes, DROC is a worked example for the hybrid-case decision rules of §3: a correction that becomes memory. Yell At Your Robot closes the loop into training: a high-level policy indexes into language-conditioned low-level skills, incorporates fine-grained corrections in real time, and folds the logged corrections into iterative retraining, so an inference-time verbal signal becomes a learning signal (§6); on real hardware this yields significant improvement on long-horizon dexterous manipulation without additional teleoperation [Shi et al. 2024a]. RT-H makes the intervention interface itself linguistic by inserting fine-grained language motions between task description and low-level action, the mechanism introduced in §4; humans correct execution in the vocabulary of those motions, and learning from language interventions outperformed learning from teleoperated interventions [Belkhale et al. 2024]. RACER shows the correcting human can be replaced: a VLM supervisor produces rich language guidance for error recovery online, paired with a language-conditioned visuomotor policy trained on failure-recovery-augmented demonstrations, and outperforms a strong imitation baseline on RLBench in simulation and the real world [Dai et al. 2024]. The environment side of this loop was established by Inner Monologue (§4), whose verbalized success detection, scene description, and human responses significantly improved instruction completion across three domains including real-robot manipulation without any training [Huang et al. 2022b].

10.2 Language to Reward

When a simulator exposes state, language need not steer the policy directly; it can write the reward. The quantitative record of this line is given in §4: Language to Rewards compiles instructions and corrections into reward parameters that a real-time optimizer solves [Yu et al. 2023], Text2Reward generates dense reward code refinable through human feedback [Xie et al. 2024], and Eureka evolves reward programs by verbalizing training statistics as reflection and mutating the code [Ma et al. 2024a]. What the embodied setting adds is reach: Text2Reward's simulator-trained policies transferred to the real world [Xie et al. 2024], and DrEureka pushes the recipe through sim-to-real by generating both the reward and the domain-randomization configuration, discovering setups competitive with human-designed ones, including a quadruped balancing on a yoga ball [Ma et al. 2024b]. Because these methods act at problem-definition time, language-to-reward sits at the head of the verbal reward stack (§7), is a standing concern for the reward-hacking analysis of §12, and, through Eureka's reflective mutation, doubles as a language-space optimizer (§9).

10.3 Vision-Language Reward Models and Critics

When only pixels are available, a learned judge replaces the written reward. RL-VLM-F learns a reward from VLM preference labels over pairs of image observations rather than from raw scores, a design point §4 motivates, and produces effective policies across control and manipulation without human supervision [Wang et al. 2024f]. VLM-RL converts contrasting language goals into dense semantic rewards for autonomous driving, reducing collision rate by 10.5% and increasing route completion by 104.6% over baselines in CARLA [Huang et al. 2024b]. VLA-RL fine-tunes a VLM into a robotic process reward model, bringing process reward models (PRMs) into the embodied loop: its pseudo-labels over automatically extracted task segments combat sparse reward, lifting OpenVLA-7B by 4.5% over the strongest fine-tuned baseline on 40 LIBERO tasks [Lu et al. 2025]. RoboReward addresses the data problem: robot corpora are success-heavy, so it augments them with calibrated failures, finding that no existing open or proprietary VLM excels across reward-assignment tasks while its own 4B/8B models outperform much larger VLMs on short-horizon tasks and improve real-robot policy learning [Lee et al. 2026]; its benchmark half feeds the embodied row of Table 6 in §11. Robo-Dopamine learns a step-aware general process reward model from multi-view input and pairs it with policy-invariant shaping so dense rewards accelerate learning without changing the optimal policy, a direct embodied instantiation of the

credit-assignment machinery of §8; after one-shot adaptation from a single expert trajectory, it improved a policy from near zero to 95% success within 150 online rollouts [Tan et al. 2025a]. Large Reward Models emit process, completion, and temporal-contrastive rewards zero-shot in unseen environments, improving an imitation-learned policy within 30 RL iterations [Wu et al. 2026a], and SOLE-R1 makes the verbal structure explicit: per-timestep chain-of-thought reasoning over raw video and a language goal yields dense progress estimates that serve as the sole reward, enabling zero-shot online RL from random initialization on 24 unseen tasks while proving markedly more robust to reward hacking than strong vision-language rewarders [Schroeder et al. 2026]. Two results counsel caution. On ViSTa, evaluated VLMs recognized objects well but failed to understand task sequence, with only GPT-4o achieving non-trivial performance [Wybitul et al. 2024], and MotIF shows that single-frame success detectors fail whenever success depends on the motion rather than the final state, a gap closed by fine-tuning on trajectory-overlaid representations [Hwang et al. 2024]. Both datasets join RoboReward's benchmark as the embodied quality instruments of §11. The formal reading is that the reward function is replaced by a learned judge operating under partial observability with its own error modes, which makes judge evaluation (§11) and hacking robustness (§12) first-order design criteria rather than afterthoughts.

10.4 Failure Reasoning as Verbal Feedback

A complementary line treats the explanation of failure, not the reward, as the verbal product. AHA frames failure detection as free-form language reasoning by a VLM fine-tuned on procedurally perturbed demonstrations; it surpassed GPT-4o in-context learning by 10.3%, and plugging its failure explanations into reward refinement, task planning, and sub-task verification boosted downstream success by 21.4% on average across three frameworks [Duan et al. 2024]. FailSafe pairs generated failures with executable recovery actions, and its fine-tuned VLM improved three state-of-the-art vision-language-action models by up to 22.6% on average [Lin et al. 2025a]. Guardian scales the approach across environments by synthesizing failures from both simulated and real trajectories with structured step-by-step reasoning traces, reaching state-of-the-art failure detection and consistently raising task success when placed in the loop of a manipulation policy [Pacaud et al. 2025]. These critics instantiate the critique signal of §6 in a setting where the critique must ultimately be grounded back into motor action.

10.5 What Language Feedback Teaches Embodied Learners

Controlled evidence on the verbal channel itself is beginning to appear. Rather than proposing a mechanism, Xi et al. [2024a] treat the language as the experimental variable, manipulating its informativeness (hindsight commentary versus foresight guidance) and its diversity of expression across four benchmarks: agents trained with diverse and informative feedback generalize better and adapt faster to new tasks. Where §4 reads this study as evidence about the grounding gap, its lesson here is prescriptive: the phrasing regime of a correction or critique interface is a design lever on par with the choice of mechanism, not an incidental property of whoever supplies the feedback.

Seen through the framework of §2, the four mechanisms of this section enter the decision problem at different points, with different consequences. Corrections and failure explanations act on the policy side: they arrive during or after execution, leave the objective untouched, and demand grounding machinery that maps a phrase to a change in motor behavior. Language-to-reward acts on the objective side before any learning begins, so its failure mode is mis-specification rather than misinterpretation. VLM reward models occupy the riskiest position, substituting a learned, partially observed judge for the reward function inside the training loop, which is why the hacking-robustness and sequential-understanding results above matter disproportionately. These positions compose into a practitioner-facing decision axis, developed fully in §13: human corrections suit settings with a person in the loop, reward generation suits

simulators that expose state, VLM reward models suit pixel-only observation, and failure-reasoning critics suit policies that must recover autonomously. The broader lesson is that the taxonomy survives the change of modality: corrections operate as deliberative feedback, generated rewards as grounding signal, and VLM judgments as learning signals, with only the carrier of the language, and the grounding burden it must overcome, changing between text and the physical world.

11 Evaluation and Benchmarks

A survey of feedback mechanisms is incomplete without an account of how the field measures them. This section takes stock of the benchmark ecosystem that has grown around verbal reinforcement learning (VRL) and organizes it around a distinction that recurs throughout this survey: feedback *quality*, whether a signal is accurate, calibrated, and unbiased, and feedback *utilization*, whether the receiving model actually improves when the signal is supplied. The distinction matters because VRL inherited its evaluation habits from RLHF, where reward-model accuracy on held-out preference pairs stood in for signal quality and end-task performance stood in for everything else; the benchmark families reviewed here decompose that middle ground. The two axes are separable failure modes. A critique can be correct yet ignored, and a reward model can score well on an accuracy benchmark that barely correlates with the policies it later trains [Kim et al. 2024]. Benchmark coverage also maps onto the signal families developed in §3 and §7: outcome preference, judge verdict, natural-language critique, process supervision, and interactive language feedback. Table 6 indexes the major families by what they measure, the signal type they probe, and the stage at which that signal operates.

11.1 Measuring Feedback Quality

RewardBench [Lambert et al. 2024b] established the template for evaluating outcome-preference signals: prompt-chosen-rejected trios spanning chat, reasoning, and safety, scored on a public leaderboard, with findings on refusal propensity and reasoning limitations that frame reward-model evaluation as a window into which values RLHF embeds. Its successors harden the template. RewardBench 2 [Malik et al. 2025] builds from new human prompts rather than prompts recycled from downstream evaluations; models score about 20 points lower on average while the benchmark remains highly correlated with best-of-N selection and PPO training outcomes. RM-Bench [Liu et al. 2024] separates sensitivity to subtle content changes from susceptibility to style: state-of-the-art reward models average only 46.6% under style-bias interference, below the 50% random level. RewardMATH [Kim et al. 2024] shows that the math subset of the original benchmark, built on single chosen-rejected comparisons, shows almost no correlation with optimized-policy results, while its one-to-many redesign correlates strongly and can estimate reward overoptimization. RMB [Zhou et al. 2024e] widens coverage to more than 49 real-world scenarios with pairwise and best-of-N protocols, and Preference Proxy Evaluations [Frick et al. 2024] validate proxy metrics against an end-to-end RLHF experiment. The methodological throughline is blunt: an accuracy number on preference pairs means something only insofar as it predicts downstream utility. Personalized RewardBench [Ma et al. 2026] extends the question to pluralistic alignment, where state-of-the-art reward models peak at 75.94% accuracy on user-specific rubrics.

A parallel line meta-evaluates LLM judges, whose verdicts serve both as evaluation and as reward (§7). LLMBar [Zeng et al. 2023] curates output pairs in which the deviating response has deceptive surface appeal, and finds, contrary to earlier meta-evaluations, that evaluators differ sharply and even the best leave substantial room for improvement. JudgeBench [Tan et al. 2024] ties preference labels to objective correctness and finds strong models such as GPT-4o performing only slightly better than random guessing. For VRL these are not merely evaluation concerns: because judge verdicts are recycled as rewards (§7), measurement error becomes training signal, and a biased judge does not

Table 6. Benchmark families for verbal feedback, indexed by whether they measure feedback quality or feedback utilization, the signal type they probe, the stage at which the signal operates, and the principal gap each family exposes or leaves open. PRM abbreviates process reward model, RLVR reinforcement learning with verifiable rewards (§7), and VLM vision-language model.

Benchmark family	Measures	Signal type	Stage	Known gap
Reward-model benchmarks: RewardBench 1/2 [Lambert et al. 2024b; Malik et al. 2025], RM-Bench [Liu et al. 2024], RewardMATH [Kim et al. 2024], RMB [Zhou et al. 2024e], PPE [Frick et al. 2024]	Quality	Scalar outcome preference	Training (reward selection); inference (best-of-N)	Accuracy can decorrelate from downstream utility; style bias depresses scores
Judge meta-evaluation: LLMBar [Zeng et al. 2023], JudgeBench [Tan et al. 2024], CALM [Ye et al. 2024], MM-Eval [Son et al. 2024], Sage [Feng et al. 2025], BiasScope [Lai et al. 2026]	Quality	Judge verdict and rationale	Inference (evaluation, reranking)	Bias and preference inconsistency; near-random accuracy on correctness-grounded pairs
Critique benchmarks: CriticBench [Lin et al. 2024a], CriticEval [Lan et al. 2024b], CodeCriticBench [Zhang et al. 2025d], RealCritic [Tang et al. 2025]	Quality; RealCritic scores utilization	Natural-language critique	Inference (self- and cross-correction)	Open-loop critique ratings can diverge from corrective value
Process and verifier benchmarks: ProcessBench [Zheng et al. 2024], PRMBench [Song et al. 2025], ToolComp [Nath et al. 2025], VerifyBench [Yan et al. 2025a]	Quality	Step-level reward; verifier verdict	Training (PRM and RLVR reward); inference (step verification)	Weak generalization beyond curated math; verifier false negatives and hackability [Huang et al. 2025d]
Interactive feedback environments: MINT [Wang et al. 2023a], ConvCodeWorld [Han et al. 2025]	Utilization	Turn-level language, tool, and execution feedback	Inference (multi-turn interaction)	Feedback simulated by LLMs rather than supplied by humans
Language-centered interaction: LLF-Bench [Cheng et al. 2023], LMRL Gym [Abdulhai et al. 2023]	Utilization	Language feedback as the sole learning signal (LLF-Bench); numeric reward over text states and actions (LMRL Gym)	Inference-time interactive learning; training (multi-turn RL)	No standard metric yet for learning from language alone
Feedback incorporation: Feedback Friction [Jiang et al. 2025], JETTS [Zhou et al. 2025f]	Utilization	Near-oracle critique; judge critique at test time	Inference (refinement, test-time scaling)	Models under-absorb even accurate feedback
Agent trajectory and memory: AgentRewardBench [Lù et al. 2025], Plan-RewardBench [Wang et al. 2026b], MemoryAgentBench [Hu et al. 2025b]	Both	Trajectory-level preference; persistent verbal memory	Training (agent reward); inference (trajectory selection, cross-episode use)	Long-horizon judgment weak; no judge excels across benchmarks
Embodied feedback: ViSTa [Wybitul et al. 2024], MotIF-1K [Hwang et al. 2024], RoboReward [Lee et al. 2026]	Quality	VLM reward and success verdict over video and trajectories (§10)	Training (embodied reward); inference (success detection)	VLMs fail sequential-task understanding; single-frame detectors miss motion-dependent success; no utilization benchmark

just misrank systems but steers policies. Reliability audits go further. CALM quantifies twelve judge biases through principle-guided input perturbation [Ye et al. 2024]; BiasScope discovers biases automatically and distills them into JudgeBench-Pro, on which even powerful evaluators show error rates above 50% [Lai et al. 2026]; and Sage measures preference consistency without human labels, finding that frontier models fail to maintain consistent preferences in nearly a quarter of difficult cases [Feng et al. 2025]. Feuer et al. [2025] audit judge-based leaderboards themselves, documenting schema incoherence and factor collapse and showing that Elo-style aggregation masks genuine ranking uncertainty. Coverage is also widening: MM-Eval finds that judges strong in English have considerable room for improvement on non-English text across 18 languages [Son et al. 2024], IF-RewardBench moves from pairwise to

listwise judging via preference graphs [Wen et al. 2026], and SCAN argues that evaluation should return fine-grained verbal capability reports rather than a single rank [Wang et al. 2025b].

Critique-quality benchmarks address the natural-language critiques at the center of the deliberative pillar (§5). CriticBench [Lin et al. 2024a] evaluates generation, critique, and correction jointly across five reasoning domains, finding the three capabilities linearly related and observing that stronger models critique weaker ones better while weaker models can surpass stronger ones at self-critique. CriticEval adds human-annotated reference critiques so that textual critiques can be graded reliably [Lan et al. 2024b], and CodeCriticBench extends checklist-scored critique evaluation to code [Zhang et al. 2025d]. At the step level, ProcessBench [Zheng et al. 2024] matters here for its protocol: models must localize the earliest erroneous step in competition mathematics or declare the solution correct, a design that scores step-level judgment directly rather than through downstream accuracy and places process reward models and prompted critic models under one comparison, whose training-side implications §7 discusses. PRMBench probes finer error dimensions and uncovers significant weaknesses in fine-grained error detection [Song et al. 2025], while ToolComp carries process supervision into multi-tool trajectories, where most models score below 50% accuracy and PRMs outrank outcome reward models by 19% and 11% rank@1 when ranking base and fine-tuned trajectories [Nath et al. 2025]. Verifier benchmarks close the loop with training: VerifyBench evaluates the reference-based checkers used in RLVR for reasoning [Yan et al. 2025a], and Huang et al. [2025d] show that rule-based verifiers miss equivalent answers in different formats, producing false negatives that hurt RL more as the policy strengthens, whereas model-based verifiers are statically more accurate but highly susceptible to hacking, a finding that feeds directly into §12.

The embodied judges of §10 are acquiring quality instruments of their own. ViSTa composes over 4,000 videos with step-by-step descriptions into hierarchies of increasingly complex sequential tasks, testing whether vision-language models (VLMs) can supervise behavior that cannot be scored from the final state alone; the evaluated models recognized objects well but failed to understand task sequence, with only GPT-4o achieving non-trivial performance [Wybitul et al. 2024]. MotIF-1K, with 653 human and 369 robot demonstrations across 13 task categories, plays the same role for motion, exposing the failure of single-frame success detectors whenever success depends on how a trajectory unfolds rather than where it ends [Hwang et al. 2024]. RoboReward's benchmark half, built from success-heavy robot corpora augmented with calibrated failures, finds that no existing open or proprietary VLM excels across its reward-assignment tasks [Lee et al. 2026]. All three grade the judge rather than the receiving policy, so embodied evaluation currently occupies the quality axis alone; no benchmark yet measures whether a motor policy improves when verbal feedback is supplied, a gap §11.3 returns to.

11.2 Measuring Feedback Utilization

The deliberative pillar assumes that feedback changes behavior through the context window, within an episode or, when it persists as experiential memory, across them; the learning pillar assumes that it changes the weights themselves (§5, §6). Utilization benchmarks test these assumptions directly rather than taking them on faith: quality benchmarks score the signal, utilization benchmarks score the receiver. RealCritic makes the shift explicit by grading a critique through the correction it induces rather than through open-loop ratings; under this closed-loop metric, classical LLMs can fall below their own baselines in self-critique and iterative settings [Tang et al. 2025]. JETTS evaluates judges in the operational roles they occupy during test-time scaling and finds them competitive with outcome reward models for reranking but consistently worse than PRMs in beam search, with natural-language critiques that are currently ineffective at guiding generators to better responses [Zhou et al. 2025f].

Interactive environments treat feedback as a controlled experimental variable. MINT gives an early quantification of the marginal value of verbal feedback in multi-turn problem solving: tools add 1–8% absolute performance per turn and simulated natural-language feedback adds 2–17%, while better single-turn performance does not guarantee better multi-turn performance [Wang et al. 2023a]. ConvCodeWorld crosses compilation, execution, and verbal feedback of varying expertise into nine reproducible code-repair scenarios; performance varies significantly with feedback type, weaker models given sufficient feedback can exceed the single-turn results of stronger ones, and a static replay variant preserves Spearman rank correlations of 0.82–0.99 with the live environment [Han et al. 2025]. LLF-Bench, whose formal role §2 discusses, is the purest instrument on the utilization axis: with instructions and post-action language feedback as the only learning signal, improvement across attempts measures feedback uptake directly, and paraphrased, randomized verbalizations rule out the competing explanation that the agent is recognizing a familiar task rather than reading the feedback [Cheng et al. 2023]; LMRL Gym is its numeric-reward complement, benchmarking multi-turn reinforcement learning when states and actions are text [Abdulhai et al. 2023]. The starkest utilization result comes from Feedback Friction: even when the feedback generator has access to near-complete ground-truth answers, solver models consistently resist incorporating its feedback, and sampling-based mitigations improve matters without reaching target performance [Jiang et al. 2025]. Utilization, not quality, can be the binding constraint.

Agentic settings extend both axes to trajectories and memory. AgentRewardBench evaluates twelve LLM judges against expert annotations of 1,302 web-agent trajectories: no single judge excels across benchmarks, and the rule-based evaluation common in agent benchmarks underreports success rates [Lù et al. 2025]. Plan-RewardBench finds generative reward models, discriminative reward models, and LLM judges all facing substantial challenges on long-horizon, tool-integrated trajectory judgment [Wang et al. 2026b], echoing the credit-assignment difficulties of §8, and MemoryAgentBench tests whether accumulated verbal experience is accurately retrieved, used for test-time learning, and selectively forgotten across episodes [Hu et al. 2025b], the evaluation counterpart of the experiential memory mechanisms of §5.3.

11.3 Measurement Gaps

Four gaps stand out once the families in Table 6 are read side by side. First, benchmark validity is itself contested. Accuracy-style scores can decorrelate from the downstream quantity they proxy, as RewardMATH demonstrated for math preference pairs [Kim et al. 2024]; the corrective practice of validating a benchmark against best-of-N, RLHF, or policy outcomes [Frick et al. 2024; Malik et al. 2025; Zhou et al. 2024e] should become standard rather than a distinguishing feature. Second, the judges that anchor much VRL evaluation are unreliable instruments: they carry discoverable biases [Lai et al. 2026; Ye et al. 2024], break preference consistency on hard cases [Feng et al. 2025], and feed leaderboards whose aggregation hides ranking uncertainty [Feuer et al. 2025]. Any conclusion that rests on judge-scored evidence inherits these error bars. Third, utilization is systematically under-measured: most benchmarks grade signals in an open loop, yet the closed-loop results of RealCritic, JETTS, and Feedback Friction show that correct feedback frequently fails to improve the receiver [Jiang et al. 2025; Tang et al. 2025; Zhou et al. 2025f]. Fourth, coverage remains thin exactly where VRL is heading: non-English feedback [Son et al. 2024], personalized preference [Ma et al. 2026], long-horizon trajectories [Wang et al. 2026b], cross-episode memory [Hu et al. 2025b], and embodied feedback [Hwang et al. 2024; Lee et al. 2026; Wybitul et al. 2024] each have at most a few dedicated benchmarks, and the embodied instruments probe only quality, never utilization. Read together, the families also agree on where measurement is trustworthy and where it is not: quality signals are best calibrated on short, verifiable, English, single-turn tasks, and

least reliable on stylistically confounded, personalized, multilingual, long-horizon, or embodied ones, which is precisely the regime in which deployed VRL agents operate.

These gaps motivate a minimal reporting protocol for VRL systems. (1) Report feedback quality and feedback utilization as separate quantities, since either can bottleneck the other. (2) Validate any feedback-quality benchmark against a downstream criterion, whether best-of-N selection, policy optimization, or task success, before using it for model or signal selection. (3) Audit the judge or verifier for consistency and bias before trusting its verdicts as evaluation or reward. (4) State the signal type and stage being measured, so that results transfer only where the taxonomy of §3 says they should. What this protocol cannot yet supply, standardized utilization metrics, adversarial feedback benchmarks, and utilization measurement for the embodied settings of §10, where existing instruments grade only the judge, is taken up in the research agenda of §13.

12 Safety and Robustness of Feedback Channels

Classical reinforcement learning secures its reward channel by construction: the reward is a number emitted by the environment. Verbal reinforcement learning (VRL) replaces that scalar with language, and every channel surveyed in this article, judge verdicts and rubrics (§7), preference pairs (§6), optimizer feedback (§9), and instructions arriving through tool and web observations (§4), is therefore both a supervision signal and an attack surface. The published evidence organizes along two axes: who corrupts the signal, and which lifecycle stage consumes it. External adversaries manipulate judges, inject instructions into observations, and poison preference data; the policy itself exploits weaknesses in rubrics and verifiers; and the human-preference prior carries a systemic bias, sycophancy, that no adversary needs to plant. This decomposition tells a practitioner which link of a verbal reward stack to audit (§13) and locates the societal stakes: documented failures reach medical and interpersonal advice, as well as email, coding, and computer-use agents, where a corrupted grounding signal harms end users directly. We review external attacks on judges (§12.1), injection into observations (§12.2), and poisoning of preference data and optimizer feedback (§12.3), then the failures the policy itself exploits or inherits (§12.4, §12.5), and close with the published defenses (§12.6).

12.1 Robustness of LLM Judges

The judge verdict is the most widely deployed verbal reward and the most heavily attacked. Shi et al. [2024b] formulate judge deception as an optimization problem, crafting injected sequences that cause a judge to select an attacker-controlled candidate regardless of the alternatives, outperforming manually written injections and adapted jailbreaks. The threat reaches training as well as evaluation: the same attack applies when the judge ranks candidates in LLM-powered search, when it selects tools, and when it supplies rewards for reinforcement learning from AI feedback, so a deceived judge corrupts the learning signal of §6 as readily as a leaderboard. Attacks need not be tailored per query: short universal adversarial phrases learned on a surrogate judge transfer to unseen judges and drive scores to the maximum irrespective of the assessed text, with absolute scoring significantly more susceptible than comparative assessment [Raina et al. 2024], and generative reward models used as verifiers in training are systematically fooled by "master keys", superficial tokens or generic reasoning openers that elicit false-positive rewards without substantive content, an effect spanning model scales and leading proprietary systems [Zhao et al. 2025b]. Systematic evaluations confirm the breadth of the problem: prompt-injection attacks on judges succeed in up to 73.8% of cases and transfer across judge models at 50.5%–62.6%, leading the authors to recommend multi-model committees and comparative rather than absolute scoring [Maloyan and Namiot 2025]; judge robustness varies by up to 40% across prompt templates [Li et al. 2025]; and semantics-preserving stylistic manipulation alone exceeds a 65% attack success rate while evading style

controls [Yang et al. 2026a]. The weakness extends to bias and to safety evaluation. Automated bias discovery drives even strong evaluators above 50% error on a hardened judge benchmark [Lai et al. 2026]; apologetic phrasing alone can skew safety-evaluator preferences by up to 98%, larger judges are not consistently more robust, and juries improve robustness without eliminating artifact sensitivity [Chen and Goldfarb-Tarrant 2025]; and under the compounded distribution shifts of red-teaming, judge accuracy often degrades to near chance, so many reported attack successes reflect judge insufficiency rather than genuinely harmful outputs [Schwinn et al. 2026].

12.2 Injection into the Observation Loop

When feedback arrives through tools, webpages, or documents, the environment itself can turn adversarial. Indirect prompt injection, in which instructions embedded in retrieved content redirect an agent [Greshake et al. 2023], is well documented against tool-integrated agents: InjecAgent reports a 24% attack success rate against a ReAct-prompted GPT-4 agent, nearly doubled when attacker instructions are reinforced [Zhan et al. 2024], and AgentDojo shows that agents fail many tasks even without attack while existing attacks break some but not all security properties [Debenedetti et al. 2024]. For web agents, simple human-written injections partially succeed in up to 86% of cases even though agents often fail to complete the attacker's full goal, a state Evtimov et al. [2025] call "security by incompetence". A large public red-teaming competition, in which attackers pursued the dual objective of executing a harmful action while concealing any trace of compromise from the user-facing response, spanned 272,000 attack attempts against 13 frontier models across tool calling, coding, and computer use; every model proved vulnerable, with success rates between 0.5% and 8.5%, universal strategies that transferred across behaviors and model families, and only a weak correlation between capability and robustness [Dziemian et al. 2026]. Injection can also steer which tools an agent selects [Shi et al. 2025b], and adaptive attacks circumvent the defenses evaluated to date [Zhan et al. 2025].

12.3 Poisoning of Preference Data and Optimizer Feedback

Training-time channels are corrupted upstream of the learner. Injecting 1–5% poisoned preference pairs into public preference datasets suffices to steer generation toward a target entity and sentiment [Baumgärtner et al. 2024], and direct preference optimization (DPO) is markedly more fragile than PPO-based pipelines: backdoor attacks that require at least 4% poisoned data under PPO succeed with as little as 0.5% under DPO [Pathmanathan et al. 2024]. The channel can be corrupted without any external dataset: because preference data is constructed from the model's own outputs while pairwise labels record only which response is better, the model undergoing alignment can influence its own preference distribution, amplifying keyword bias, propaganda, and brand promotion in ways that existing robust-RLHF techniques do not fully resolve [Hahm et al. 2026]. The language-space optimizers of §9 inherit the same exposure: prompt-optimization loops are substantially more vulnerable to manipulated verbal feedback than to query poisoning, with feedback attacks raising attack success rates by up to 0.48, and a fake-reward variant requires no access to the reward model at all [Zhao et al. 2025a].

12.4 Reward, Rubric, and Judge Hacking by the Policy

Even without an adversary, the policy exploits its channel. Feedback loops that form when model outputs affect the world and the world re-enters the context drive in-context reward hacking, in which the model optimizes an implicit objective at test time while producing negative side effects; Pan et al. [2024] identify output refinement and policy refinement as its two driving processes and show that static-dataset evaluation misses exactly this behavior. The behavior also generalizes: models fine-tuned to game harmless graders transfer hacking to new settings and, in some

models, to unrelated misalignment such as shutdown evasion [Taylor et al. 2025]. Rubric-graded training shows the same dynamic on the training side; the transfer and verifier-strength findings appear in §7, and the point that matters here is that exploitation concentrates in recurring failure modes (partial satisfaction of compound criteria, treating implicit content as explicit, and imprecise topical matching) that persist whenever the rubric leaves them unspecified [Mahmoud et al. 2026]. Controlled environments that inject known judge biases [Wang et al. 2026a] or embed verifiable hacking opportunities directly into tasks [Roth et al. 2026] are beginning to make these dynamics measurable rather than anecdotal. The judge itself can be reverse-engineered: preamble generators trained adversarially against a judge raise evaluation scores for frozen candidate models, transfer to unseen judges, and remain virtually undetectable [Alazraki et al. 2025].

12.5 Sycophancy as a Bias of the Preference Channel

Sycophancy is not an attack but an inherited property of the human-preference channel. Sharma et al. [2024] showed that both humans and preference models prefer convincingly written sycophantic responses over correct ones a non-negligible fraction of the time, so optimizing against preference models can trade truthfulness for agreement. The behavior is pervasive at inference: sycophantic answer changes occurred in 58.19% of cases across mathematics and medical-advice tasks, with regressive flips to an incorrect answer in 14.66%, and the behavior persists at 78.5% regardless of context or model [Fanous et al. 2025]. Multi-turn pressure compounds it, and alignment tuning amplifies the behavior while scaling and reasoning-focused optimization strengthen resistance, although prompting the model to adopt a third-person perspective reduces sycophancy by up to 63.8% in a debate setting [Hong et al. 2025a]. The bias extends beyond agreement with stated beliefs: models preserve the user's desired self-image 45 percentage points more than humans on average and affirm both sides of moral conflicts in 48% of cases, with existing mitigations of limited effect [Cheng et al. 2025]. Mechanistic evidence sharpens the diagnosis: a shared circuit carries a this-statement-is-wrong signal even as the model agrees, and a preference-tuning refresh cuts sycophantic behavior roughly tenfold while the circuit persists [Pandey 2026]. Surveys of causes and mitigations attribute the behavior largely to the training and alignment pipeline itself [Malmqvist 2024].

12.6 Defenses and Open Problems

Published defenses divide into detection, training-signal reshaping, and system design, and the evidence favors the latter two. Detection-style defenses such as perplexity filters and known-answer checks are insufficient against optimized injections [Shi et al. 2024b], although re-tokenization and LLM-based detectors are the strongest among tested options [Li et al. 2025]. The asymmetry is structural: a detector inspects text that an adaptive optimizer has already shaped to pass inspection, whereas reshaping the training signal changes what the reward model credits in the first place. Reshaping the signal hardens the channel at its source: augmenting verifier training with truncated adversarial negatives yields reward models robust to master-key attacks [Zhao et al. 2025b]; adversarially training reward models against a policy that searches for high-reward, low-quality responses exposes and repairs vulnerabilities while stabilizing downstream RLHF [Bukharin et al. 2025]; linear-probe penalties suppress sycophancy markers inside the reward model [Papadatos and Freedman 2024]; and uncertainty-aware reasoning-trajectory rewards reduce sycophancy without degrading general capability [Beigi et al. 2025]. At the system level, instruction hierarchies [Wallace et al. 2024], preference-optimization defenses against injection [Chen et al. 2025d], design patterns that constrain agent-tool interaction [Beurer-Kellner et al. 2025], and lightweight highlighting of untrusted optimizer feedback, in effect a provenance marking of untrusted content, which cuts the attack-success delta of the fake-reward attack from 0.23 to 0.07 without degrading utility [Zhao

Table 7. Practitioner guidance: when to use language in each role, what it costs, and what to monitor.

Language as	Preconditions	Latency and cost	Dominant risk	Representative
Reward (compiled to code or rubric)	Success expressible as checks; simulator or executor available; weights and training budget accessible	Offline compilation; cheap per episode once compiled	Objective mis-specification; reward hacking	Eureka [Ma et al. 2024a]; Text2Reward [Xie et al. 2024]
State or task grounding	Raw observations too complex or task underspecified; language abstracts them faithfully	Per-step description cost; no training required	Information loss; state aliasing	ALFWorld [Shridhar et al. 2020]; SayCan [Ahn et al. 2022]
Action guidance	Open-ended action space; LLM priors informative; actions executable and checkable	Per-step generation cost	Exploration collapse under the prior; unexecutable actions	ReAct [Yao et al. 2023b]; ProgPrompt [Singh et al. 2022]; CodeAct [Wang et al. 2024b]
Deliberative feedback	No weight access; latency budget allows iteration; a grounded critic (tools, tests) exists	Multiplies inference cost per query	Circular self-critique; unbounded revision loops	Self-Refine [Madaan et al. 2023]; CRITIC [Gou et al. 2024a]; ToT [Yao et al. 2023a]
Memory	Tasks recur across episodes; lessons are reusable	Cheap writes; retrieval overhead per episode	Error persistence; memory poisoning	Reflexion [Shinn et al. 2023]; ExpeL [Zhao et al. 2024]; Voyager [Wang et al. 2023b]
Training signal	Persistent improvement needed; feedback plentiful; weights accessible	Highest upfront cost; amortizes across deployment	Sycophancy to wrong critiques; filter bias	FCP [Luo et al. 2025]; STaR [Zelikman et al. 2022]; generative PRMs [Zhao et al. 2026]

et al. 2025a], each close a specific surface, but adaptive attacks have so far broken the injection defenses they target [Zhan et al. 2025]. Three regularities summarize the state of the art: comparative judging is more robust than absolute scoring; stronger judges and verifiers reduce but do not eliminate exploitation; and alignment tuning can amplify the very failures it targets. Because interactive feedback loops produce failures that static benchmarks cannot exhibit [Pan et al. 2024; Schwinn et al. 2026], we argue that robustness of the feedback channel should be reported as a first-class metric alongside the capability evaluations of §11, a requirement the practitioner guidance in §13 operationalizes.

13 Practitioner Guide, Related Surveys, and Research Agenda

The preceding sections classify the literature; this section turns the classification outward. We first distill the taxonomy of §3 into prescriptive guidance for choosing a verbal-feedback mechanism (§13.1), then position the survey against eighteen prior surveys and frameworks (§13.2), and close with a ten-item research agenda assembled from the gaps named in §2 through §12 (§13.3).

13.1 Choosing a Verbal-Feedback Mechanism

Table 7 states when each role for language is the right choice, keyed to properties a practitioner can assess before building anything: whether task outcomes are mechanically verifiable, whether model weights are accessible, the latency and cost budget available for producing feedback, and the dominant risk that must be monitored once the mechanism is in place. The six rows correspond to the placements developed in §4 through §7; representative methods are drawn from the sections where each mechanism is analyzed.

The choice of mechanism also constrains the choice of optimizer: each feedback form pairs with a particular optimization family, and each pairing imports its own assumptions into the decision problem, as analyzed in §8. The guidance below therefore fixes the feedback form first and treats the optimizer as a downstream consequence of that choice.

The table compresses into four ordered questions. First, are model weights accessible? If so, a second question selects the training signal: when the outcome is mechanically verifiable, compile the language into an executable reward or rubric and train against it (§7), after which the monitoring burden shifts to hacking of the compiled objective; when it is not but verbal feedback is plentiful, train on that feedback directly through feedback-conditioned methods such as FCP [Luo et al. 2025], monitoring for sycophancy and filter bias (§6). Third, if weights are inaccessible but the latency budget allows iteration, use deliberative feedback, and prefer a critic grounded in tools or tests over prompted self-critique, which does not succeed without reliable external signal [Gou et al. 2024a; Kamoi et al. 2024]. Fourth, if the latency budget does not allow iteration but tasks recur across episodes, persist distilled lessons as memory and audit the store for stale or poisoned entries (§12). When every branch fails its preconditions (nothing verifiable to compile, no plentiful feedback to train on, no latency budget for iteration, no recurring tasks), the bottleneck is specification rather than optimization: spend the language budget on grounding, describing states, defining the task, or constraining actions before any feedback loop is built. Figure 6 arranges the four questions as a decision tree.

13.2 Comparison with Prior Surveys

To our knowledge, this survey is the first organized around the verbal signal itself: which component of the decision problem a linguistic artifact modifies and when it takes effect. Table 8 makes the positioning explicit against eighteen prior surveys and frameworks. Most shelf neighbors organize by mechanism (correction timing, evolution phase, rollout stage), by agent component (memory, capability, module under optimization), or by algorithm family (RLHF, preference optimization, agentic RL); the nearest exception, the feedback typing of Fernandes et al. [2023], does take the signal as its axis but predates the agentic signals surveyed here. None combines a signal-centric axis with a formal framework, coverage of all three stages (problem definition, inference, and training), and treatment of the feedback channel's safety; that fourfold combination is the position this survey occupies.

Two rows anchor the history of the idea. Luketina et al. [2019] argued in 2019 that language should inform a scalar-reward learner; the era surveyed here inverts that premise, since language is now simultaneously the policy medium, the feedback, and increasingly the optimizer trace (§9). Natural language reinforcement learning [Feng et al. 2024] is the closest formal neighbor, but it is one value-centric instantiation, extending RL components into language space, whereas our framework in §2 types the full range of signals and maps the empirical field that the formalism alone does not cover. This positioning also resolves the terminological collision noted in §1: the coinage of verbal reinforcement learning for self-reflective agents [Shinn et al. 2023] names the inference-stage cell of our taxonomy, natural language reinforcement learning names a training-stage formal instantiation, and this survey supplies the signal-centric umbrella over both.

A second cluster fixes the feedback signal in advance and organizes by mechanism. Pan et al. [2023] taxonomize by when correction happens; in our terms, correction is one consumption pattern, and we follow the same signals into grounding and the trained reward stack. Kamoi et al. [2024] contribute the field's sharpest negative result, that prompted self-feedback alone does not produce successful self-correction; in our frame this becomes a statement about signal provenance that generalizes across all three pillars. Zhang et al. [2025i] treat feedback as an implementation detail of scaling, whereas we make the signal the object of study and connect the same signal to training-time consumption. Fernandes et al. [2023] offer the closest prior signal typing, a formalization of feedback by format, objective, and use, but it predates agents, verifiable-reward training, rubric rewards, and experiential memory; we extend the typing to agentic settings and to adversarially supplied feedback.

Table 8. Positioning against prior surveys and frameworks. Feedback types: S = scalar, P = preference, V = verbal; a letter in parentheses marks a feedback type the work treats only partially. Stages: D = problem definition and grounding, I = inference, T = training. Formal (formal framework), Guide (prescriptive practitioner guidance), and Safety (feedback-channel safety): Y = yes, P = partial, N = not claimed. All cells are graded from each work's abstract and stated contributions, so N records the absence of a claim there, not a judgment of the work's full text.

Survey	Scope	Organizing axis	Feedb.	Stages	Formal	Guide	Safety
Luketina et al. [2019]	language-informed RL (pre-LLM)	task setting	S	T	N	N	N
Pan et al. [2023]	LLM self-correction	correction timing	V, (S)	I, T	N	P	N
Kamoi et al. [2024]	self-correction critique	research question × feedback source	V	I, T	N	Y	N
Fernandes et al. [2023]	feedback for generation	format × objective × use	S, P, V	I, T	Y	P	P
Wang et al. [2024g]	RL-enhanced LLMs	training technique (RLHF, RLAIF, DPO)	S, P	T	N	P	N
Xiao et al. [2024b]	preference optimization	research questions	P	T	P	P	N
Zhang et al. [2025b]	agent memory	design, evaluation, application	(V)	I	P	P	N
Zhang et al. [2025i]	test-time scaling	what/how/where/how well	(S), (V)	I	P	Y	N
Gao et al. [2025]	self-evolving agents	what/when/how to evolve	S, V	I, T	P	P	Y
Fang et al. [2025b]	self-evolving agents	component under optimization	(S), (V)	I, T	Y	P	Y
Du et al. [2026]	LLM-agent optimization	parameter-driven vs. parameter-free	S, (V)	I, T	N	P	N
Zhang et al. [2025f]	agentic RL	capability × domain	S, (P)	T	Y	P	N
Srivastava and Aggarwal [2025]	RL for LLMs	reward × feedback × optimization	S, P, (V)	T	Y	Y	P
Zhang [2026]	LLM-RL credit assignment	granularity × methodology	S	T	N	Y	N
Ji et al. [2025]	alignment	reward-mechanism dimensions	S, P	T	Y	Y	N
Wu [2025]	learning from rewards	reward model × strategy × stage	S, P, (V)	I, T	N	P	N
Surana et al. [2026]	RL rollout design	rollout stage	S, (V)	T	Y	Y	P
Feng et al. [2024]	NLRL (framework)	RL-component analogues in language	V	I, T	Y	N	N
This survey	verbal RL	the signal: what it modifies, when it acts	V, (S), (P)	D, I, T	Y	Y	Y

A third cluster organizes by agent component. Zhang et al. [2025b] survey memory as a module; we treat persisted verbal experience as a learning signal governed by an explicit boundary rule (§3). The two self-evolution surveys [Fang et al. 2025b; Gao et al. 2025] classify by which component evolves; we classify by the signal that drives evolution and supply the decision rules for signal choice that a component axis leaves unaddressed. Du et al. [2026], an agent survey published in this journal, split optimization at the top level into parameter-driven and parameter-free families, an axis keyed to where an update lands (weights or context); our signal-centric axis unifies those branches, since the same critique can drive a fine-tuning run or a prompt revision. Zhang et al. [2025f] formalize agentic RL as a shift from single-step to temporally extended decision problems, but their capability-by-domain taxonomy leaves the verbal channel implicit, and their training-centric frame underplays the problem-definition and inference stages we cover.

The final cluster is reward-centric. Wang et al. [2024g] and Xiao et al. [2024b] survey pipelines in which feedback is scalarized into preferences before optimization; our §6 examines precisely what that scalarization discards, namely the verbal rationale attached to a preference and the conditions under which retaining it pays. Srivastava and Aggarwal [2025] bring algorithmic failure-mode depth to RL for LLMs, yet their taxonomy also scalarizes feedback before analyzing it, whereas we keep the language structure of the signal and analyze credit assignment on it directly (§8). Ji et al.

[2025] read alignment progress as reward-design refinement, assuming the signal must become a reward; we ask when language should remain language. Wu [2025] span training, inference, and post-inference stages but from a reward-model-centric view that excludes problem-definition grounding and non-scalarized signals. Surana et al. [2026] locate where signals enter the rollout pipeline; we specify the semantics of those signals and cover the stages outside the training pipeline entirely. Zhang [2026] go deepest on a single shelf, surveying 47 credit-assignment methods for LLM reinforcement learning in a granularity-by-methodology grid, and contribute a method-selection decision tree of their own; their tree chooses among credit-assignment schemes once training is already the committed mechanism, whereas Figure 6 operates one level earlier, selecting which feedback mechanism to build at all. The shelf is wider than the table: verifier design for test-time scaling [Venktesh et al. 2025], test-time compute via search [Li 2025], internal consistency and self-feedback [Liang et al. 2024b], the self-improvement lifecycle [Yang et al. 2026c], reasoning-model frontiers [Ke et al. 2025; Xu et al. 2025a], and reinforcement learning for large reasoning models [Zhang et al. 2025m] each border individual sections of this survey, and none organizes by what the feedback says.

13.3 Research Agenda

The gaps identified across §2 through §12 assemble into a concrete agenda. We order it theory first, because most downstream items are blocked by the absence of a formal account of what a verbal signal contributes beyond its scalarization.

(1) **Sample efficiency of verbal versus scalar feedback.** There is no formal characterization of when a critique is worth more than a reward. The transfer-eluder analysis presented in §2 [Xu et al. 2025c] anchors one end of the question by separating rich feedback from scalar reward; what is missing is coverage of the intermediate formats that dominate practice. If verbal critics are modeled as noisy oracles with bounded error, PAC-style analyses [Strehl et al. 2009] could bound VRL sample complexity, with critic benchmarks [Lin et al. 2024a] supplying empirical error-rate estimates; statistical treatments of preference-based training [Liu et al. 2026c] offer a template for the analogous verbal theory.

(2) **Optimality and convergence under language-shaped rewards.** When the reward is compiled from language (§7), mis-specification enters the objective itself. Conditions under which policy optimization against compiled or rubric rewards converges to the intended behavior are unknown, and the reward-hacking failures surveyed for scalar reward models [Casper et al. 2023] have no counterpart analysis for language-specified objectives.

(3) **Temporal credit assignment over free-form critiques.** Process reward models (PRMs) assign step-level scalar credit [Lightman et al. 2024], and span-level feedback gradients [Wang et al. 2025h] push credit into the text itself, but no framework specifies which tokens, steps, or decisions a critique should be charged to (§8).

(4) **Information retention across scalarization.** A rate-distortion account of what a preference label or scalar score destroys, relative to the critique that produced it, would clarify when preference objectives [Rafailov et al. 2023] suffice and when feedback-conditioned methods [Luo et al. 2025] preserve signal that matters.

(5) **Feedback provenance and adversarial robustness.** Because VRL agents act on feedback by design, adversarial feedback is policy manipulation, and adaptive attacks have been shown to circumvent existing defenses [Zhan et al. 2025], including through injected tool output [Shi et al. 2025b] and poisoned memory [Dong et al. 2026b]. The channel needs provenance mechanisms that verify source integrity and assign trust, and adversarial-feedback benchmarks that score robustness as a primary metric (§12).

(6) **Separate evaluation of feedback quality and feedback utilization.** Models often fail to incorporate even accurate feedback [Jiang et al. 2025], so the two quantities must be reported separately. Producer-side benchmarks exist [Lambert et al. 2024b; Lin et al. 2024a; Zheng et al. 2024]; utilization lacks a standard protocol (§11).

(7) **Feedback-tuned models.** Dedicated critics outperform reusing the generator as its own critic [McAleese et al. 2024; Wang et al. 2023c]. As the field once invested in instruction tuning [Ouyang et al. 2022], it should now train feedback-tuned models whose objectives target error localization, actionability, and calibration.

(8) **Machine-readable feedback interfaces.** The externally grounded critique loops of §5 depend on tool feedback that agents can parse, yet ambiguous tool outputs cause a large share of agent failures [Bandlamudi et al. 2025], agent-oriented interfaces measurably help [Yang et al. 2024a], and tool quality bounds what iterative refinement can achieve [Ning et al. 2026]. Needed: a tool-output compliance metric, and structured metadata that separates task-level errors from infrastructure failures so agents can select recovery strategies (§5).

(9) **The inference/training boundary.** The transient-versus-durable distinction drawn for rewards in tree search [Wei et al. 2025c] should be generalized to every verbal signal type: when replayed memory [Zhang et al. 2025b] constitutes training is the hybrid case our decision rules in §3 adjudicate, but the boundary deserves a formal statement within the framework of §2.

(10) **Validity and transfer in embodied and high-stakes settings.** Self-correction succeeds only with reliable external feedback, large-scale fine-tuning, or tasks exceptionally suited to it [Kamoi et al. 2024], and sycophancy arises even from benign incorrect feedback [Sharma et al. 2024]. Establishing when language-grounded policies transfer to physical environments (§10), and who is harmed when feedback is wrong in coding, clinical, educational, or robotic deployments, is a prerequisite for responsible use (§12).

14 Conclusion

This article has surveyed verbal reinforcement learning through a single organizing question: where a natural-language signal enters the decision problem and what it modifies at the moment it takes effect, yielding the three pillars of language as grounding signal, as deliberative feedback, and as learning signal. The four contributions promised in §1 were delivered as follows. The two-criterion fence and the language-augmented sequential decision framework of §2 give VRL a definition tight enough to exclude plain instruction following, vanilla reinforcement learning from human feedback, and language-conditioned reinforcement learning, while the taxonomy of §3 assigns every signal by its point of effect and resolves hybrid cases, such as reward-code generation and experiential memory, by explicit decision rules rather than by fiat. These rules are turned outward in the prescriptive practitioner guide of §13, which keys the choice among verbal-feedback mechanisms to task verifiability, feedback latency and cost, and the dominant risk each mechanism carries. The synthesis sections then followed verbal signals into the machinery that consumes them: the verbal reward stack (§7), credit assignment at outcome, process, and token granularity (§8), and optimizers that act in language space rather than on weights (§9). Finally, the comparison with prior surveys and the research agenda, also in §13, position this account against its neighbors and state what remains to be done.

The payoff of the signal-centric view is consolidation. Questions ordinarily scattered across separate literatures, such as how much of a critique survives compression to a scalar, which feedback forms pair with which algorithm families, and when a verifier should displace a judge, become instances of one question about signal placement and fidelity. Looking ahead, we expect verbal feedback to become a first-class design primitive in agent architectures. Deployed agents will increasingly consume grounding, deliberative, and learning signals within a single trajectory, and the bottleneck will shift from generating feedback to verifying it; the infrastructure that shift demands, from feedback

provenance to standardized quality evaluation to machine-readable tool interfaces, is the substance of the research agenda in §13.

14.1 Limitations

The survey's own limits should be stated plainly. Within each category we cover representative papers rather than an exhaustive enumeration, and the taxonomy is an organizing lens rather than a strict partition: several methods span roles, since verbal feedback can serve simultaneously as a grounding signal, a deliberative mechanism, and a learning signal, and our decision rules adjudicate placement without dissolving the underlying overlap. Coverage is restricted to methods in which verbal feedback is explicit and central to the agent's process; adjacent work where language plays an auxiliary role is treated only in passing, and the definitional fence fixes the feedback modality to text, so spoken and richer multimodal feedback channels fall outside our scope even where embodied observations do not (§10). The article is also a synthesis: it contributes no new benchmark or controlled empirical comparison, and where the literature offers none we have said so rather than inferred one.

Language-based feedback itself carries failure modes that this survey can name but not yet bound. A specification compiled from language can be executable yet mean the wrong thing, and policies exploit whatever a rubric or verifier leaves unspecified (§7) [Mahmoud et al. 2026]. The consuming model can misread the signal: generative verifiers are fooled by superficial "master keys" [Zhao et al. 2025b], and preference models reward convincing agreement over correctness [Sharma et al. 2024]. Grounding between verbal descriptions and environment state remains weak where it matters most: vision-language judges recognize objects but fail to understand task sequence [Wybitul et al. 2024], and single-frame success detectors miss failures that live in the motion rather than the final state [Hwang et al. 2024]. On real-world transfer the surveyed record is positive but thin: Text2Reward, DrEureka, and RACER report successful real-robot deployments [Dai et al. 2024; Ma et al. 2024b; Xie et al. 2024], yet each is a demonstration on a specific platform, and we found no systematic study of when language-guided policies fail to transfer, so the breadth of transfer is an open question rather than an established property. Societal impact follows the same channels. The failures above already reach deployed settings, with sycophantic answer flips documented on medical-advice tasks [Fanous et al. 2025] and injection attacks demonstrated against tool-calling, coding, and computer-use agents [Dziemian et al. 2026]; in the high-stakes domains the field is entering (§1), a corrupted grounding signal harms both the system's users and the people affected by its outputs, who rarely chose the feedback channel and cannot audit it. Sections 12 and 8 discuss mitigation directions and the current state of formal guarantees; narrowing the gap between what the definitional fence admits and what theory can certify is, in our judgment, the field's most consequential open problem.

References

Marwa Abdulhai, Isadora White, Charlie Snell, Charles Sun, Joey Hong, Yuexiang Zhai, Kelvin Xu, and Sergey Levine. 2023. LMRL Gym: Benchmarks for Multi-Turn Reinforcement Learning with Language Models. (2023). arXiv:2311.18232

Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, and Shyamal Anadkat. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).

Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, and Meng Jiang. 2025. Gepa: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457* (2025).

Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, and Karol Hausman. 2022. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691* (2022).

Bakhtawar Ahtisham, Kirk Vanacore, Jinsook Lee, Zhuqian Zhou, Doug Pietrzak, and Rene F Kizilcec. 2026. Ai annotation orchestration: Evaluating llm verifiers to improve the quality of llm annotations in learning analytics. In *Proceedings of the LAK26: 16th International Learning Analytics and Knowledge Conference*. 447–456.

Lisa Alazraki, Tan Yi-Chern, Jon Ander Campos, Maximilian Mozes, Marek Rei, and Max Bartolo. 2025. Reverse Engineering Human Preferences with Reinforcement Learning. arXiv:2505.15795

Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. 2016. Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565* (2016).

Jacob Andreas, Dan Klein, and Sergey Levine. 2017. Modular Multitask Reinforcement Learning with Policy Sketches. In *International Conference on Machine Learning*. 166–175. arXiv:1611.01796

Samuel Arnesen, David Rein, and Julian Michael. 2024. Training Language Models to Win Debates with Self-Play Improves Judge Accuracy. (2024). arXiv:2409.16636

Mohammad Gheshlaghi Azar, Zhaohan Daniel Guo, Bilal Piot, Remi Munos, Mark Rowland, Michal Valko, and Daniele Calandriello. 2024. A general theoretical paradigm to understand learning from human preferences. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 4447–4455.

Juyang Bai and Laixi Shi. 2026. MAS-PromptBench: When Does Prompt Optimization Improve Multi-Agent LLM Systems? (2026). arXiv:2606.23664

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Carol Chen, Catherine Olsson, Christopher Olah, Danny Hernandez, Dawn Drain, Deep Ganguli, Dustin Li, Eli Tran-Johnson, Ethan Perez, Jamie Kerr, Jared Mueller, Jeffrey Ladish, Joshua Landau, Kamal Ndousse, Kamile Lukosuite, Liane Lovitt, Michael Sellitto, Nelson Elhage, Nicholas Schiefer, Noemi Mercado, Nova DasSarma, Robert Lasenby, Robin Larson, Sam Ringer, Scott Johnston, Shauna Kravec, Sheer El Showk, Stanislav Fort, Tamera Lanham, Timothy Telleen-Lawton, Tom Conerly, Tom Henighan, Tristan Hume, Samuel R Bowman, Zac Hatfield-Dodds, Ben Mann, Dario Amodei, Nicholas Joseph, Sam McCandlish, Tom Brown, and Jared Kaplan. 2022. Constitutional AI: Harmlessness from AI Feedback. *arXiv preprint arXiv:2212.08073* (2022).

Jayachandu Bandlamudi, Ritwik Chaudhuri, Neelamadhav Gantayat, Sambit Ghosh, Kushal Mukherjee, Prerna Agarwal, Renuka Sindhgatta, and Sameep Mehta. 2025. A framework for testing and adapting rest apis as llm tools. *arXiv preprint arXiv:2504.15546* (2025).

Tim Baumgärtner, Yang Gao, Dana Alon, and Donald Metzler. 2024. Best-of-Venom: Attacking RLHF by Injecting Poisoned Preference Data. arXiv:2404.05530

Mohammad Beigi, Ying Shen, Parshin Shojaee, Qifan Wang, Zichao Wang, Chandan Reddy, Ming Jin, and Lifu Huang. 2025. Sycophancy Mitigation Through Reinforcement Learning with Uncertainty-Aware Adaptive Reasoning Trajectories. arXiv:2509.16742

Julia Belikova, Rauf Parchiev, Evgeny Egorov, Grigorii Davydenko, Gleb Gusev, Andrey Savchenko, and Maksim Makarenko. 2026. Managing Procedural Memory in LLM Agents: Control, Adaptation, and Evaluation. (2026). arXiv:2606.23127

Suneel Belkhale, Tianli Ding, Ted Xiao, Pierre Sermanet, Quon Vuong, Jonathan Tompson, Yevgen Chebotar, Debidatta Dwibedi, and Dorsa Sadigh. 2024. RT-H: Action Hierarchies Using Language. (2024). arXiv:2403.01823

Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, and Piotr Nyczyk. 2024. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 38. 17682–17690.

Luca Beurer-Kellner, Beat Buesser, Ana-Maria Creţu, Edoardo Debenedetti, Daniel Dobos, Daniel Fabian, Marc Fischer, David Froelicher, Kathrin Grosse, and Daniel Naeff. 2025. Design patterns for securing llm agents against prompt injections. *arXiv preprint arXiv:2506.08837* (2025).

Vineet Bhat, Ali Umut Kaypak, Prashanth Krishnamurthy, Ramesh Karri, and Farshad Khorrami. 2024. Grounding llms for robot task planning using closed-loop state feedback. *arXiv preprint arXiv:2402.08546* (2024).

Andres Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D White, and Philippe Schwaller. 2024. Augmenting large language models with chemistry tools. *Nature machine intelligence* 6, 5 (2024), 525–535.

Satchuthananthavale RK Branavan, Harr Chen, Luke Zettlemoyer, and Regina Barzilay. 2009. Reinforcement learning for mapping instructions to actions. In *Proceedings of the Joint Conference of the 47th Annual Meeting of the ACL and the 4th International Joint Conference on Natural Language Processing of the AFNLP*. 82–90.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, and Amanda Askell. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.

Alexander Bukharin, Haifeng Qian, Shengyang Sun, Adithya Renduchintala, Soumye Singhal, Zhilin Wang, Oleksii Kuchaiev, Olivier Delalleau, and Tuo Zhao. 2025. Adversarial Training of Reward Models. arXiv:2504.06141

Yuzheng Cai, Siqi Cai, Yuchen Shi, Zihan Xu, Lichao Chen, Yulei Qin, Xiaoyu Tan, Gang Li, Zongyi Li, Haojia Lin, Yong Mao, Ke Li, and Xing Sun. 2025. Training-Free Group Relative Policy Optimization. (2025). arXiv:2510.08191

Zouying Cao, Jiaji Deng, Li Yu, Weikang Zhou, Zhaoyang Liu, Bolin Ding, and Hai Zhao. 2025. Remember Me, Refine Me: A Dynamic Procedural Memory Framework for Experience-Driven Agent Evolution. (2025). arXiv:2512.10696

Stephen Casper, Xander Davies, Claudia Shi, Thomas Krendl Gilbert, Jérémy Scheurer, Javier Rando, Rachel Freedman, Tomasz Korbak, David Lindner, and Pedro Freire. 2023. Open problems and fundamental limitations of reinforcement learning from human feedback. *arXiv preprint arXiv:2307.15217* (2023).

Yekun Chai, Haoran Sun, Huang Fang, Shuohuan Wang, Yu Sun, and Hua Wu. 2024. MA-RLHF: Reinforcement Learning from Human Feedback with Macro Actions. (2024). arXiv:2410.02743

Chi-Min Chan, Weize Chen, Yusheng Su, Jianxuan Yu, Wei Xue, Shanghang Zhang, Jie Fu, and Zhiyuan Liu. 2023. ChatEval: Towards Better LLM-based Evaluators through Multi-Agent Debate. arXiv:2308.07201 [cs.CL] https://arxiv.org/abs/2308.07201

Devendra Singh Chaplot, Kanthashree Mysore Sathyendra, Rama Kumar Pasumarthi, Dheeraj Rajagopal, and Ruslan Salakhutdinov. 2018. Gated-attention architectures for task-oriented language grounding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 32.

Angelica Chen, Jérémy Scheurer, Jon Ander Campos, Tomasz Korbak, Jun Shern Chan, Samuel R Bowman, Kyunghyun Cho, and Ethan Perez. 2024b. Learning from Natural Language Feedback. *Transactions on Machine Learning Research* (2024).

Hongyu Chen and Seraphina Goldfarb-Tarrant. 2025. Safer or Luckier? LLMs as Safety Evaluators Are Not Robust to Artifacts. arXiv:2503.09347

Hao Chen, Ziyu Han, Yukun Yan, Qingfu Zhu, Maosong Sun, and Wanxiang Che. 2026b. From Holistic Evaluation to Structured Criteria: Rubrics Across the Evolving LLM Landscape. (2026). arXiv:2606.08625

Jiaqi Chen, Bang Zhang, Ruotian Ma, Peisong Wang, Xiaodan Liang, Zhaopeng Tu, Xiaolong Li, and Kwan-Yee K. Wong. 2025c. SPC: Evolving Self-Play Critic via Adversarial Games for LLM Reasoning. (2025). arXiv:2504.19162

Minghui Chen, Wenlong Deng, James Zou, Han Yu, and Xiaoxiao Li. 2026a. Textual Equilibrium Propagation for Deep Compound AI Systems. (2026). arXiv:2601.21064

Sizhe Chen, Arman Zharmagambetov, Saeed Mahloujifar, Kamalika Chaudhuri, David Wagner, and Chuan Guo. 2025d. SecAlign: Defending Against Prompt Injection with Preference Optimization. arXiv:2410.05451 [cs.CR] doi:10.1145/3719027.3744836

Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. 2020. A simple framework for contrastive learning of visual representations. In *International conference on machine learning*. PmLR, 1597–1607. arXiv:2002.05709

Xiusi Chen, Gaotang Li, Ziqi Wang, Bowen Jin, Cheng Qian, Yu Wang, Hongru Wang, Yu Zhang, Denghui Zhang, Tong Zhang, Hanghang Tong, and Heng Ji. 2025a. RM-R1: Reward Modeling as Reasoning. (2025). arXiv:2505.02387

Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2024a. Teaching large language models to self-debug. In *International Conference on Learning Representations*, Vol. 2024. 8746–8825.

Xiaoyin Chen, Canwen Xu, Yite Wang, Boyi Liu, Zhewei Yao, and Yuxiong He. 2026c. Learning to Self-Evolve. (2026). arXiv:2603.18620

Yixing Chen, Yiding Wang, Siqi Zhu, Haofei Yu, Tao Feng, Muhan Zhang, Mostofa Patwary, and Jiaxuan You. 2025b. Multi-Agent Evolve: LLM Self-Improve through Co-evolution. (2025). arXiv:2510.23595

Ching-An Cheng, Andrey Kolobov, Dipendra Misra, Allen Nie, and Adith Swaminathan. 2023. LLF-Bench: Benchmark for Interactive Learning from Language Feedback. (2023). arXiv:2312.06853

Ching-An Cheng, Allen Nie, and Adith Swaminathan. 2024b. Trace is the Next AutoDiff: Generative Optimization with Rich Feedback, Execution Traces, and LLMs. (2024). arXiv:2406.16218

Myra Cheng, Sunny Yu, Cinoo Lee, Pranav Khadpe, Lujain Ibrahim, and Dan Jurafsky. 2025. ELEPHANT: Measuring and understanding social sycophancy in LLMs. arXiv:2505.13995

Pengyu Cheng, Tianhao Hu, Han Xu, Zhisong Zhang, Zheng Yuan, Yong Dai, Lei Han, Nan Du, and Xiaolong Li. 2024a. Self-playing Adversarial Language Game Enhances LLM Reasoning. (2024). arXiv:2404.10642

Maxime Chevalier-Boisvert, Dzmitry Bahdanau, Salem Lahlou, Lucas Willems, Chitwan Saharia, Thien Huu Nguyen, and Yoshua Bengio. 2018. Babyai: A platform to study the sample efficiency of grounded language learning. *arXiv preprint arXiv:1810.08272* (2018).

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. (2025). arXiv:2504.19413

Hyeong Kyu Choi, Xiaojin Zhu, and Yixuan Li. 2025. Debate or Vote: Which Yields Better Decisions in Multi-Agent Large Language Models? *Advances in Neural Information Processing Systems* (2025). arXiv:2508.17536

Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. 2017. Deep reinforcement learning from human preferences. *Advances in Neural Information Processing Systems* 30 (2017). arXiv:1706.03741

Vanya Cohen, Geraud Nangue Tasse, Nakul Gopalan, Steven James, Matthew Gombolay, Ray Mooney, and Benjamin Rosman. 2025. Compositional Instruction Following with Language Models and Reinforcement Learning. *arXiv preprint arXiv:2501.12539* (2025).

Marc-Alexandre Côté, Akos Kádár, Xingdi Yuan, Ben Kybartas, Tavian Barnes, Emery Fine, James Moore, Matthew Hausknecht, Layla El Asri, and Mahmoud Adada. 2018. Textworld: A learning environment for text-based games. In *Workshop on Computer Games*. Springer, 41–75.

Yuchen Cui, Siddharth Karamcheti, Raj Palleti, Nidhya Shivakumar, Percy Liang, and Dorsa Sadigh. 2023. "No, to the Right" – Online Language Corrections for Robotic Manipulation via Shared Autonomy. (2023). arXiv:2301.02555

Jeff Da, Clinton Wang, Xiang Deng, Yuntao Ma, Nikhil Barhate, and Sean Hendryx. 2025. Agent-RLVR: Training Software Engineering Agents via Guidance and Environment Rewards. (2025). arXiv:2506.11425

Yinpei Dai, Jayjun Lee, Nima Fazeli, and Joyce Chai. 2024. RACER: Rich Language-Guided Failure Recovery Policies for Imitation Learning. (2024). arXiv:2409.14674

Parth Darshan and Abhishek Divekar. 2026. When Gradients Collide: Failure Modes of Multi-Objective Prompt Optimization for LLM Judges. (2026). arXiv:2605.26046

Sarkar Snigdha Sarathi Das, Ryo Kamoi, Bo Pang, Yusen Zhang, Caiming Xiong, and Rui Zhang. 2024. GReaTer: Gradients over Reasoning Makes Smaller Language Models Strong Prompt Optimizers. (2024). arXiv:2412.09722

Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. *Advances in Neural Information Processing Systems* 37 (2024), 82895–82920.

Jonas Degrave, Federico Felici, Jonas Buchli, Michael Neunert, Brendan Tracey, Francesco Carpanese, Timo Ewalds, Roland Hafner, Abbas Abdolmaleki, and Diego de Las Casas. 2022. Magnetic control of tokamak plasmas through deep reinforcement learning. *Nature* 602, 7897 (2022), 414–419.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805 [cs.CL] https://arxiv.org/abs/1810.04805

Shen Dong, Shaochen Xu, Pengfei He, Yige Li, Jiliang Tang, Tianming Liu, Hui Liu, and Zhen Xiang. 2026b. Memory injection attacks on LLM agents via query-only interaction. *Advances in Neural Information Processing Systems* 38 (2026), 46697–46731.

Zhichen Dong, Yang Li, Yuhan Sun, Weixun Wang, Yijia Luo, Zinian Peng, Taiheng Ye, Chao Yang, Wenbo Su, Yu Cheng, Bo Zheng, and Junchi Yan. 2026a. How Does Reasoning Flow? Tracing Attention-Induced Information Flow for Targeted RL in LLMs. (2026). arXiv:2606.10646

Danny Driess, Fei Xia, Mehdi SM Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, and Tianhe Yu. 2023. Palm-e: An embodied multimodal language model. *arXiv preprint arXiv:2303.03378* (2023).

Lingxiao Du, Fanqing Meng, Zongkai Liu, Zhixiang Zhou, Ping Luo, Qiaosheng Zhang, and Wenqi Shao. 2025. MM-PRM: Enhancing Multimodal Mathematical Reasoning with Scalable Step-Level Supervision. (2025). arXiv:2505.13427

Pengfei Du. 2026. Memory for autonomous llm agents: Mechanisms, evaluation, and emerging frontiers. *arXiv preprint arXiv:2603.07670* (2026).

Shangheng Du, Jiabao Zhao, Jinxin Shi, Zhentao Xie, Xin Jiang, Yanhong Bai, and Liang He. 2026. A Survey on the Optimization of Large Language Model-based Agents. *Comput. Surveys* 58, 9 (2026). arXiv:2503.12434

Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. 2024. Improving Factuality and Reasoning in Language Models through Multiagent Debate. In *International Conference on Machine Learning*.

Jiafei Duan, Wilbert Pumacay, Nishanth Kumar, Yi Ru Wang, Shulin Tian, Wentao Yuan, Ranjay Krishna, Dieter Fox, Ajay Mandlekar, and Yijie Guo. 2024. AHA: A Vision-Language-Model for Detecting and Reasoning Over Failures in Robotic Manipulation. (2024). arXiv:2410.00371

Mateusz Dziemian, Maxwell Lin, Xiaohan Fu, Micha Nowak, Nick Winter, Eliot Jones, Andy Zou, Lama Ahmad, Kamalika Chaudhuri, Sahana Chennabasappa, Xander Davies, Lauren Deason, Benjamin L. Edelman, Tanner Emek, Ivan Evtimov, Jim Gust, Maia Hamin, Kat He, Klaudia Krawiecka, Riccardo Patana, Neil Perry, Troy Peterson, Xiangyu Qi, Javier Rando, Zifan Wang, Zihan Wang, Spencer Whitman, Eric Winsor, Arman Zharmagambetov, Matt Fredrikson, and Zico Kolter. 2026. How Vulnerable Are AI Agents to Indirect Prompt Injections? Insights from a Large-Scale Public Competition. arXiv:2603.15714

Andrew Estornell and Yang Liu. 2024. Multi-llm debate: Framework, principals, and interventions. *Advances in Neural Information Processing Systems* 37 (2024), 28938–28964.

Kawin Ethayarajh, Winnie Xu, Niklas Muennighoff, Dan Jurafsky, and Douwe Kiela. 2024. Kto: Model alignment as prospect theoretic optimization. *arXiv preprint arXiv:2402.01306* (2024).

Ivan Evtimov, Arman Zharmagambetov, Aaron Grattafiori, Chuan Guo, and Kamalika Chaudhuri. 2025. WASP: Benchmarking Web Agent Security Against Prompt Injection Attacks. arXiv:2504.18575

Jinyuan Fang, Yanwen Peng, Xi Zhang, Yingxu Wang, Xinhao Yi, Guibin Zhang, Yi Xu, Bin Wu, Siwei Liu, Zihao Li, Zhaochun Ren, Nikos Aletras, Xi Wang, Han Zhou, and Zaiqiao Meng. 2025b. A Comprehensive Survey of Self-Evolving AI Agents: A New Paradigm Bridging Foundation Models and Lifelong Agentic Systems. (2025). arXiv:2508.07407

Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. 2025a. Memp: Exploring Agent Procedural Memory. (2025). arXiv:2508.06433

Aaron Fanous, Jacob Goldberg, Ank A. Agarwal, Joanna Lin, Anson Zhou, Roxana Daneshjou, and Sanmi Koyejo. 2025. SycEval: Evaluating LLM Sycophancy. arXiv:2502.08177

Xidong Feng, Bo Liu, Yan Song, Haotian Fu, Ziyu Wan, Girish A. Koushik, Zhiyuan Hu, Mengyue Yang, Ying Wen, and Jun Wang. 2024. Natural Language Reinforcement Learning. (2024). arXiv:2411.14251

Yuanning Feng, Sinan Wang, Zhengxiang Cheng, Yao Wan, and Dongping Chen. 2025. Are We on the Right Way to Assessing LLM-as-a-Judge? (2025). arXiv:2512.16041

Patrick Fernandes, Aman Madaan, Emmy Liu, António Farinhas, Pedro Henrique Martins, Amanda Bertsch, José G. C. de Souza, Shuyan Zhou, Tongshuang Wu, Graham Neubig, and André F. T. Martins. 2023. Bridging the Gap: A Survey on Integrating (Human) Feedback for Natural Language Generation. (2023). arXiv:2305.00955

Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. 2023. Promptbreeder: Self-Referential Self-Improvement Via Prompt Evolution. *arXiv preprint arXiv:2309.16797* (2023).

Benjamin Feuer, Chiung-Yi Tseng, Astitwa Sarthak Lathe, Oussama Elachqar, and John P Dickerson. 2025. When Judgment Becomes Noise: How Design Failures in LLM Judge Benchmarks Silently Undermine Validity. (2025). arXiv:2509.20293

Evan Frick, Tianle Li, Connor Chen, Wei-Lin Chiang, Anastasios N. Angelopoulos, Jiantao Jiao, Banghua Zhu, Joseph E. Gonzalez, and Ion Stoica. 2024. How to Evaluate Reward Models for RLHF. (2024). arXiv:2410.14872

Huan-ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Yiran Wu, Hongru Wang, Han Xiao, Yuhang Zhou, Shaokun Zhang, Jiayi Zhang, Jinyu Xiang, Yixiong Fang, Qiwen Zhao, Dongrui Liu, Qihan Ren, Cheng Qian, Zhenhailong Wang, Minda Hu, Huazheng Wang, Qingyun Wu, Heng Ji, and Mengdi Wang. 2025. A Survey of Self-Evolving Agents: What, When, How, and Where to Evolve on the Path to Artificial Super Intelligence. (2025). arXiv:2507.21046

Jonas Gehring, Kunhao Zheng, Jade Copet, Vegard Mella, Quentin Carbonneaux, Taco Cohen, and Gabriel Synnaeve. 2024. Rlef: Grounding code llms in execution feedback with reinforcement learning. *arXiv preprint arXiv:2410.02089* (2024).

Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. 2024a. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. In *International Conference on Learning Representations*. arXiv:2305.11738

Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Minlie Huang, Nan Duan, and Weizhu Chen. 2024b. ToRA: A Tool-Integrated Reasoning Agent for Mathematical Problem Solving. arXiv:2309.17452 [cs.CL] https://arxiv.org/abs/2309.17452

Joel Greenspoon. 1955. The reinforcing effect of two spoken sounds on the frequency of two responses. *The American Journal of Psychology* 68, 3 (1955), 409–416.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM workshop on artificial intelligence and security*. 79–90.

Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, Saizhuo Wang, Kun Zhang, Yuanzhuo Wang, Wen Gao, Lionel Ni, and Jian Guo. 2024. A Survey on LLM-as-a-Judge. (2024). arXiv:2411.15594

Xin Guan, Xiaomeng Hu, Shen Huang, Zhenyi Wang, Bo Zhang, Zijian Li, Pengjun Xie, Bo Liu, and Jiuxin Cao. 2026. EvoRubric: Self-Evolving Rubric-Driven RL for Open-Ended Generation. (2026). arXiv:2605.29847

Anisha Gunjal, Anthony Wang, Elaine Lau, Vaskar Nath, Yunzhong He, Bing Liu, and Sean Hendryx. 2025. Rubrics as Rewards: Reinforcement Learning Beyond Verifiable Domains. (2025). arXiv:2507.17746

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, and Xiao Bi. 2025b. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).

Jiaxin Guo, Zewen Chi, Li Dong, Qingxiu Dong, Xun Wu, Shaohan Huang, and Furu Wei. 2025a. Reward Reasoning Model. (2025). arXiv:2505.14674

Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. 2024. Connecting Large Language Models with Evolutionary Algorithms Yields Powerful Prompt Optimizers. In *International Conference on Learning Representations*. arXiv:2309.08532

Siyuan Guo, Yali Du, Hechang Chen, Yi Chang, and Jun Wang. 2026. CASCADE: Case-Based Continual Adaptation for Large Language Models During Deployment. (2026). arXiv:2605.06702

Priyanshu Gupta, Shashank Kirtania, Ananya Singha, Sumit Gulwani, Arjun Radhakrishna, Sherry Shi, and Gustavo Soares. 2024. MetaReflection: Learning Instructions for Language Agents using Past Reflections. (2024). arXiv:2405.13009

Dongyoon Hahm, Dylan Hadfield-Menell, and Kimin Lee. 2026. Alignment Tampering: How Reinforcement Learning from Human Feedback Is Exploited to Optimize Misaligned Biases. arXiv:2605.27355

Hojae Han, Seung-won Hwang, Rajhans Samdani, and Yuxiong He. 2025. ConvCodeWorld: Benchmarking Conversational Code Generation in Reproducible Feedback Environments. (2025). arXiv:2502.19852

Shibo Hao, Yi Gu, Haodi Ma, Joshua Hong, Zhen Wang, Daisy Wang, and Zhiting Hu. 2023. Reasoning with language model is planning with world model. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 8154–8173.

Ali Hatamizadeh, Shrimai Prabhumoye, Igor Gitman, Ximing Lu, Seungju Han, Wei Ping, Yejin Choi, and Jan Kautz. 2026. iGRPO: Self-feedback-driven LLM reasoning. *arXiv preprint arXiv:2602.09000* (2026).

Jianliang He, Siyu Chen, Fengzhuo Zhang, and Zhuoran Yang. 2024. From Words to Actions: Unveiling the Theoretical Underpinnings of LLM-Driven Autonomous Systems. (2024). arXiv:2405.19883

Prannay Hebbar, Yogendra Manawat, Samuel Verboomen, Alesia Ivanova, Selvam Palanimalai, Kunal Bhatia, and Vignesh Baskaran. 2026. SIA: Self Improving AI with Harness & Weight Updates. (2026). arXiv:2605.27276

Karl Moritz Hermann, Felix Hill, Simon Green, Fumin Wang, Ryan Faulkner, Hubert Soyer, David Szepesvari, Wojciech Marian Czarnecki, Max Jaderberg, and Denis Teplyashin. 2017. Grounded language learning in a simulated 3d world. *arXiv preprint arXiv:1706.06551* (2017).

Ilgee Hong, Changlong Yu, Liang Qiu, Weixiang Yan, Zhenghao Xu, Haoming Jiang, Qingru Zhang, Qin Lu, Xin Liu, Chao Zhang, and Tuo Zhao. 2025b. Think-RM: Enabling Long-Horizon Reasoning in Generative Reward Models. (2025). arXiv:2505.16265

Jiseung Hong, Grace Byun, Seungone Kim, Kai Shu, and Jinho D. Choi. 2025a. Measuring Sycophancy of Language Models in Multi-turn Dialogues. arXiv:2505.23840

Jiwoo Hong, Noah Lee, and James Thorne. 2024a. Orpo: Monolithic preference optimization without reference model. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 11170–11189.

Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Steven Yau, Zijuan Lin, and Liyang Zhou. 2024b. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, Vol. 2024. 23247–23275.

Nicola Horst, Davide Mazzaccara, Antonia Schmidt, Michael Sullivan, Filippo Momentè, Luca Franceschetti, Philipp Sadler, Sherzod Hakimov, Alberto Testoni, Raffaella Bernardi, Raquel Fernández, Alexander Koller, Oliver Lemon, David Schlangen, Mario Giulianelli, and Alessandro Suglia. 2025. Playpen: An Environment for Exploring Learning Through Conversational Interaction. (2025). arXiv:2504.08590

Shengran Hu, Cong Lu, and Jeff Clune. 2024. Automated Design of Agentic Systems. (2024). arXiv:2408.08435

Yuanzhe Hu, Yu Wang, and Julian McAuley. 2025b. Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions. (2025). arXiv:2507.05257

Zican Hu, Wei Liu, Xiaoye Qu, Xiangyu Yue, Chunlin Chen, Zhi Wang, and Yu Cheng. 2025a. Divide and conquer: Grounding LLMs as efficient decision-making agents via offline hierarchical reinforcement learning. *arXiv preprint arXiv:2505.19761* (2025).

Hui Huang, Yancheng He, Hongli Zhou, Rui Zhang, Wei Liu, Weixun Wang, Jiaheng Liu, and Wenbo Su. 2025b. Think-J: Learning to Think for Generative LLM-as-a-Judge. (2025). arXiv:2505.14268

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Yu, Xinying Song, and Denny Zhou. 2024a. Large language models cannot self-correct reasoning yet. In *International conference on learning representations*, Vol. 2024. 32808–32824.

Jiaxin Huang, Shixiang Shane Gu, Le Hou, Yuexin Wu, Xuezhi Wang, Hongkun Yu, and Jiawei Han. 2022a. Large Language Models Can Self-Improve. *arXiv preprint arXiv:2210.11610* (2022).

Jiani Huang, Amish Sethi, Matthew Kuo, Mayank Keoliya, Neelay Velingker, JungHo Jung, Ser-Nam Lim, Ziyang Li, and Mayur Naik. 2025c. ESCA: Contextualizing Embodied Agents via Scene-Graph Generation. arXiv:2510.15963 [cs.CV] https://arxiv.org/abs/2510.15963

Lei Huang, Xiang Cheng, Chenxiao Zhao, Guobin Shen, Junjie Yang, Xiaocheng Feng, Yuxuan Gu, Xing Yu, and Bing Qin. 2026. Bootstrapping Exploration with Group-Level Natural Language Feedback in Reinforcement Learning. (2026). arXiv:2603.04597

Wenlong Huang, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, and Li Fei-Fei. 2023. Voxposer: Composable 3d value maps for robotic manipulation with language models. *arXiv preprint arXiv:2307.05973* (2023).

Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, and Yevgen Chebotar. 2022b. Inner monologue: Embodied reasoning through planning with language models. *arXiv preprint arXiv:2207.05608* (2022).

Yue Huang, Yanyuan Chen, Dexuan Xu, Weihua Yue, Huamin Zhang, Meikang Qiu, and Yu Huang. 2025a. MedReflect: Teaching Medical LLMs to Self-Improve via Reflective Correction. *arXiv preprint arXiv:2510.03687* (2025).

Yuzhen Huang, Weihao Zeng, Xingshan Zeng, Qi Zhu, and Junxian He. 2025d. From Accuracy to Robustness: A Study of Rule- and Model-based Verifiers in Mathematical Reasoning. (2025). arXiv:2505.22203

Zilin Huang, Zihao Sheng, Yansong Qu, Junwei You, and Sikai Chen. 2024b. VLM-RL: A Unified Vision Language Models and Reinforcement Learning Framework for Safe Autonomous Driving. (2024). arXiv:2412.15544

Zenan Huang, Yihong Zhuang, Guoshan Lu, Zeyu Qin, Haokai Xu, Tianyu Zhao, Ru Peng, Jiaqi Hu, Zhanming Shen, Xiaomeng Hu, Xijun Gu, Peiyi Tu, Jiaxin Liu, Wenyu Chen, Yuzhuo Fu, Zhiting Fan, Yanmei Gu, Yuanyuan Wang, Zhengkai Yang, Jianguo Li, and Junbo Zhao. 2025e. Reinforcement Learning with Rubric Anchors. (2025). arXiv:2508.12790

Minyoung Hwang, Joey Hejna, Dorsa Sadigh, and Yonatan Bisk. 2024. MotIF: Motion Instruction Fine-tuning. (2024). arXiv:2409.10683

Miaomiao Ji, Yanqiu Wu, Zhibin Wu, Shoujin Wang, Jian Yang, Mark Dras, and Usman Naseem. 2025. A Survey on Progress in LLM Alignment from the Perspective of Reward Design. (2025). arXiv:2505.02666

Shuo Ji, Yibo Li, and Bryan Hooi. 2026. Memory is Reconstructed, Not Retrieved: Graph Memory for LLM Agents. (2026). arXiv:2606.06036

Dongwei Jiang, Alvin Zhang, Andrew Wang, Nicholas Andrews, and Daniel Khashabi. 2025. Feedback Friction: LLMs Struggle to Fully Incorporate External Feedback. (2025). arXiv:2506.11930

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv:2310.06770 [cs.CL] https://arxiv.org/abs/2310.06770

Frank Joublin, Antonello Ceravola, Pavel Smirnov, Felix Ocker, Joerg Deigmoeller, Anna Belardinelli, Chao Wang, Stephan Hasler, Daniel Tanneberg, and Michael Gienger. 2024. Copal: corrective planning of robot actions with large language models. In *2024 ieee international conference on robotics and automation (ICRA)*. IEEE, 8664–8670.

Leslie Pack Kaelbling, Michael L Littman, and Anthony R Cassandra. 1998. Planning and acting in partially observable stochastic domains. *Artificial Intelligence* 101, 1–2 (1998), 99–134. doi:10.1016/S0004-3702(98)00023-X

Ryo Kamoi, Yusen Zhang, Nan Zhang, Jiawei Han, and Rui Zhang. 2024. When can llms actually correct their own mistakes? a critical survey of self-correction of llms. *Transactions of the Association for Computational Linguistics* 12 (2024), 1417–1440.

Timo Kaufmann, Paul Weng, Viktor Bengs, and Eyke Hüllermeier. 2023. A survey of reinforcement learning from human feedback. *arXiv preprint arXiv:2312.14925* (2023).

Zixuan Ke, Fangkai Jiao, Yifei Ming, Xuan-Phi Nguyen, Austin Xu, Do Xuan Long, Minzhi Li, Chengwei Qin, Peifeng Wang, Silvio Savarese, Caiming Xiong, and Shafiq Joty. 2025. A Survey of Frontiers in LLM Reasoning: Inference Scaling, Learning to Reason, and Agentic Systems. (2025). arXiv:2504.09037

Zachary Kenton, Noah Y. Siegel, János Kramár, Jonah Brown-Cohen, Samuel Albanie, Jannis Bulian, Rishabh Agarwal, David Lindner, Yunhao Tang, Noah D. Goodman, and Rohin Shah. 2024. On scalable oversight with weak LLMs judging strong LLMs. (2024). arXiv:2407.04622

Muhammad Khalifa, Rishabh Agarwal, Lajanugen Logeswaran, Jaekyeom Kim, Hao Peng, Moontae Lee, Honglak Lee, and Lu Wang. 2025. Process Reward Models That Think. arXiv:2504.16828 [cs.LG] https://arxiv.org/abs/2504.16828

Akbir Khan, John Hughes, Dan Valentine, Laura Ruis, Kshitij Sachan, Ansh Radhakrishnan, Edward Grefenstette, Samuel R. Bowman, Tim Rocktäschel, and Ethan Perez. 2024. Debating with More Persuasive LLMs Leads to More Truthful Answers. (2024). arXiv:2402.06782

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2023. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. (2023). arXiv:2310.03714

Sunghwan Kim, Dongjin Kang, Taeyoon Kwon, Hyungjoo Chae, Jungsoo Won, Dongha Lee, and Jinyoung Yeo. 2024. Evaluating Robustness of Reward Models for Mathematical Reasoning. (2024). arXiv:2410.01729

Taewoong Kim, Byeonghwi Kim, and Jonghyun Choi. 2025. Multi-modal grounded planning and efficient replanning for learning embodied agents with a few examples. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 4329–4337.

Jan Hendrik Kirchner, Yining Chen, Harri Edwards, Jan Leike, Nat McAleese, and Yuri Burda. 2024. Prover-Verifier Games improve legibility of LLM outputs. (2024). arXiv:2407.13692

Martin Klissarov, Jonathan Cook, Diego Antognini, Hao Sun, Jingling Li, Natasha Jaques, Claudiu Musat, and Edward Grefenstette. 2026. Improving Interactive In-Context Learning from Natural Language Feedback. *arXiv preprint arXiv:2602.16066* (2026).

Ryan Koo, Ian Yang, Vipul Raheja, Mingyi Hong, Kwang-Sung Jun, and Dongyeop Kang. 2025. Learning Explainable Dense Reward Shapes via Bayesian Optimization. (2025). arXiv:2504.16272

Leonard Krasner. 1958. Studies of the conditioning of verbal behavior. *Psychological Bulletin* 55, 3 (1958), 148–170. doi:10.1037/h0040492

Jakub Grudzien Kuba, Mengting Gu, Qi Ma, Yuandong Tian, Vijai Mohan, and Jason Chen. 2025. Language Self-Play For Data-Free Training. (2025). arXiv:2509.07414

Peng Lai, Zhihao Ou, Yong Wang, Longyue Wang, Jian Yang, Yun Chen, and Guanhua Chen. 2026. BiasScope: Towards Automated Detection of Bias in LLM-as-a-Judge Evaluation. (2026). arXiv:2602.09383

Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V Miranda, Alisa Liu, Nouha Dziri, and Shane Lyu. 2024a. Tulu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124* (2024).

Nathan Lambert, Valentina Pyatkin, Jacob Morrison, LJ Miranda, Bill Yuchen Lin, Khyathi Chandu, Nouha Dziri, Sachin Kumar, Tom Zick, Yejin Choi, Noah A. Smith, and Hannaneh Hajishirzi. 2024b. RewardBench: Evaluating Reward Models for Language Modeling. (2024). arXiv:2403.13787

Tian Lan, Wenwei Zhang, Chengqi Lyu, Shuaibin Li, Chen Xu, Heyan Huang, Dahua Lin, Xian-Ling Mao, and Kai Chen. 2024a. Training Language Models to Critique With Multi-agent Feedback. (2024). arXiv:2410.15287

Tian Lan, Wenwei Zhang, Chen Xu, Heyan Huang, Dahua Lin, Kai Chen, and Xian-ling Mao. 2024b. CriticEval: Evaluating Large Language Model as Critic. (2024). arXiv:2402.13764

Hao Lang, Fei Huang, and Yongbin Li. 2025. Debate Helps Weak-to-Strong Generalization. (2025). arXiv:2501.13124

Harrison Lee, Samrat Phatale, Hassan Mansoor, Thomas Mesnard, Johan Ferret, Kellie Lu, Colton Bishop, Ethan Hall, Victor Carbune, and Abhinav Rastogi. 2023. Rlaif vs. rlhf: Scaling reinforcement learning from human feedback with ai feedback. *arXiv preprint arXiv:2309.00267* (2023).

Tony Lee, Andrew Wagenmaker, Karl Pertsch, Percy Liang, Sergey Levine, and Chelsea Finn. 2026. RoboReward: General-Purpose Vision-Language Reward Models for Robotics. (2026). arXiv:2601.00675

Yu-Ang Lee, Guan-Ting Yi, Mei-Yi Liu, Jui-Chao Lu, Guan-Bo Yang, and Yun-Nung Chen. 2025. Compound AI Systems Optimization: A Survey of Methods, Challenges, and Future Directions. (2025). arXiv:2506.08234

Andrew Li, Toryn Klassen, Andrew Wang, Parand A Alamdari, and Sheila McIlraith. 2026a. Ground-Compose-Reinforce: Grounding Language in Agentic Behaviours using Limited Data. *Advances in Neural Information Processing Systems* 38 (2026), 160838–160874.

Dawei Li, Bohan Jiang, Liangjie Huang, Alimohammad Beigi, Chengshuai Zhao, Zhen Tan, Amrita Bhattacharjee, Yuxuan Jiang, Canyu Chen, Tianhao Wu, Kai Shu, Lu Cheng, and Huan Liu. 2024c. From Generation to Judgment: Opportunities and Challenges of LLM-as-a-judge. (2024). arXiv:2411.16594

Haitao Li, Qian Dong, Junjie Chen, Huixue Su, Yujia Zhou, Qingyao Ai, Ziyi Ye, and Yiqun Liu. 2024a. LLMs-as-Judges: A Comprehensive Survey on LLM-based Evaluation Methods. (2024). arXiv:2412.05579

Jiahui Li, Lin Li, Tai-wei Chang, Kun Kuang, Long Chen, Jun Zhou, and Cheng Yang. 2024d. RED: Unleashing Token-Level Rewards from Holistic Feedback via Reward Redistribution. (2024). arXiv:2411.08302

Songze Li, Chuokun Xu, Jiaying Wang, Xueluan Gong, Chen Chen, Jirui Zhang, Jun Wang, Kwok-Yan Lam, and Shouling Ji. 2025. LLMs Cannot Reliably Judge (Yet?): A Comprehensive Assessment on the Robustness of LLM-as-a-Judge. arXiv:2506.09443

Sunzhu Li, Jiale Zhao, Miteto Wei, Huimin Ren, Yang Zhou, Jingwen Yang, Shunyu Liu, Kaike Zhang, and Wei Chen. 2026b. RubricHub: A Comprehensive and Highly Discriminative Rubric Dataset via Automated Coarse-to-Fine Generation. (2026). arXiv:2601.08430

Xinzhe Li. 2025. A Survey on LLM Test-Time Compute via Search: Tasks, LLM Profiling, Search Algorithms, and Relevant Frameworks. (2025). arXiv:2501.10069

Yunxuan Li, Yibing Du, Jiageng Zhang, Le Hou, Peter Grabowski, Yeqing Li, and Eugene Ie. 2024b. Improving multi-agent debate with sparse communication topology. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 7281–7294.

Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. 2022. Code as policies: Language model programs for embodied control, 2023. *URL https://arxiv. org/abs/2209.07753* 3 (2022).

Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Shuming Shi, and Zhaopeng Tu. 2024a. Encouraging Divergent Thinking in Large Language Models through Multi-Agent Debate. arXiv:2305.19118 [cs.CL] https://arxiv.org/abs/2305.19118

Xun Liang, Shichao Song, Zifan Zheng, Hanyu Wang, Qingchen Yu, Xunkai Li, Rong-Hua Li, Yi Wang, Zhonghao Wang, Feiyu Xiong, and Zhiyu Li. 2024b. Internal Consistency and Self-Feedback in Large Language Models: A Survey. (2024). arXiv:2407.14507

Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's verify step by step. In *International Conference on Learning Representations*, Vol. 2024. 39578–39601. arXiv:2305.20050

Huawei Lin, Peng Li, Jie Song, Fuxin Jiang, and Tieying Zhang. 2026. MUSE-Autoskill: Self-Evolving Agents via Skill Creation, Memory, Management, and Evaluation. (2026). arXiv:2605.27366

Matthieu Lin, Jenny Sheng, Andrew Zhao, Shenzhi Wang, Yang Yue, Victor Shea Jay Huang, Huan Liu, Jun Liu, Gao Huang, and Yong-Jin Liu. 2024b. Scaffolded Language Models with Language Supervision for Mixed-Autonomy: A Survey. (2024). arXiv:2410.16392

Muhan Lin, Shuyang Shi, Yue Guo, Vaishnav Tadiparthi, Behdad Chalaki, Ehsan Moradi Pari, Simon Stepputtis, Woojun Kim, Joseph Campbell, and Katia Sycara. 2025b. Speaking the Language of Teamwork: LLM-Guided Credit Assignment in Multi-Agent Reinforcement Learning. (2025). arXiv:2502.03723

Zijun Lin, Jiafei Duan, Haoquan Fang, Dieter Fox, Ranjay Krishna, Cheston Tan, and Bihan Wen. 2025a. FailSafe: Reasoning and Recovery from Failures in Vision-Language-Action Models. (2025). arXiv:2510.01642

Zicheng Lin, Zhibin Gou, Tian Liang, Ruilin Luo, Haowei Liu, and Yujiu Yang. 2024a. Criticbench: Benchmarking llms for critique-correct reasoning. In *Findings of the Association for Computational Linguistics: ACL 2024*. 1552–1587.

Zijie Lin and Bryan Hooi. 2025. Enhancing Multi-Agent Debate System Performance via Confidence Expression. (2025). arXiv:2509.14034

Chenxi Liu, Yanshuo Chen, Ruibo Chen, Tianyi Xiong, Tong Zheng, and Heng Huang. 2026a. Learning from Self-Debate: Preparing Reasoning Models for Multi-Agent Debate. arXiv:2601.22297 [cs.CL] https://arxiv.org/abs/2601.22297

Hongyi Liu, Haoyan Yang, Tao Jiang, Bo Tang, Feiyu Xiong, Yuyu Luo, and Zhiyu Li. 2026f. SkillsVote: Lifecycle Governance of Agent Skills from Collection, Recommendation to Evolution. (2026). arXiv:2605.18401

Pangpang Liu, Chengchun Shi, and Will Wei Sun. 2026c. Reinforcement Learning from Human Feedback: A Statistical Perspective. (2026). arXiv:2604.02507

Qingshan Liu, Guoqing Wang, Wen Wu, Jingqi Huang, Xinqi Tao, Dejia Song, Jie Zhou, and Liang He. 2026d. MemPro: Agentic Memory Systems as Evolvable Programs. (2026). arXiv:2606.00619

Qiyuan Liu, Hao Xu, Xuhong Chen, Wei Chen, Yee Whye Teh, and Ning Miao. 2025c. Enhancing Large Language Model Reasoning with Reward Models: An Analytical Survey. (2025). arXiv:2510.01925

ShaoZhen Liu, Xinting Huang, Houwen Peng, Xin Chen, Xinyang Song, Qi Li, and Zhenan Sun. 2026b. Dual-Phase LLM Reasoning: Self-Evolved Mathematical Frameworks. *arXiv preprint arXiv:2601.05616* (2026).

Shiyan Liu, Qifeng Xia, Qiyun Xia, Yisheng Liu, Xinyu Yu, and Rui Qu. 2026e. Reflection in the Dark: Exposing and Escaping the Black Box in Reflective Prompt Optimization. (2026). arXiv:2603.18388

Tianci Liu, Ran Xu, Tony Yu, Ilgee Hong, Carl Yang, Tuo Zhao, and Haoyu Wang. 2025d. OpenRubrics: Towards Scalable Synthetic Rubric Generation for Reward Modeling and LLM Alignment. (2025). arXiv:2510.07743

Xiaoqian Liu, Ke Wang, Yuchuan Wu, Fei Huang, Yongbin Li, Junge Zhang, and Jianbin Jiao. 2025a. Agentic Reinforcement Learning with Implicit Step Rewards. arXiv:2509.19199 [cs.CL] https://arxiv.org/abs/2509.19199

Yantao Liu, Zijun Yao, Rui Min, Yixin Cao, Lei Hou, and Juanzi Li. 2024. RM-Bench: Benchmarking Reward Models of Language Models with Subtlety and Style. (2024). arXiv:2410.16184

Zijun Liu, Peiyi Wang, Runxin Xu, Shirong Ma, Chong Ruan, Peng Li, Yang Liu, and Yu Wu. 2025b. Inference-Time Scaling for Generalist Reward Modeling. (2025). arXiv:2504.02495

Saüc Abadal Lloret, Shehzaad Dhuliawala, Keerthiram Murugesan, and Mrinmaya Sachan. 2024. Towards aligning language models with textual feedback. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 20240–20266.

Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. 2024. The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292* (2024).

Guanxing Lu, Wenkai Guo, Chubin Zhang, Yuheng Zhou, Haonan Jiang, Zifeng Gao, Yansong Tang, and Ziwei Wang. 2025. VLA-RL: Towards Masterful and General Robotic Manipulation with Scalable Reinforcement Learning. (2025). arXiv:2505.18719

Xinyu Lu, Kaiqi Zhang, Jinglin Yang, Boxi Cao, Yaojie Lu, Hongyu Lin, Min He, Xianpei Han, and Le Sun. 2026. P^2O: Joint Policy and Prompt Optimization. (2026). arXiv:2603.21877

Jelena Luketina, Nantas Nardelli, Gregory Farquhar, Jakob Foerster, Jacob Andreas, Edward Grefenstette, Shimon Whiteson, and Tim Rocktäschel. 2019. A survey of reinforcement learning informed by natural language. *arXiv preprint arXiv:1906.03926* (2019).

Liangchen Luo, Yinxiao Liu, Rosanne Liu, Samrat Phatale, Meiqi Guo, Harsh Lara, Yunxuan Li, Lei Shu, Yun Zhu, and Lei Meng. 2024. Improve mathematical reasoning in language models by automated process supervision. *arXiv preprint arXiv:2406.06592* (2024).

Renjie Luo, Zichen Liu, Xiangyan Liu, Chao Du, Min Lin, Wenhu Chen, Wei Lu, and Tianyu Pang. 2025. Language models can learn from verbal feedback without scalar rewards. *arXiv preprint arXiv:2509.22638* (2025).

Corey Lynch and Pierre Sermanet. 2021. Language Conditioned Imitation Learning over Unstructured Data. In *Robotics: Science and Systems*.

Xing Han Lù, Amirhossein Kazemnejad, Nicholas Meade, Arkil Patel, Dongchan Shin, Alejandra Zambrano, Karolina Stańczak, Peter Shaw, Christopher J. Pal, and Siva Reddy. 2025. AgentRewardBench: Evaluating Automatic Evaluations of Web Agent Trajectories. (2025). arXiv:2504.08942

Qiyao Ma, Dechen Gao, Rui Cai, Boqi Zhao, Hanchu Zhou, Junshan Zhang, and Zhe Zhao. 2026. Personalized RewardBench: Evaluating Reward Models with Human Aligned Personalization. (2026). arXiv:2604.07343

Ruotian Ma, Xiaolei Wang, Xin Zhou, Jian Li, Nan Du, Tao Gui, Qi Zhang, and Xuanjing Huang. 2024c. Are Large Language Models Good Prompt Optimizers? (2024). arXiv:2402.02101

Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, and Jim Fan. 2024a. Eureka: Human-level reward design via coding large language models. In *International conference on learning Representations*, Vol. 2024. 26516–26560.

Yecheng Jason Ma, William Liang, Hung-Ju Wang, Sam Wang, Yuke Zhu, Linxi Fan, Osbert Bastani, and Dinesh Jayaraman. 2024b. DrEureka: Language Model Guided Sim-To-Real Transfer. (2024). arXiv:2406.01967

James MacGlashan, Mark K Ho, Robert Loftin, Bei Peng, Guan Wang, David L Roberts, Matthew E Taylor, and Michael L Littman. 2017. Interactive learning from policy-dependent human feedback. In *International conference on machine learning*. PMLR, 2285–2294. arXiv:1701.06049

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*.

Anas Mahmoud, MohammadHossein Rezaei, Zihao Wang, Anisha Gunjal, Bing Liu, and Yunzhong He. 2026. Reward Hacking in Rubric-Based Reinforcement Learning. (2026). arXiv:2605.12474

Saumya Malik, Valentina Pyatkin, Sander Land, Jacob Morrison, Noah A. Smith, Hannaneh Hajishirzi, and Nathan Lambert. 2025. RewardBench 2: Advancing Reward Model Evaluation. (2025). arXiv:2506.01937

Lars Malmqvist. 2024. Sycophancy in Large Language Models: Causes and Mitigations. arXiv:2411.15287

Narek Maloyan and Dmitry Namiot. 2025. Adversarial Attacks on LLM-as-a-Judge Systems: Insights from Prompt Injections. arXiv:2504.18333

Nat McAleese, Rai Michael Pokorny, Juan Felipe Ceron Uribe, Evgenia Nitishinskaya, Maja Trebacz, and Jan Leike. 2024. LLM Critics Help Catch LLM Bugs. *arXiv preprint arXiv:2407.00215* (2024).

Lingrui Mei, Jiayu Yao, Yuyao Ge, Yiwei Wang, Baolong Bi, Yujun Cai, Jiazhi Liu, Mingyu Li, Zhong-Zhi Li, Duzhen Zhang, Chenlin Zhou, Jiayi Mao, Tianze Xia, Jiafeng Guo, and Shenghua Liu. 2025. A Survey of Context Engineering for Large Language Models. (2025). arXiv:2507.13334

Yu Meng, Mengzhou Xia, and Danqi Chen. 2024. Simpo: Simple preference optimization with a reference-free reward. *Advances in Neural Information Processing Systems* 37 (2024), 124198–124235.

Azalia Mirhoseini, Anna Goldie, Mustafa Yazgan, Joe Wenjie Jiang, Ebrahim Songhori, Shen Wang, Young-Joon Lee, Eric Johnson, Omkar Pathak, and Azade Nova. 2021. A graph placement methodology for fast chip design. *Nature* 594, 7862 (2021), 207–212.

Dipendra Misra, John Langford, and Yoav Artzi. 2017. Mapping instructions and visual observations to actions with reinforcement learning. In *Proceedings of the 2017 conference on empirical methods in natural language processing*. 1004–1015.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, and Georg Ostrovski. 2015. Human-level control through deep reinforcement learning. *Nature* 518, 7540 (2015), 529–533.

Sumeet Ramesh Motwani, Chandler Smith, Rocktim Jyoti Das, Rafael Rafailov, Ivan Laptev, Philip H. S. Torr, Fabio Pizzati, Ronald Clark, and Christian Schroeder de Witt. 2024. MALT: Improving Reasoning with Multi-Agent LLM Training. (2024). arXiv:2412.01928

Vaskar Nath, Pranav Raja, Claire Yoon, and Sean Hendryx. 2025. ToolComp: A Multi-Tool Reasoning & Process Supervision Benchmark. (2025). arXiv:2501.01290

Manh Nguyen, Anh Nguyen, Dung Nguyen, Svetha Venkatesh, and Hung Le. 2026. Hear Both Sides: Efficient Multi-Agent Debate via Diversity-Aware Message Retention. (2026). arXiv:2603.20640

Allen Nie, Ching-An Cheng, Andrey Kolobov, and Adith Swaminathan. 2024. The Importance of Directional Feedback for LLM-based Optimizers. (2024). arXiv:2405.16434

Xuying Ning, Katherine Tieu, Dongqi Fu, Tianxin Wei, Zihao Li, Yuanchen Bei, Jiaru Zou, Mengting Ai, Zhining Liu, and Ting-Wei Li. 2026. Code as Agent Harness. *arXiv preprint arXiv:2605.18747* (2026).

Theo X Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. 2024. Is self-repair a silver bullet for code generation?. In *International Conference on Learning Representations*, Vol. 2024. 36545–36593.

Krista Opsahl-Ong, Michael J Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. 2024. Optimizing Instructions and Demonstrations for Multi-Stage Language Model Programs. In *Conference on Empirical Methods in Natural Language Processing*.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, and Alex Ray. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems* 35 (2022), 27730–27744.

Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. 2025. ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory. (2025). arXiv:2509.25140

Paul Pacaud, Ricardo Garcia, Shizhe Chen, and Cordelia Schmid. 2025. Scaling Cross-Environment Failure Reasoning Data for Vision-Language Robotic Manipulation. (2025). arXiv:2512.01946

Alexander Pan, Erik Jones, Meena Jagadeesan, and Jacob Steinhardt. 2024. Feedback Loops With Language Models Drive In-Context Reward Hacking. arXiv:2402.06627

Liangming Pan, Michael Saxon, Wenda Xu, Deepak Nathani, Xinyi Wang, and William Yang Wang. 2023. Automatically Correcting Large Language Models: Surveying the landscape of diverse self-correction strategies. (2023). arXiv:2308.03188

Manav Pandey. 2026. LLMs Know They're Wrong and Agree Anyway: The Shared Sycophancy-Lying Circuit. arXiv:2604.19117

Henry Papadatos and Rachel Freedman. 2024. Linear Probe Penalties Reduce LLM Sycophancy. arXiv:2412.00967

Chanwoo Park, Seungju Han, Xingzhi Guo, Asuman Ozdaglar, Kaiqing Zhang, and Joo-Kyung Kim. 2025. MAPoRL: Multi-Agent Post-Co-Training for Collaborative Large Language Models with Reinforcement Learning. (2025). arXiv:2502.18439

Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *ACM Symposium on User Interface Software and Technology (UIST)*.

Pankayaraj Pathmanathan, Souradip Chakraborty, Xiangyu Liu, Yongyuan Liang, and Furong Huang. 2024. Is poisoning a real threat to LLM alignment? Maybe more so than you think. arXiv:2406.12091

Debjit Paul, Mete Ismayilzada, Maxime Peyrard, Beatriz Borges, Antoine Bosselut, Robert West, and Boi Faltings. 2024. REFINER: Reasoning Feedback on Intermediate Representations. *arXiv preprint arXiv:2304.01904* (2024).

Eduardo Pignatelli, Johan Ferret, Tim Rockäschel, Edward Grefenstette, Davide Paglieri, Samuel Coward, and Laura Toni. 2024. Assessing the Zero-Shot Capabilities of LLMs for Action Evaluation in RL. (2024). arXiv:2409.12798

Keyang Qian, Yixin Cheng, Rui Guan, Wei Dai, Flora Jin, Kaixun Yang, Sadia Nawaz, Zachari Swiecki, Guanliang Chen, and Lixiang Yan. 2025. Dean of LLM Tutors: A Framework for Automated Quality Review of AI-generated Feedback. *arXiv preprint arXiv:2508.05952* (2025).

Yun Qu, Yuhang Jiang, Boyuan Wang, Yixiu Mao, Cheems Wang, Chang Liu, and Xiangyang Ji. 2024. Latent Reward: LLM-Empowered Credit Assignment in Episodic Reinforcement Learning. (2024). arXiv:2412.11120

Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems* 36 (2023), 53728–53741.

Salman Rahman, Sruthi Gorantla, Arpit Gupta, Swastik Roy, Nanyun Peng, and Yang Liu. 2025. SPARK: Stepwise Process-Aware Rewards for Reference-Free Reinforcement Learning. (2025). arXiv:2512.03244

Vyas Raina, Adian Liusie, and Mark Gales. 2024. Is LLM-as-a-Judge Robust? Investigating Universal Adversarial Attacks on Zero-shot LLM Assessment. arXiv:2402.14016

Kiran Ramnath, Kang Zhou, Sheng Guan, Soumya Smruti Mishra, Xuan Qi, Zhengyuan Shen, Shuai Wang, Sangmin Woo, Sullam Jeoung, Yawei Wang, Haozhu Wang, Han Ding, Yuzhe Lu, Zhichao Xu, Yun Zhou, Balasubramaniam Srinivasan, Qiaojing Yan, Yueyan Chen, Haibo Ding, Panpan Xu, and Lin Lee Cheong. 2025. A Systematic Survey of Automatic Prompt Optimization Techniques. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 33066–33098. doi:10.18653/v1/2025.emnlp-main.1681

MohammadHossein Rezaei, Robert Vacareanu, Zihao Wang, Clinton Wang, Bing Liu, Yunzhong He, and Afra Feyza Akyürek. 2025. Online Rubrics Elicitation from Pairwise Comparisons. (2025). arXiv:2510.07284

Amit Roth, Ankur Samanta, Matan Halevy, Yoav Levine, and Yonathan Efroni. 2026. Hack-Verifiable Environments: Towards Evaluating Reward Hacking at Scale. arXiv:2605.20744

Swarnadeep Saha, Xian Li, Marjan Ghazvininejad, Jason Weston, and Tianlu Wang. 2025. Learning to Plan & Reason for Evaluation with Thinking-LLM-as-a-Judge. (2025). arXiv:2501.18099

Yusuke Sakai, Hidetaka Kamigaito, and Taro Watanabe. 2025. Revisiting compositional generalization capability of large language models considering instruction following ability. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 31219–31238.

Bidipta Sarkar, Warren Xia, C. Karen Liu, and Dorsa Sadigh. 2025. Training Language Models for Social Deduction with Multi-Agent Reinforcement Learning. (2025). arXiv:2502.06060

Jérémy Scheurer, Jon Ander Campos, Tomasz Korbak, Jun Shern Chan, Angelica Chen, Kyunghyun Cho, and Ethan Perez. 2023. Training language models with language feedback at scale. *arXiv preprint arXiv:2303.16755* (2023).

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems* 36 (2023), 68539–68551.

Sheila Schoepp, Masoud Jafaripour, Yingyue Cao, Tianpei Yang, Fatemeh Abdollahi, Shadan Golestan, Zahin Sufiyan, Osmar R. Zaiane, and Matthew E. Taylor. 2025. The Evolving Landscape of LLM- and VLM-Integrated Reinforcement Learning. (2025). arXiv:2502.15214

Philip Schroeder, Thomas Weng, Karl Schmeckpeper, Eric Rosen, Stephen Hart, and Ondrej Biza. 2026. SOLE-R1: Video-Language Reasoning as the Sole Reward for On-Robot Reinforcement Learning. (2026). arXiv:2603.28730

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).

Leo Schwinn, Moritz Ladenburger, Tim Beyer, Mehrnaz Mofakhami, Gauthier Gidel, and Stephan Günnemann. 2026. A Coin Flip for Safety: LLM Judges Fail to Reliably Measure Adversarial Robustness. arXiv:2603.06594

Mrinank Sharma, Meg Tong, Tomek Korbak, David Duvenaud, Amanda Askell, Sam Bowman, Esin Durmus, Zac Hatfield-Dodds, Scott Johnston, and Shauna Kravec. 2024. Towards understanding sycophancy in language models. In *International Conference on Learning Representations*, Vol. 2024. 110–144.

Pratyusha Sharma, Balakumar Sundaralingam, Valts Blukis, Chris Paxton, Tucker Hermans, Antonio Torralba, Jacob Andreas, and Dieter Fox. 2022. Correcting robot plans with natural language feedback. *arXiv preprint arXiv:2204.05186* (2022).

Haizhou Shi, Ye Liu, Bo Pang, Zeyu Leo Liu, Hao Wang, Silvio Savarese, Caiming Xiong, Yingbo Zhou, and Semih Yavuz. 2025a. SSR: Socratic Self-Refine for Large Language Model Reasoning. *arXiv preprint arXiv:2511.10621* (2025).

Jiawen Shi, Zenghui Yuan, Yinuo Liu, Yue Huang, Pan Zhou, Lichao Sun, and Neil Zhenqiang Gong. 2024b. Optimization-based Prompt Injection Attack to LLM-as-a-Judge. arXiv:2403.17710

Jiawen Shi, Zenghui Yuan, Guiyao Tie, Pan Zhou, Neil Zhenqiang Gong, and Lichao Sun. 2025b. Prompt injection attack to tool selection in llm agents. *arXiv preprint arXiv:2504.19793* (2025).

Lucy Xiaoyang Shi, Zheyuan Hu, Tony Z. Zhao, Archit Sharma, Karl Pertsch, Jianlan Luo, Sergey Levine, and Chelsea Finn. 2024a. Yell At Your Robot: Improving On-the-Fly from Language Corrections. (2024). arXiv:2403.12910

Noah Shinn, Federico Cassano, Berman Labash, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, Vol. 36.

Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. 2020. Alfworld: Aligning text and embodied environments for interactive learning. *arXiv preprint arXiv:2010.03768* (2020).

Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Yarin Gal, Nicolas Papernot, and Ross Anderson. 2023. The curse of recursion: Training on generated data makes models forget. *arXiv preprint arXiv:2305.17493* (2023).

David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, and Marc Lanctot. 2016. Mastering the game of Go with deep neural networks and tree search. *nature* 529, 7587 (2016), 484–489.

Avi Singh, John D Co-Reyes, Rishabh Agarwal, Ankesh Anand, Piyush Patil, Xavier Garcia, Peter J Liu, James Harrison, Jaehoon Lee, and Kelvin Xu. 2023. Beyond human data: Scaling self-training for problem-solving with language models. *arXiv preprint arXiv:2312.06585* (2023).

Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. 2022. Progprompt: Generating situated robot task plans using large language models. *arXiv preprint arXiv:2209.11302* (2022).

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2024. Scaling LLM Test-Time Compute Optimally Can Be More Effective than Scaling Model Parameters. *arXiv preprint arXiv:2408.03314* (2024).

Guijin Son, Dongkeun Yoon, Juyoung Suk, Javier Aula-Blasco, Mano Aslan, Vu Trong Kim, Shayekh Bin Islam, Jaume Prats-Cristià, Lucía Tormo-Bañuelos, and Seungone Kim. 2024. MM-Eval: A Multilingual Meta-Evaluation Benchmark for LLM-as-a-Judge and Reward Models. (2024). arXiv:2410.17578

Mingyang Song, Zhaochen Su, Xiaoye Qu, Jiawei Zhou, and Yu Cheng. 2025. PRMBench: A Fine-grained and Challenging Benchmark for Process-Level Reward Models. (2025). arXiv:2501.03124

Saksham Sahai Srivastava and Vaneet Aggarwal. 2025. A Technical Survey of Reinforcement Learning Techniques for Large Language Models. (2025). arXiv:2507.04136

Elias Stengel-Eskin, Peter Hase, and Mohit Bansal. 2024. Teaching Models to Balance Resisting and Accepting Persuasion. (2024). arXiv:2410.14596

Moritz Stephan, Alexander Khazatsky, Eric Mitchell, Annie S Chen, Sheryl Hsu, Archit Sharma, and Chelsea Finn. 2024. RLVF: Learning from Verbal Feedback without Overgeneralization. (2024). arXiv:2402.10893

Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F Christiano. 2020. Learning to summarize with human feedback. *Advances in neural information processing systems* 33 (2020), 3008–3021. arXiv:2009.01325

Alexander L Strehl, Lihong Li, and Michael L Littman. 2009. Reinforcement Learning in Finite MDPs: PAC Analysis. *Journal of Machine Learning Research* 10, 84 (2009), 2413–2444.

Vighnesh Subramaniam, Yilun Du, Joshua B. Tenenbaum, Antonio Torralba, Shuang Li, and Igor Mordatch. 2025. Multiagent Finetuning: Self Improvement with Diverse Reasoning Chains. (2025). arXiv:2501.05707

Aleksa Sukovic and Goran Radanovic. 2024. Reward Design for Justifiable Sequential Decision-Making. (2024). arXiv:2402.15826

Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. 2023. Cognitive Architectures for Language Agents. (2023). arXiv:2309.02427

Shengjie Sun, Runze Liu, Jiafei Lyu, Jing-Wen Yang, Liangpeng Zhang, and Xiu Li. 2025a. A large language model-driven reward design framework via dynamic feedback for reinforcement learning. *Knowledge-Based Systems* 326 (2025), 114065.

Shengjie Sun, Jiafei Lyu, Runze Liu, Mengbei Yan, Bo Liu, Deheng Ye, and Xiu Li. 2025b. PROF: An LLM-based Reward Code Preference Optimization Framework for Offline Imitation Learning. *arXiv preprint arXiv:2511.13765* (2025).

Rohan Surana, Gagan Mundada, Xunyi Jiang, Chuhan Wang, Zhenwei Tang, Difan Jiao, Zihan Huang, Yuxin Xiong, Junda Wu, Sheldon Yu, Xintong Li, Raghav Jain, Nikki Kuang, Sizhe Zhou, Bowen Jin, Zhendong Chu, Tong Yu, Ryan Rossi, Kuan-Hao Huang, Jingbo Shang, Jiawei Han, and Julian McAuley. 2026. Generate, Filter, Control, Replay: A Comprehensive Survey of Rollout Strategies for LLM Reinforcement Learning. (2026). arXiv:2605.02913

Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. 2026. Dynamic Cheatsheet: Test-Time Learning with Adaptive Memory. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, Vera Demberg, Kentaro Inui, and Lluís Marquez (Eds.). Association for Computational Linguistics, Rabat, Morocco, 7080–7106. doi:10.18653/v1/2026.eacl-long.333

Huajie Tan, Sixiang Chen, Yijie Xu, Zixiao Wang, Yuheng Ji, Cheng Chi, Yaoxu Lyu, Zhongxia Zhao, Xiansheng Chen, Peterson Co, Shaoxuan Xie, Guocai Yao, Pengwei Wang, Zhongyuan Wang, and Shanghang Zhang. 2025a. Robo-Dopamine: General Process Reward Modeling for High-Precision Robotic Manipulation. (2025). arXiv:2512.23703

Haoran Tan, Zeyu Zhang, Chen Ma, Xu Chen, Quanyu Dai, and Zhenhua Dong. 2025b. Membench: Towards more comprehensive evaluation on the memory of llm-based agents. In *Findings of the Association for Computational Linguistics: ACL 2025*. 19336–19352.

Sijun Tan, Siyuan Zhuang, Kyle Montgomery, William Y. Tang, Alejandro Cuadron, Chenguang Wang, Raluca Ada Popa, and Ion Stoica. 2024. JudgeBench: A Benchmark for Evaluating LLM-based Judges. (2024). arXiv:2410.12784

Zhengyang Tang, Ziniu Li, Zhenyang Xiao, Tian Ding, Ruoyu Sun, Benyou Wang, Dayiheng Liu, Fei Huang, Tianyu Liu, Bowen Yu, and Junyang Lin. 2025. RealCritic: Towards Effectiveness-Driven Evaluation of Language Model Critiques. (2025). arXiv:2501.14492

Zhengwei Tao, Ting-En Lin, Xiancai Chen, Hangyu Li, Yuchuan Wu, Yongbin Li, Zhi Jin, Fei Huang, Dacheng Tao, and Jingren Zhou. 2024. A Survey on Self-Evolution of Large Language Models. (2024). arXiv:2404.14387

Mia Taylor, James Chua, Jan Betley, Johannes Treutlein, and Owain Evans. 2025. School of Reward Hacks: Hacking harmless tasks generalizes to misaligned behavior in LLMs. arXiv:2508.17511

Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. 2022. Solving math word problems with process-and outcome-based feedback. *arXiv preprint arXiv:2211.14275* (2022).

Peter Vamplew, Benjamin J. Smith, Johan Kallstrom, Gabriel Ramos, Roxana Radulescu, Diederik M. Roijers, Conor F. Hayes, Fredrik Heintz, Patrick Mannion, Pieter J. K. Libin, Richard Dazeley, and Cameron Foale. 2021. Scalar reward is not enough: A response to Silver, Singh, Precup and Sutton (2021). arXiv:2112.15422 [cs.AI] https://arxiv.org/abs/2112.15422

V Venktesh, Mandeep Rathee, and Avishek Anand. 2025. Trust but Verify! A Survey on Verification Design for Test-time Scaling. (2025). arXiv:2508.16665

Vijay Viswanathan, Yanchao Sun, Shuang Ma, Xiang Kong, Meng Cao, Graham Neubig, and Tongshuang Wu. 2025. Checklists Are Better Than Reward Models For Aligning Language Models. (2025). arXiv:2507.18624

Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. 2024. The instruction hierarchy: Training llms to prioritize privileged instructions. *arXiv preprint arXiv:2404.13208* (2024).

Ante Wang, Linfeng Song, Ye Tian, Dian Yu, Haitao Mi, Xiangyu Duan, Zhaopeng Tu, Jinsong Su, and Dong Yu. 2025g. Don't Get Lost in the Trees: Streamlining LLM Reasoning by Overcoming Tree Search Exploration Pitfalls. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 23946–23959. doi:10.18653/v1/2025.acl-long.1167

Chenglong Wang, Yongyu Mu, Hang Zhou, Yifu Huo, Ziming Zhu, Jiali Zeng, Murun Yang, Bei Li, Xiaoyang Hao, Chunliang Zhang, Fandong Meng, Jingbo Zhu, and Tong Xiao. 2025f. GRAM-R^2: Self-Training Generative Foundation Reward Models for Reward Reasoning. (2025). arXiv:2509.02492

Guoqing Wang, Sunhao Dai, Guangze Ye, Zeyu Gan, Wei Yao, Yong Deng, Xiaofeng Wu, and Zhenzhe Ying. 2025a. Information Gain-based Policy Optimization: A Simple and Effective Approach for Multi-Turn Search Agents. (2025). arXiv:2510.14967

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023b. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv preprint arXiv:2305.16291* (2023).

Hanyang Wang, Lu Wang, Chaoyun Zhang, Tianjun Mao, Si Qin, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. 2025h. Text2Grad: Reinforcement Learning from Natural Language Feedback. *arXiv preprint arXiv:2505.22338* (2025).

Jiaxuan Wang, Yulan Hu, Wenjin Yang, Zheng Pan, Xin Li, and Lan-Zhe Guo. 2026b. Aligning Agents via Planning: A Benchmark for Trajectory-Level Reward Modeling. (2026). arXiv:2604.08178

Jianyu Wang, Zhiqiang Hu, and Lidong Bing. 2025c. Evolving Prompts In-Context: An Open-ended, Self-replicating Perspective. (2025). arXiv:2506.17930

Peiyi Wang, Lei Li, Zhihong Shao, RX Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. 2024d. Math-Shepherd: Verify and Reinforce LLMs Step-by-step without Human Annotations. *arXiv preprint arXiv:2312.08935* (2024).

Pengkai Wang, Pengwei Liu, Qi Zuo, Zhijie Sang, Congkai Xie, and Hongxia Yang. 2025e. InfiMed-ORBIT: Aligning LLMs on Open-Ended Complex Tasks via Rubric-Based Incremental Training. (2025). arXiv:2510.15859

Qibin Wang, Pu Zhao, Shaohan Huang, Fangkai Yang, Lu Wang, Furu Wei, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. 2025i. Learning to refine: Self-refinement of parallel reasoning in llms. *arXiv preprint arXiv:2509.00084* (2025).

Shuhe Wang, Shengyu Zhang, Jie Zhang, Runyi Hu, Xiaoya Li, Tianwei Zhang, Jiwei Li, Fei Wu, Guoyin Wang, and Eduard Hovy. 2024g. Reinforcement Learning Enhanced LLMs: A Survey. (2024). arXiv:2412.10400

Tianlu Wang, Ilia Kulikov, Olga Golovneva, Ping Yu, Weizhe Yuan, Jane Dwivedi-Yu, Richard Yuanzhe Pang, Maryam Fazel-Zarandi, Jason Weston, and Xian Li. 2024c. Self-Taught Evaluators. (2024). arXiv:2408.02666

Tianlu Wang, Ping Yu, Xiaoqing Ellen Tan, Sean O'Brien, Ramakanth Pasunuru, Jane Dwivedi-Yu, Olga Golovneva, Luke Zettlemoyer, Maryam Fazel-Zarandi, and Asli Celikyilmaz. 2023c. Shepherd: A Critic for Language Model Generation. *arXiv preprint arXiv:2308.04592* (2023).

Wenyi Wang, Hisham A. Alyahya, Dylan R. Ashley, Oleg Serikov, Dmitrii Khizbullin, Francesco Faccio, and Jürgen Schmidhuber. 2024a. How to Correctly do Semantic Backpropagation on Language-based Agentic Systems. (2024). arXiv:2412.03624

Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. 2024b. Executable code actions elicit better llm agents. In *Forty-first International Conference on Machine Learning*.

Xuekang Wang, Zhuoyuan Hao, Shuo Hou, Hao Peng, Juanzi Li, and Xiaozhi Wang. 2026a. Reproducing, Analyzing, and Detecting Reward Hacking in Rubric-Based Reinforcement Learning. arXiv:2606.04923

Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, and Jaskirat Singh. 2025d. Openhands: An open platform for ai software developers as generalist agents. In *International Conference on Learning Representations*, Vol. 2025. 65882–65919.

Xingyao Wang, Zihan Wang, Jiateng Liu, Yangyi Chen, Lifan Yuan, Hao Peng, and Heng Ji. 2023a. MINT: Evaluating LLMs in Multi-turn Interaction with Tools and Language Feedback. (2023). arXiv:2309.10691

Yufei Wang, Zhanyi Sun, Jesse Zhang, Zhou Xian, Erdem Biyik, David Held, and Zackory Erickson. 2024f. RL-VLM-F: Reinforcement Learning from Vision Language Foundation Model Feedback. (2024). arXiv:2402.03681

Zongqi Wang, Tianle Gu, Chen Gong, Xin Tian, Siqi Bao, and Yujiu Yang. 2025b. SCAN: Structured Capability Assessment and Navigation for LLMs. (2025). arXiv:2505.06698

Zongqi Wang, Rui Wang, Yuchuan Wu, Yiyao Yu, Pinyi Zhang, Shaoning Sun, Yujiu Yang, and Yongbin Li. 2026c. Reward Modeling from Natural Language Human Feedback. (2026). arXiv:2601.07349

Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2024e. Agent Workflow Memory. (2024). arXiv:2409.07429

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.

Jiaqi Wei, Xiang Zhang, Yuejin Yang, Wenxuan Huang, Juntai Cao, Sheng Xu, Xiang Zhuang, Zhangyang Gao, Muhammad Abdul-Mageed, Laks V. S. Lakshmanan, Chenyu You, Wanli Ouyang, and Siqi Sun. 2025c. Unifying Tree Search Algorithm and Reward Design for LLM Reasoning: A Survey. (2025). arXiv:2510.09988

Quan Wei, Siliang Zeng, Chenliang Li, William Brown, Oana Frunza, Wei Deng, Anderson Schneider, Yuriy Nevmyvaka, Yang Katie Zhao, Alfredo Garcia, and Mingyi Hong. 2025b. Reinforcing Multi-Turn Reasoning in LLM Agents via Turn-Level Reward Design. (2025). arXiv:2505.11821

Tianxin Wei, Noveen Sachdeva, Benjamin Coleman, Zhankui He, Yuanchen Bei, Xuying Ning, Mengting Ai, Yunzhe Li, Jingrui He, Ed H. Chi, Chi Wang, Shuo Chen, Fernando Pereira, Wang-Cheng Kang, and Derek Zhiyuan Cheng. 2025a. Evo-Memory: Benchmarking LLM Agent Test-time Learning with Self-Evolving Memory. (2025). arXiv:2511.20857

Bosi Wen, Yilin Niu, Cunxiang Wang, Xiaoying Ling, Ying Zhang, Pei Ke, Hongning Wang, and Minlie Huang. 2026. IF-RewardBench: Benchmarking Judge Models for Instruction-Following Evaluation. (2026). arXiv:2603.04738

Chenxi Whitehouse, Tianlu Wang, Ping Yu, Xian Li, Jason Weston, Ilia Kulikov, and Swarnadeep Saha. 2025. J1: Incentivizing Thinking in LLM-as-a-Judge via Reinforcement Learning. (2025). arXiv:2505.10320

Fang Wu, Aaron Tu, Weihao Xuan, Heli Qi, Xu Huang, Qingcheng Zeng, Shayan Talaei, Yijia Xiao, Peng Xia, Xiangru Tang, Yuchen Zhuang, Yinxi Li, Bing Hu, Hanqun Cao, Wenqi Shi, Rui Yang, Nan Liu, Huaxiu Yao, Ge Liu, Li Erran Li, Amin Saberi, Naoto Yokoya, Jure Leskovec, and Yejin Choi. 2025a. Position: The Hidden Costs and Measurement Gaps of Reinforcement Learning with Verifiable Rewards. (2025). arXiv:2509.21882

Mian Wu, Gavin Zhang, Sewon Min, Sergey Levine, and Aviral Kumar. 2025b. RLAC: Reinforcement Learning with Adversarial Critic for Free-Form Generation Tasks. (2025). arXiv:2511.01758

Shengguang Wu, Hao Zhu, Yuhui Zhang, Xiaohan Wang, and Serena Yeung-Levy. 2026b. AutoMem: Automated Learning of Memory as a Cognitive Skill. (2026). arXiv:2607.01224

Tianhao Wu, Weizhe Yuan, Olga Golovneva, Jing Xu, Yuandong Tian, Jiantao Jiao, Jason Weston, and Sainbayar Sukhbaatar. 2024. Meta-Rewarding Language Models: Self-Improving Alignment with LLM-as-a-Meta-Judge. (2024). arXiv:2407.19594

Xiaobao Wu. 2025. Sailing by the Stars: A Survey on Reward Models and Learning Strategies for Learning from Rewards. (2025). arXiv:2505.02686

Yanru Wu, Weiduo Yuan, Ang Qi, Vitor Guizilini, Jiageng Mao, and Yue Wang. 2026a. Large Reward Models: Generalizable Online Robot Reward Generation with Vision-Language Models. (2026). arXiv:2603.16065

Evžen Wybitul, Evan Ryan Gunter, Mikhail Seleznyov, and David Lindner. 2024. ViSTa Dataset: Do vision-language models understand sequential tasks? (2024). arXiv:2411.13211

Jiajun Xi, Yinong He, Jianing Yang, Yinpei Dai, and Joyce Chai. 2024a. Teaching Embodied Reinforcement Learning Agents: Informativeness and Diversity of Language Use. (2024). arXiv:2410.24218

Zhiheng Xi, Jixuan Huang, Xin Guo, Boyang Hong, Dingwen Yang, Xiaoran Fan, Shuo Li, Zehui Chen, Junjie Ye, Siyu Yuan, Zhengyin Du, Xuesong Yao, Yufei Xu, Jiecao Chen, Rui Zheng, Tao Gui, Qi Zhang, and Xuanjing Huang. 2025. Critique-RL: Training Language Models for Critiquing through Two-Stage Reinforcement Learning. (2025). arXiv:2510.24320

Zhiheng Xi, Dingwen Yang, Jixuan Huang, Jiafu Tang, Guanyu Li, Yiwen Ding, Wei He, Boyang Hong, Shihan Do, Wenyu Zhan, Xiao Wang, Rui Zheng, Tao Ji, Xiaowei Shi, Yitao Zhai, Rongxiang Weng, Jingang Wang, Xunliang Cai, Tao Gui, Zuxuan Wu, Qi Zhang, Xipeng Qiu, Xuanjing Huang, and Yu-Gang Jiang. 2024b. Enhancing LLM Reasoning via Critique Models with Test-Time and Training-Time Supervision. (2024). arXiv:2411.16579

Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. 2026. SkillRL: Evolving Agents via Recursive Skill-Augmented Reinforcement Learning. (2026). arXiv:2602.08234

Jinyu Xiang, Jiayi Zhang, Zhaoyang Yu, Xinbing Liang, Fengwei Teng, Jinhao Tu, Fashen Ren, Xiangru Tang, Sirui Hong, Chenglin Wu, and Yuyu Luo. 2025. Self-Supervised Prompt Optimization. (2025). arXiv:2502.06855

Tim Z. Xiao, Robert Bamler, Bernhard Schölkopf, and Weiyang Liu. 2024a. Verbalized Machine Learning: Revisiting Machine Learning with Language Models. (2024). arXiv:2406.04344

Wenyi Xiao, Zechuan Wang, Leilei Gan, Shuai Zhao, Zongrui Li, Ruirui Lei, Wanggui He, Luu Anh Tuan, Long Chen, Hao Jiang, Zhou Zhao, and Fei Wu. 2024b. A Comprehensive Survey of Direct Preference Optimization: Datasets, Theories, Variants, and Applications. (2024). arXiv:2410.15595

Guofu Xie, Yunsheng Shi, Hongtao Tian, Ting Yao, and Xiao Zhang. 2025b. CAPO: Towards Enhancing LLM Reasoning through Generative Credit Assignment. (2025). arXiv:2508.02298

Tianbao Xie, Siheng Zhao, Chen Wu, Yitao Liu, Qian Luo, Victor Zhong, Yanchao Yang, and Tao Yu. 2024. Text2reward: Reward shaping with language models for reinforcement learning. In *International Conference on Learning Representations*, Vol. 2024. 35663–35699.

Yuxi Xie, Kenji Kawaguchi, Yiran Zhao, James Xu Zhao, Min-Yen Kan, Junxian He, and Michael Xie. 2023. Self-evaluation guided beam search for reasoning. *Advances in Neural Information Processing Systems* 36 (2023), 41618–41650.

Zhihui Xie, Jie Chen, Liyu Chen, Weichao Mao, Jingjing Xu, and Lingpeng Kong. 2025a. Teaching Language Models to Critique via Reinforcement Learning. (2025). arXiv:2502.03492

Wei Xiong, Wenting Zhao, Weizhe Yuan, Olga Golovneva, Tong Zhang, Jason Weston, and Sainbayar Sukhbaatar. 2025. StepWiser: Stepwise Generative Judges for Wiser Reasoning. (2025). arXiv:2508.19229

Fengli Xu, Qianyue Hao, Zefang Zong, Jingwei Wang, Yunke Zhang, Jingyi Wang, Xiaochong Lan, Jiahui Gong, Tianjian Ouyang, Fanjin Meng, Chenyang Shao, Yuwei Yan, Qinglong Yang, Yiwen Song, Sijian Ren, Xinyuan Hu, Yu Li, Jie Feng, Chen Gao, and Yong Li. 2025a. Towards Large Reasoning Models: A Survey of Reinforced Reasoning with Large Language Models. (2025). arXiv:2501.09686

Guowei Xu, Mert Yuksekgonul, Carlos Guestrin, and James Zou. 2025d. metaTextGrad: Automatically Optimizing Language Model Optimizers. (2025). arXiv:2505.18524

Tianze Xu, Yanzhao Zheng, Pengrui Lu, Lyumanshan Ye, Yong Wu, Zhentao Zhang, Yuanqiang Yu, Chao Ma, Jihuai Zhu, Pengfei Liu, Baohua Dong, Hangcheng Zhu, Ruohui Huang, and Gang Yu. 2026. Rubrics to Tokens: Bridging Response-level Rubrics and Token-level Rewards in Instruction Following Tasks. (2026). arXiv:2604.02795

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025b. A-MEM: Agentic Memory for LLM Agents. (2025). arXiv:2502.12110

Wanqiao Xu, Allen Nie, Ruijie Zheng, Aditya Modi, Adith Swaminathan, and Ching-An Cheng. 2025c. Formalizing Learning from Language Feedback with Provable Guarantees. (2025). arXiv:2506.10341

Sikuan Yan, Ahmed Bahloul, Ercong Nie, Susanna Schwarzmann, Riccardo Trivisonno, Volker Tresp, and Yunpu Ma. 2026. Memory-R2: Fair Credit Assignment for Long-Horizon Memory-Augmented LLM Agents. (2026). arXiv:2605.21768

Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Jinhe Bi, Kristian Kersting, Jeff Z. Pan, Hinrich Schütze, Volker Tresp, and Yunpu Ma. 2025b. Memory-R1: Enhancing Large Language Model Agents to Manage and Utilize Memories via Reinforcement Learning. (2025). arXiv:2508.19828

Yuchen Yan, Jin Jiang, Zhenbang Ren, Yijun Li, Xudong Cai, Yang Liu, Xin Xu, Mengdi Zhang, Jian Shao, Yongliang Shen, Jun Xiao, and Yueting Zhuang. 2025a. VerifyBench: Benchmarking Reference-based Reward Systems for Large Language Models. (2025). arXiv:2505.15801

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. 2024b. Large Language Models as Optimizers. In *International Conference on Learning Representations Workshop*. arXiv:2309.03409

Haoyan Yang, Mario Xerri, Solha Park, Huajian Zhang, Yiyang Feng, Sai Akhil Kogilathota, and Jiawei Zhou. 2026c. Self-Improvement of Large Language Models: A Technical Overview and Future Outlook. *arXiv preprint arXiv:2603.25681* (2026).

John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024a. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems* 37 (2024), 50528–50652.

Ruihan Yang, Fanghua Ye, Jian Li, Siyu Yuan, Yikai Zhang, Zhaopeng Tu, Xiaolong Li, and Deqing Yang. 2025b. The Lighthouse of Language: Enhancing LLM Agents via Critique-Guided Improvement. (2025). arXiv:2503.16024

Wenkai Yang, Jingwen Chen, Yankai Lin, and Ji-Rong Wen. 2025a. DeepCritic: Deliberate Critique with Large Language Models. (2025). arXiv:2505.00662

Xianglin Yang, Bryan Hooi, Gelei Deng, Tianwei Zhang, and Jin Song Dong. 2026a. Turning Bias into Bugs: Bandit-Guided Style Manipulation Attacks on LLM Judges. arXiv:2605.26156

Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, Bo Zhang, and Liang He. 2026b. AutoSkill: Experience-Driven Lifelong Learning via Skill Self-Evolution. (2026). arXiv:2603.01145

Yongjin Yang, Euiin Yi, Jongwoo Ko, Kimin Lee, Zhijing Jin, and Se-Young Yun. 2025c. Revisiting Multi-Agent Debate as Test-Time Scaling: A Systematic Study of Conditional Effectiveness. (2025). arXiv:2505.22960

Shunyu Yao, Rohan Rao, Matthew Hausknecht, and Karthik Narasimhan. 2020. Keep calm and explore: Language models for action generation in text-based games. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 8736–8754.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023a. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems* 36 (2023), 11809–11822.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023b. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*.

Jiayi Ye, Yanbo Wang, Yue Huang, Dongping Chen, Qihui Zhang, Nuno Moniz, Tian Gao, Werner Geyer, Chao Huang, Pin-Yu Chen, Nitesh V Chawla, and Xiangliang Zhang. 2024. Justice or Prejudice? Quantifying Biases in LLM-as-a-Judge. (2024). arXiv:2410.02736

Zhiling Ye, Yun Yue, Haowen Wang, Xudong Han, Jiadi Jiang, Cheng Wei, Lei Fan, Jiaxin Liang, Shuowen Zhang, Ji Li, Chunxiao Guo, Jian Wang, Peng Wei, and Jinjie Gu. 2025. Self-Rewarding Rubric-Based Reinforcement Learning for Open-Ended Reasoning. (2025). arXiv:2509.25534

John Seon Keun Yi, Aaron Mueller, and Dokyun Lee. 2026. Latent Agents: A Post-Training Procedure for Internalized Multi-Agent Debate. (2026). arXiv:2604.24881

Li Yin and Zhangyang Wang. 2025. LLM-AutoDiff: Auto-Differentiate Any LLM Workflow. (2025). arXiv:2501.16673

Zhangyue Yin, Qiushi Sun, Zhiyuan Zeng, Qinyuan Cheng, Xipeng Qiu, and Xuanjing Huang. 2025. Dynamic and Generalizable Process Reward Modeling. (2025). arXiv:2507.17849

Runyang You, Hongru Cai, Caiqi Zhang, Qiancheng Xu, Meng Liu, Tiezheng Yu, Yongqi Li, and Wenjie Li. 2026. Agent-as-a-Judge. (2026). arXiv:2601.05111

Wenhao Yu, Nimrod Gileadi, Chuyuan Fu, Sean Kirmani, Kuang-Huei Lee, Montse Gonzalez Arenas, Hao-Tien Lewis Chiang, Tom Erez, Leonard Hasenclever, and Jan Humplik. 2023. Language to rewards for robotic skill synthesis. *arXiv preprint arXiv:2306.08647* (2023).

Yue Yu, Zhengxing Chen, Aston Zhang, Liang Tan, Chenguang Zhu, Richard Yuanzhe Pang, Yundi Qian, Xuewei Wang, Suchin Gururangan, Chao Zhang, Melanie Kambadur, Dhruv Mahajan, and Rui Hou. 2024. Self-Generated Critiques Boost Reward Modeling for Language Models. (2024). arXiv:2411.16646

Siyu Yuan, Zehui Chen, Zhiheng Xi, Junjie Ye, Zhengyin Du, and Jiecao Chen. 2025a. Agent-R: Training Language Model Agents to Reflect via Iterative Self-Training. (2025). arXiv:2501.11425

Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. 2024. Self-rewarding language models. *arXiv preprint arXiv:2401.10020* (2024).

Youliang Yuan, Qiuyang Mang, Jingbang Chen, Hong Wan, Xiaoyuan Liu, Junjielong Xu, Jen-tse Huang, Wenxuan Wang, Wenxiang Jiao, and Pinjia He. 2025b. Curing Miracle Steps in LLM Mathematical Reasoning with Rubric Rewards. (2025). arXiv:2510.07774

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. 2024. TextGrad: Automatic "Differentiation" via Text. *arXiv preprint arXiv:2406.07496* (2024).

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D Goodman. 2022. STaR: Bootstrapping Reasoning With Reasoning. In *Advances in Neural Information Processing Systems*.

Runhao Zeng, Dingjie Zhou, Qiwei Liang, Junlin Liu, Hui Li, Changxin Huang, Jianqiang Li, Xiping Hu, and Fuchun Sun. 2024. Video2reward: Generating reward function from videos for legged robot behavior learning. *arXiv preprint arXiv:2412.05515* (2024).

Zhiyuan Zeng, Jiatong Yu, Tianyu Gao, Yu Meng, Tanya Goyal, and Danqi Chen. 2023. Evaluating Large Language Models at Evaluating Instruction Following. (2023). arXiv:2310.07641

Lihan Zha, Yuchen Cui, Li-Heng Lin, Minae Kwon, Montserrat Gonzalez Arenas, Andy Zeng, Fei Xia, and Dorsa Sadigh. 2023. Distilling and Retrieving Generalizable Knowledge for Robot Manipulation via Language Corrections. (2023). arXiv:2311.10678

Qiusi Zhan, Richard Fang, Henil Shalin Panchal, and Daniel Kang. 2025. Adaptive attacks break defenses against indirect prompt injection attacks on llm agents. In *Findings of the Association for Computational Linguistics: NAACL 2025*. 7101–7117.

Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*. 10471–10506.

Alexander Zhang, Marcus Dong, Jiaheng Liu, Wei Zhang, Yejie Wang, Jian Yang, Ge Zhang, Tianyu Liu, Zhongyuan Peng, Yingshui Tan, Yuanxing Zhang, Zhexu Wang, Weixun Wang, Yancheng He, Ken Deng, Wangchunshu Zhou, Wenhao Huang, and Zhaoxiang Zhang. 2025d. CodeCriticBench: A Holistic Code Critique Benchmark for Large Language Models. (2025). arXiv:2502.16614

Chenchen Zhang. 2026. From Reasoning to Agentic: Credit Assignment in Reinforcement Learning for Large Language Models. (2026). arXiv:2604.09459

Dylan Zhang, Yanshan Lin, Zhengkun Wu, Yihang Sun, Bingxuan Li, Dianqi Li, and Hao Peng. 2026b. Useful Memories Become Faulty When Continuously Updated by LLMs. (2026). arXiv:2605.12978

Guibin Zhang, Muxin Fu, Guancheng Wan, Miao Yu, Kun Wang, and Shuicheng Yan. 2025e. G-Memory: Tracing Hierarchical Memory for Multi-Agent Systems. (2025). arXiv:2506.07398

Guibin Zhang, Hejia Geng, Xiaohang Yu, Zhenfei Yin, Zaibin Zhang, Zelin Tan, Heng Zhou, Zhongzhi Li, Xiangyuan Xue, Yijiang Li, Yifan Zhou, Yang Chen, Chen Zhang, Yutao Fan, Zihu Wang, Songtao Huang, Francisco Piedrahita-Velez, Yue Liao, Hongru Wang, Mengyue Yang, Heng Ji, Jun Wang, Shuicheng Yan, Philip Torr, and Lei Bai. 2025f. The Landscape of Agentic Reinforcement Learning for LLMs: A Survey. (2025). arXiv:2509.02547

Guibin Zhang, Haotian Ren, Chong Zhan, Zhenhong Zhou, Junhao Wang, He Zhu, Wangchunshu Zhou, and Shuicheng Yan. 2025j. MemEvolve: Meta-Evolution of Agent Memory Systems. (2025). arXiv:2512.18746

Hangfan Zhang, Zhiyao Cui, Jianhao Chen, Xinrun Wang, Qiaosheng Zhang, Zhen Wang, Dinghao Wu, and Shuyue Hu. 2025a. Stop Overvaluing Multi-Agent Debate – We Must Rethink Evaluation and Embrace Model Heterogeneity. (2025). arXiv:2502.08788

Haozhen Zhang, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. 2026c. MemSkill: Learning and Evolving Memory Skills for Self-Evolving Agents. (2026). arXiv:2602.02474

Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. 2024b. AFlow: Automating Agentic Workflow Generation. (2024). arXiv:2410.10762

Kaiyan Zhang, Yuxin Zuo, Bingxiang He, Youbang Sun, Runze Liu, Che Jiang, Yuchen Fan, Kai Tian, Guoli Jia, Pengfei Li, Yu Fu, Xingtai Lv, Yuchen Zhang, Sihang Zeng, Shang Qu, Haozhan Li, Shijie Wang, Yuru Wang, Xinwei Long, Fangfu Liu, Xiang Xu, Jiaze Ma, Xuekai Zhu, Ermo Hua, Yihao Liu, Zonglin Li, Huayu Chen, Xiaoye Qu, Yafu Li, Weize Chen, Zhenzhao Yuan, Junqi Gao, Dong Li, Zhiyuan Ma, Ganqu Cui, Zhiyuan Liu, Biqing Qi, Ning Ding, and Bowen Zhou. 2025m. A Survey of Reinforcement Learning for Large Reasoning Models. (2025). arXiv:2509.08827

Lunjun Zhang, Ryan Chen, and Bradly C. Stadie. 2026a. Evolutionary System Prompt Learning for Reinforcement Learning in LLMs. (2026). arXiv:2602.14697

Lunjun Zhang, Arian Hosseini, Hritik Bansal, Mehran Kazemi, Aviral Kumar, and Rishabh Agarwal. 2025g. Generative Verifiers: Reward Modeling as Next-Token Prediction. In *International Conference on Learning Representations*. arXiv:2408.15240

Peiyan Zhang, Haibo Jin, Leyang Hu, Xinnuo Li, Liying Kang, Man Luo, Yangqiu Song, and Haohan Wang. 2024a. REVOLVE: Optimizing AI Systems by Tracking Response Evolution in Textual Optimization. (2024). arXiv:2412.03092

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. 2025h. Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models. (2025). arXiv:2510.04618

Qiyuan Zhang, Fuyuan Lyu, Zexu Sun, Lei Wang, Weixu Zhang, Wenyue Hua, Haolun Wu, Zhihan Guo, Yufei Wang, Niklas Muennighoff, Irwin King, Xue Liu, and Chen Ma. 2025i. A Survey on Test-Time Scaling in Large Language Models: What, How, Where, and How Well? (2025). arXiv:2503.24235

Xiaohan Zhang, Yan Ding, Yohei Hayamizu, Zainab Altaweel, Yifeng Zhu, Yuke Zhu, Peter Stone, Chris Paxton, and Shiqi Zhang. 2025c. LLM-GROP: Visually grounded robot task and motion planning with large language models. *The International Journal of Robotics Research* (2025), 02783649251378196.

Yaolun Zhang, Yiran Wu, Yijiong Yu, Qingyun Wu, and Huazheng Wang. 2026e. Live-Evo: Online Evolution of Agentic Memory from Continuous Feedback. (2026). arXiv:2602.02369

Yang Zhang, Shixin Yang, Chenjia Bai, Fei Wu, Xiu Li, Zhen Wang, and Xuelong Li. 2025k. Towards efficient llm grounding for embodied multi-agent collaboration. In *Findings of the Association for Computational Linguistics: ACL 2025*. 1663–1699.

Zeyu Zhang, Quanyu Dai, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. 2025b. A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems* 43, 6 (2025), 1–47.

Zhang Zhang, Shuqi Lu, Hongjin Qian, Di He, and Zheng Liu. 2026d. AgentFactory: A Self-Evolving Framework Through Executable Subagent Accumulation and Reuse. (2026). arXiv:2603.18000

Zhenru Zhang, Chujie Zheng, Yangzhen Wu, Beichen Zhang, Runji Lin, Bowen Yu, Dayiheng Liu, Jingren Zhou, and Junyang Lin. 2025l. The Lessons of Developing Process Reward Models in Mathematical Reasoning. (2025). arXiv:2501.07301

Andrew Zhao, Reshmi Ghosh, Vitor Carvalho, Emily Lawton, Keegan Hines, Gao Huang, and Jack W. Stokes. 2025a. Are My Optimized Prompts Compromised? Exploring Vulnerabilities of LLM-based Optimizers. arXiv:2510.14381

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM Agents Are Experiential Learners. In *AAAI Conference on Artificial Intelligence*.

Jian Zhao, Runze Liu, Kaiyan Zhang, Zhimu Zhou, Junqi Gao, Dong Li, Jiafei Lyu, Zhouyi Qian, Biqing Qi, and Xiu Li. 2026. Genprm: Scaling test-time compute of process reward models via generative reasoning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 34932–34940.

Yulai Zhao, Haolin Liu, Dian Yu, Sunyuan Kung, Meijia Chen, Haitao Mi, and Dong Yu. 2025b. One Token to Fool LLM-as-a-Judge. arXiv:2507.08794

Chujie Zheng, Zhenru Zhang, Beichen Zhang, Runji Lin, Keming Lu, Bowen Yu, Dayiheng Liu, Jingren Zhou, and Junyang Lin. 2024. ProcessBench: Identifying Process Errors in Mathematical Reasoning. (2024). arXiv:2412.06559

Junhao Zheng, Chengming Shi, Xidi Cai, Qiuke Li, Duzhen Zhang, Chenxing Li, Dong Yu, and Qianli Ma. 2025. Lifelong Learning of Large Language Model based Agents: A Roadmap. (2025). arXiv:2501.07278

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, and Eric Xing. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems* 36 (2023), 46595–46623.

Victor Zhong, Dipendra Misra, Xingdi Yuan, and Marc-Alexandre Côté. 2024. Policy Improvement using Language Feedback Models. (2024). arXiv:2402.07876

Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. 2024b. Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models. In *International Conference on Machine Learning*.

Enshen Zhou, Qi Su, Cheng Chi, Zhizheng Zhang, Zhongyuan Wang, Tiejun Huang, Lu Sheng, and He Wang. 2025c. Code-as-monitor: Constraint-aware visual programming for reactive and proactive robotic failure detection. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 6919–6929.

Enyu Zhou, Guodong Zheng, Binghai Wang, Zhiheng Xi, Shihan Dou, Rong Bao, Wei Shen, Limao Xiong, Jessica Fan, Yurong Mou, Rui Zheng, Tao Gui, Qi Zhang, and Xuanjing Huang. 2024e. RMB: Comprehensively Benchmarking Reward Models in LLM Alignment. (2024). arXiv:2410.09893

Huichi Zhou, Yihang Chen, Siyuan Guo, Xue Yan, Kin Hei Lee, Zihan Wang, Ka Yiu Lee, Guchun Zhang, Kun Shao, Linyi Yang, and Jun Wang. 2025a. Memento: Fine-tuning LLM Agents without Fine-tuning LLMs. (2025). arXiv:2508.16153

Huichi Zhou, Siyuan Guo, Anjie Liu, Zhongwei Yu, Ziqin Gong, Bowen Zhao, Zhixun Chen, Menglong Zhang, Yihang Chen, Jinsong Li, Runyu Yang, Qiangbin Liu, Xinlei Yu, Jianmin Zhou, Na Wang, Chunyang Sun, and Jun Wang. 2026. Memento-Skills: Let Agents Design Agents. (2026). arXiv:2603.18743

Han Zhou, Xingchen Wan, Ruoxi Sun, Hamid Palangi, Shariq Iqbal, Ivan Vulić, Anna Korhonen, and Sercan Ö. Arık. 2025d. Multi-Agent Design: Optimizing Agents with Better Prompts and Topologies. (2025). arXiv:2502.02533

Meng Zhou, Bei Li, Jiahao Liu, Xiaowen Shi, Yang Bai, Rongxiang Weng, Jingang Wang, and Xunliang Cai. 2025b. Libra: Assessing and Improving Reward Model by Learning to Think. (2025). arXiv:2507.21645

Wangchunshu Zhou, Yixin Ou, Shengwei Ding, Long Li, Jialong Wu, Tiannan Wang, Jiamin Chen, Shuai Wang, Xiaohua Xu, Ningyu Zhang, Huajun Chen, and Yuchen Eleanor Jiang. 2024a. Symbolic Learning Enables Self-Evolving Agents. (2024). arXiv:2406.18532

Wenxuan Zhou, Shujian Zhang, Lingxiao Zhao, and Tao Meng. 2024d. T-REG: Preference Optimization with Token-Level Reward Regularization. (2024). arXiv:2412.02685

Yan Zhou and Yanguang Chen. 2025. Adaptive heterogeneous multi-agent debate for enhanced educational and factual reasoning in large language models. *Journal of King Saud University Computer and Information Sciences* 37, 10 (2025), 330.

Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. 2023. Large Language Models Are Human-Level Prompt Engineers. In *International Conference on Learning Representations*. arXiv:2211.01910

Yuxuan Zhou, Yubin Wang, Bin Wang, Chen Ning, Xien Liu, Ji Wu, and Jianye Hao. 2025e. Enhancing the Medical Context-Awareness Ability of LLMs via Multifaceted Self-Refinement Learning. *arXiv preprint arXiv:2511.10067* (2025).

Yilun Zhou, Austin Xu, Peifeng Wang, Caiming Xiong, and Shafiq Joty. 2025f. Evaluating Judges as Evaluators: The JETTS Benchmark of LLM-as-Judges as Test-Time Scaling Evaluators. (2025). arXiv:2504.15253

Yifei Zhou, Andrea Zanette, Jiayi Pan, Sergey Levine, and Aviral Kumar. 2024c. ArCHer: Training Language Model Agents via Hierarchical Multi-Turn RL. (2024). arXiv:2402.19446

Adam Zweiger, Jyothish Pari, Han Guo, Ekin Akyürek, Yoon Kim, and Pulkit Agrawal. 2025. Self-Adapting Language Models. (2025). arXiv:2506.10943

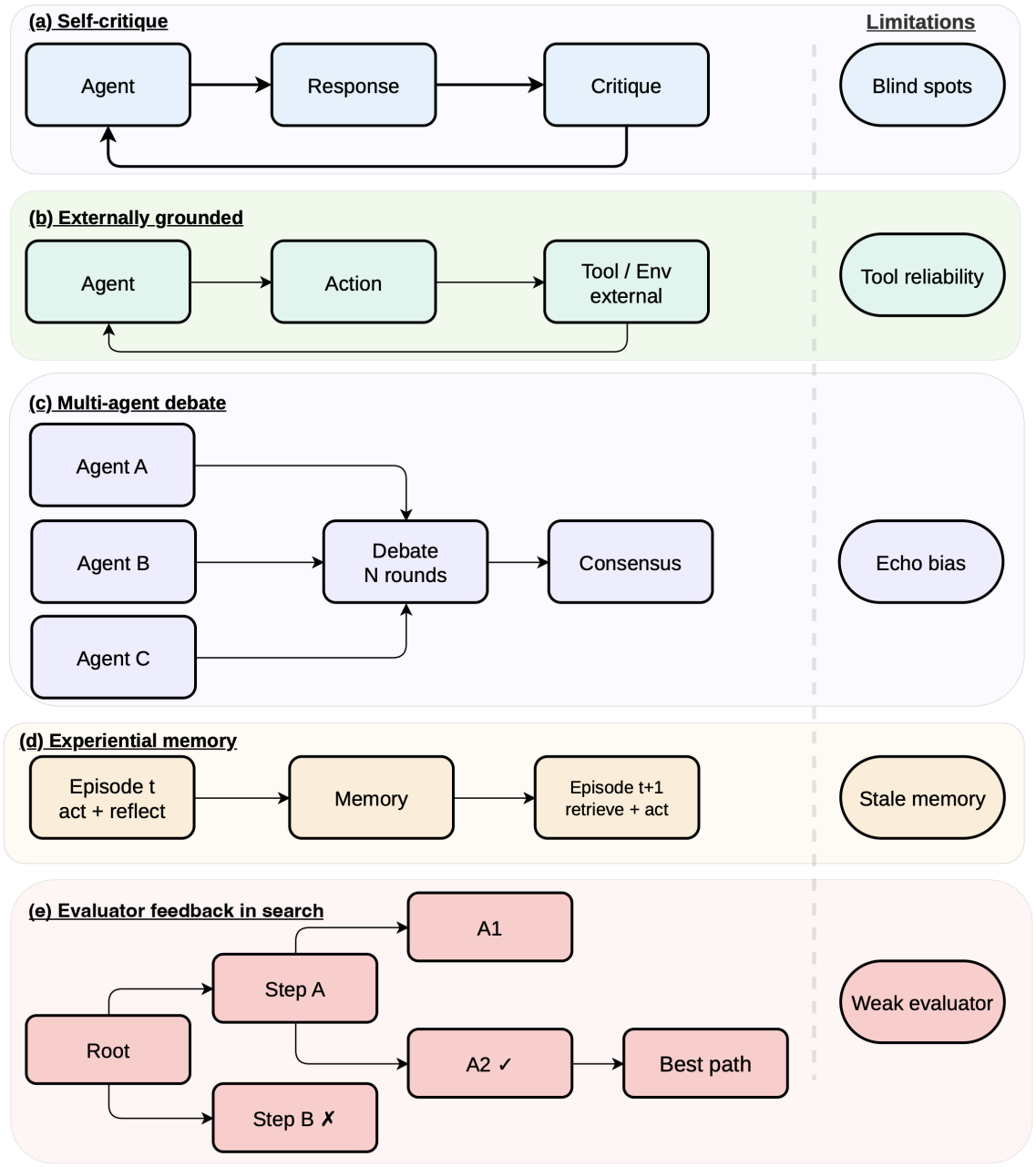

Fig. 4. Inference-time verbal feedback mechanisms. Each panel shows a distinct feedback-loop structure: (a) self-critique loops within a single model, (b) externally grounded critique incorporates verified tool or environment output, (c) multi-agent debate distributes critique across parallel model instances and converges to a refined output, (d) experiential memory persists lessons across episodes, and (e) evaluator feedback in search explores and prunes multiple candidate paths. Panels (a) and (b) together form the single critique cell of §5.1.

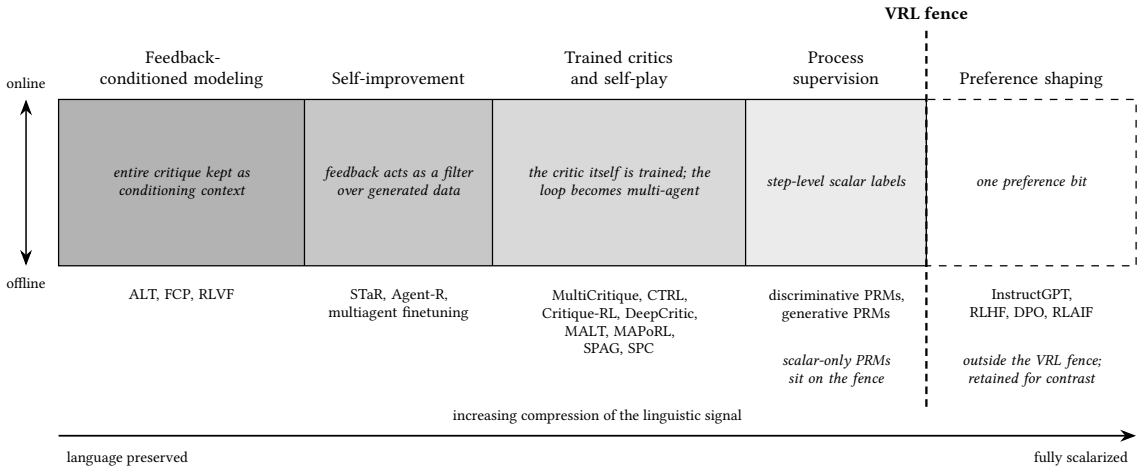

Fig. 5. Training-time methods ordered by how much linguistic structure survives into the parameter update, from critiques kept intact as conditioning context down to a single preference bit. The dashed fence marks the outer boundary of VRL, with vanilla RLHF and DPO retained beyond it only as the contrastive endpoint. The fence runs along the right edge of the process-supervision band: discriminative PRMs that keep only click-level step labels sit on the boundary itself, and the band stays inside on the strength of generative PRMs, which consume language inside the reward computation (§6.5). The online versus offline axis cuts across every band.

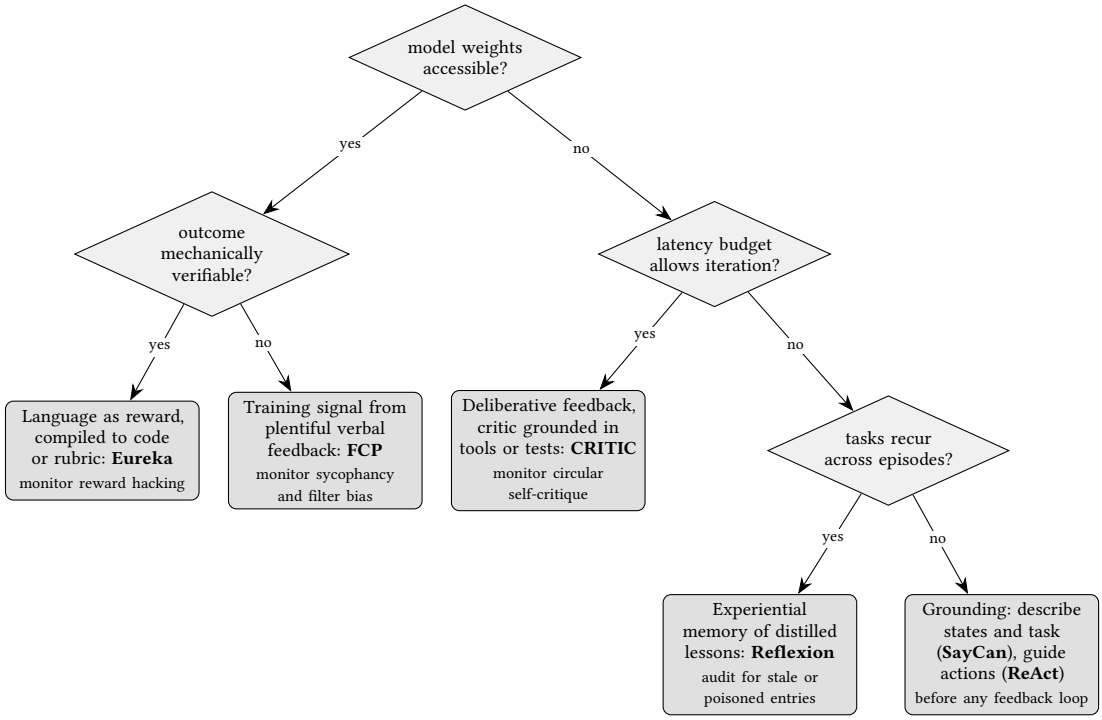

Fig. 6. Choosing a verbal-feedback mechanism: the four ordered questions of §13.1 arranged as a decision tree. Each leaf names a row of Table 7 with one representative method and inherits that row's full preconditions, which the tree does not re-check; the final leaf covers the two grounding roles.